

TECHNICAL REPORT OF KIMI K3

KIMI K3 技术报告

Kimi Team

Kimi Team

ABSTRACT

摘要

We introduce Kimi K3, a 2.8T parameter Mixture-of-Experts model with 104 billion activated parameters, native vision capabilities, and a 1-million-token context window. Kimi K3 is built on Kimi Delta Attention [63] and Attention Residuals [57], which improve information flow across sequence length and model depth. Together with Stable LatentMoE, which effectively activates 16 of 896 routed experts per token, and refined training and data recipes, these advances yield an approximately $2.5\times$ improvement in overall scaling efficiency over Kimi K2 [58]. Post-training highlights reinforcement learning across general, agentic, and coding domains and multiple reasoning-effort levels, enabling compositional generalization and robust long-horizon execution. At 2.8T scale, Kimi K3 is supported by infrastructure advances in multiple areas: algorithm-system co-design for KDA, perfectly balanced expert-parallel training with efficient memory management, million-token agentic RL with persistent rollout and sandbox states, and deployment innovations.

我们介绍 Kimi K3，这是一个具有 1040 亿激活参数的 2.8 万亿参数 Mixture-of-Experts 模型，具备原生视觉能力，并拥有 100 万令牌的上下文窗口。Kimi K3 基于 Kimi Delta Attention [63] 和 Attention Residuals [57] 构建，这些技术改进了序列长度和模型深度之间的信息流动。结合 Stable LatentMoE，该技术能有效激活每个令牌的 896 个路由专家中的 16 个，以及经过优化的训练和数据配方，这些进步使得 Kimi K3 在整体扩展效率上相比 Kimi K2 [58] 提升了大约 $2.5\times$ 。在微调阶段，强化学习覆盖了通用、自主代理和编码领域，并支持多种推理努力水平，从而实现了组合泛化和稳健的长期执行能力。在 2.8 万亿规模下，Kimi K3 得到了多个领域的基础设施进步支持：KDA 的算法-系统协同设计、高效内存管理下的专家并行训练完美平衡、百万令牌自主代理强化学习（带持久化展开和沙盒状态），以及部署创新。

Extensive evaluations show that Kimi K3 achieves frontier-level performance across long-horizon coding, agentic, knowledge, reasoning, and vision tasks. While its overall performance still trails the most powerful proprietary models, namely Claude Fable 5 and GPT-5.6 Sol, Kimi K3 consistently outperforms other open and proprietary models evaluated in our suite. We release the full Kimi K3 model weights to facilitate future research and accelerate the broader deployment and adoption of frontier intelligence.¹

大量评估表明，Kimi K3 在长视野编码、自主性、知识、推理和视觉任务中均达到了前沿水平的性能。尽管其整体性能仍落后于最强大的专有模型，即 Claude Fable 5 和 GPT-5.6 Sol，但 Kimi K3 在我们测试套件中评估的其他开源和专有模型中始终表现优异。我们发布完整的 Kimi K3 模型权重，以促进未来的研究，并加速前沿智能的广泛应用和采用。¹

Coding All maxed out on thinking effort: max or xhigh.

编码全部满载于思考努力：最大或极高。

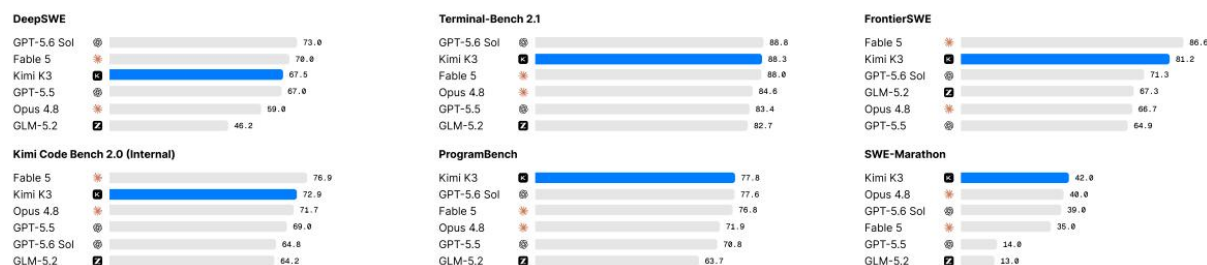
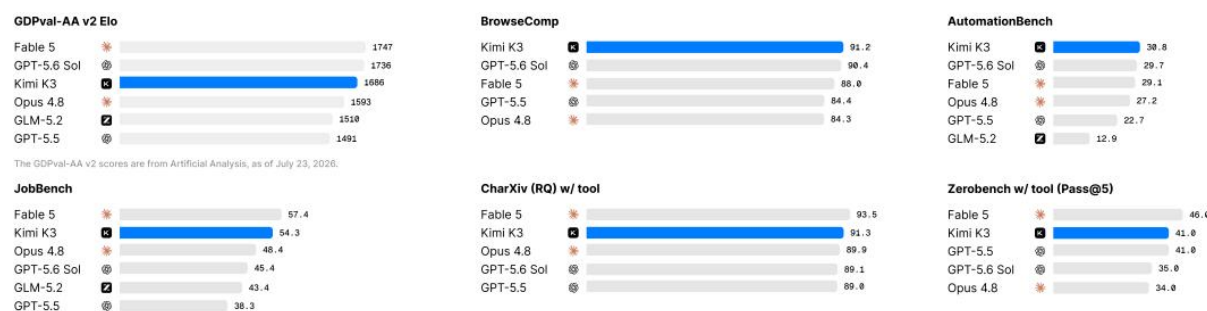


Figure 1: images/a4e3304ed55d7979ea634af9511258544b4ce6e66395e041ec3c90d5d0fa31c4.jpg

General & Visual Agents All maxed out on thinking effort: max or xhigh.

通用与视觉代理均已完全耗尽思考精力：最大或极高。



Note: All Fable 5 results are with potential fallbacks. All GPT-5.6 Sol results include potential cyberguards.

Figure 1: Kimi K3 main results.

注意：所有 Fable 5 的结果均包含潜在的回退方案。所有 GPT-5.6 Sol 的结果均包含潜在的网络防护。

图 1: Kimi K3 主要结果。

1 Introduction

1 引言

For much of the development of Large Language Models (LLMs), scaling meant investing more computation before deployment by training larger models on more data [54, 45]. The rise of reasoning models has established test-time computation as a second axis of scaling: OpenAI's o-series scales reinforcement learning and test-time reasoning [84, 83]; Anthropic's extended-thinking models allocate adaptive thinking budgets and interleave reasoning with tool use [6, 7]; DeepSeek-R1 [40] and Kimi K1.5 [118] show that large-scale reinforcement learning can elicit sophisticated reasoning behaviors from strong pre-trained models; and Kimi K2.5 Agent Swarm [59] further extends test-time scaling from sequential reasoning to parallel agent coordination. These advances have made test-time scaling a central focus of frontier research. However, while the open-source model ecosystem has advanced rapidly on the second axis, it has progressed slowly on the first: many recent models remain within or slightly above the 1T-class parameter regime [145, 29, 135, 120]. As increasingly sophisticated reasoning and agentic reinforcement learning methods are applied to pre-trained foundations of similar scale, open-source progress risks converging while the gap to the strongest proprietary systems widens. With Kimi K3, we pursue both scaling axes together to the frontier: scaling the pre-trained foundation to unprecedented 3T-class parameters while scaling reinforcement learning, reasoning effort, and long-horizon interaction at 1M context length.

在大型语言模型 (LLMs) 发展的大部分时间里, 扩展意味着在部署前投入更多的计算资源, 通过在更多数据上训练更大的模型 [54, 45]。推理模型的兴起确立了测试时计算作为扩展的第二个轴: OpenAI 的 o 系列模型扩展了强化学习和测试时推理 [84, 83]; Anthropic 的扩展思维模型分配自适应的思维预算, 并将推理与工具使用交错进行 [6, 7]; DeepSeek-R1 [40] 和 Kimi K1.5 [118] 表明, 大规模强化学习可以从强大的预训练模型中引出复杂的推理行为; 而 Kimi K2.5 代理群 [59] 进一步将测试时扩展从顺序推理扩展到并行代理协调。这些进展使测试时扩展成为前沿研究的核心焦点。然而, 尽管开源模型生态系统在第二个轴上取得了快速进展, 但在第一个轴上进展缓慢: 许多最近的模型仍停留在或略高于 1T 级参数规模 [145, 29, 135, 120]。随着越来越复杂的推理和代理强化学习方法应用于相似规模的预训练基础模型, 开源进展面临收敛的风险, 而与最强的专有系统之间的差距却在扩大。借助 Kimi K3, 我们同时追求两个扩展轴直至前沿: 将预训练基础扩展到前所未有的 3T 级参数, 同时在 1M 上下文长度下扩展强化学习、推理努力和长时交互。

We introduce Kimi K3, a native multimodal Mixture-of-Experts model with 2.8 trillion total parameters, 104 billion activated parameters, and a context window of up to one million tokens. Its architecture scales information flow across sequence length, network depth, and model width. Kimi Delta Attention (KDA) [63] provides efficient long-sequence mixing, with periodically interleaved Gated MLA layers preserving global interaction. Attention Residuals (AttnRes) [57] allows each layer to selectively attend to representations from all preceding layers. Stable LatentMoE expands the routed expert space to 896 experts, with 16 activated per token, while normalization, SiTU-GLU, and Quantile Balancing stabilize optimization at extreme sparsity. These architectural advances, combined with refined data and training recipes, yield an approximately $2.5\times$ improvement in overall scaling efficiency over Kimi K2 [58].

我们引入了 Kimi K3, 这是一个原生的多模态 Mixture-of-Experts 模型, 总参数量为 2.8 万亿, 激活参数量为 1040 亿, 上下文窗口可达一百万 token。其架构在序列长度、网络深度和模型宽度上扩展了信息流。Kimi Delta Attention (KDA) [63] 提供了高效的长序列混合, 通过周期性交错的 Gated MLA 层保持全局交互。Attention Residuals (AttnRes) [57] 允许每一层选择性地关注所有先前层的表示。Stable LatentMoE 将路由专家空间扩展到 896 个专家, 每个 token 激活 16 个专家, 同时通过归一化、SiTU-GLU 和 Quantile Balancing 在极端稀疏性下稳定优化。这些架构改进结合精细的数据和训练方案, 使得整体扩展效率相比 Kimi K2 [58] 提高了大约 $2.5\times$ 。

We pair this pre-training foundation with post-training designed explicitly for 1M context test-time scaling. Kimi K3 undergoes reinforcement learning across long-horizon coding, general agents, general reasoning and knowledge tasks, each spanning multiple reasoning-effort levels. Training environments include verifiable search and professional knowledge work, software engineering and kernel optimization, multimodal reasoning with vision-in-the-loop tool use, persistent assistant workflows, web development, and autonomous execution tasks. These environments train a general loop of reasoning, acting, observing, verifying, and adapting, often over hundreds or thousands of tool calls and millions of accumulated context tokens. Domain- and effort-specialized policies are consolidated into a unified model through multi-teacher on-policy distillation [75, 134, 29].

我们将这种预训练基础与专门设计用于 1M 上下文测试时间扩展的后训练相结合。Kimi K3 通过长视野编码、通用代理、通用推理和知识任务进行强化学习, 每个任务涵盖多个推理努力级别。训练环境包括可验证的搜索和专业工作、软件工程和内核优化、结合视觉反馈的多模态推理工具使用、持久助理工作流程、网页开发和自主执行任务。这些环境训练了一个通用的推理、行动、观察、验证和适应循环, 通常涉及数百或数千次工具调用和数百万个累积上下文标记。通过多教师在线蒸馏, 将领域和努力专门化的策略整合到一个统一模型中 [75, 134, 29]。

Realizing this regime requires infrastructure that scales with architecture complexity, model size, and trajectory length. For systems co-design for KDA, we develop fused kernels, KDA Context Parallelism, and state-aware prefix caching to make KDA efficient within devices, across devices, and across requests. For 2.8T-parameter MoE pre-training, MoonEP provides perfectly balanced expert execution with static computation shapes and zero-copy communication, while memory efficient training and multimodal encoder optimizations sustain utilization within bounded memory. For million-token agentic RL, our co-located system combines partial rollouts, external KV-cache retention, adaptive throttling and resumable microVM sandboxes to preserve long-lived model and environment state. Finally, specialized kernels, and cache- and budget-aware fleet

scheduling translate these innovations into predictable production serving.

实现这一架构需要与架构复杂度、模型规模和轨迹长度相匹配的基础设施。对于 KDA 的系统协同设计，我们开发了融合内核、KDA 上下文并行性以及状态感知的前缀缓存，以在设备内部、跨设备以及跨请求中提高 KDA 的效率。对于 2.8 万亿参数的 MoE 预训练，MoonEP 通过静态计算形状和零拷贝通信提供完全平衡的专家执行，同时通过内存高效的训练和多模态编码器优化，在受限内存中持续保持高利用率。对于百万标记的代理强化学习，我们的协同系统结合了部分轨迹回放、外部 KV 缓存保留、自适应限流和可恢复的微虚拟机沙盒，以保持长期的模型和环境状态。最后，专用内核以及缓存和预算感知的舰队调度将这些创新转化为可预测的生产服务。

The resulting model establishes a new open frontier. On benchmarks spanning long-horizon coding, agentic, knowledge, reasoning, and vision tasks, Kimi K3 trails the strongest proprietary systems overall—Claude Fable 5 and GPT-5.6 Sol—and is consistently ahead of the other open and proprietary models evaluated in our suite, as shown in Fig. 1.

所得到的模型建立了一个新的开放前沿。在涵盖长期编码、自主性、知识、推理和视觉任务的基准测试中，Kimi K3 在整体表现上落后于最强的专有系统——Claude Fable 5 和 GPT-5.6 Sol。如图 1 所示，Kimi K3 在我们测试套件中评估的其他开放和专有模型中始终表现领先。

Our contributions are summarized as follows:

我们的贡献总结如下：

- Pre-training at the open frontier. We train a 2.8T-parameter native multimodal MoE model with 104B activated parameters and a 1M-token context window. KDA, AttnRes, Stable LatentMoE, refined data and training recipes collectively improve overall scaling efficiency by approximately $2.5\times$ over Kimi K2.
- Reinforcement learning for multi-effort test-time scaling. We conduct RL across general, agentic, and coding domains and multiple reasoning-effort levels, then consolidate the resulting capabilities into a unified model.
- Infrastructure for multi-trillion-parameter, million-token intelligence. We introduce KDA systems co-designs; MoonEP and memory-efficient infrastructure for 2.8T-parameter MoE pre-training; a co-located RL system with resumable sandboxes for million-token agentic trajectories; and more infrastructure innovations.
- An open frontier model. We release the full Kimi K3 model weights, making frontier intelligence available for research, deployment, and further innovation.
- 在开放前沿进行预训练。我们训练了一个拥有 2.8 万亿参数的原生多模态 MoE 模型，激活参数达到 1040 亿，上下文窗口为 100 万 token。KDA、AttnRes、Stable LatentMoE、优化的数据和训练方案共同将整体扩展效率提升了约 $2.5\times$ ，超越了 Kimi K2。
- 多努力测试时扩展的强化学习。我们在通用、代理和编码领域以及多个推理努力层级上进行强化学习，然后将获得的能力整合到一个统一的模型中。
- 多万亿参数、百万 token 智能的基础设施。我们引入了 KDA 系统协同设计；用于 2.8 万亿参数 MoE 预训练的 MoonEP 和内存高效基础设施；用于百万 token 代理轨迹的协同定位强化学习系统，具备可恢复的沙盒；以及更多基础设施创新。
- 一个开放前沿模型。我们发布了完整的 Kimi K3 模型权重，使前沿智能可用于研究、部署和进一步创新。

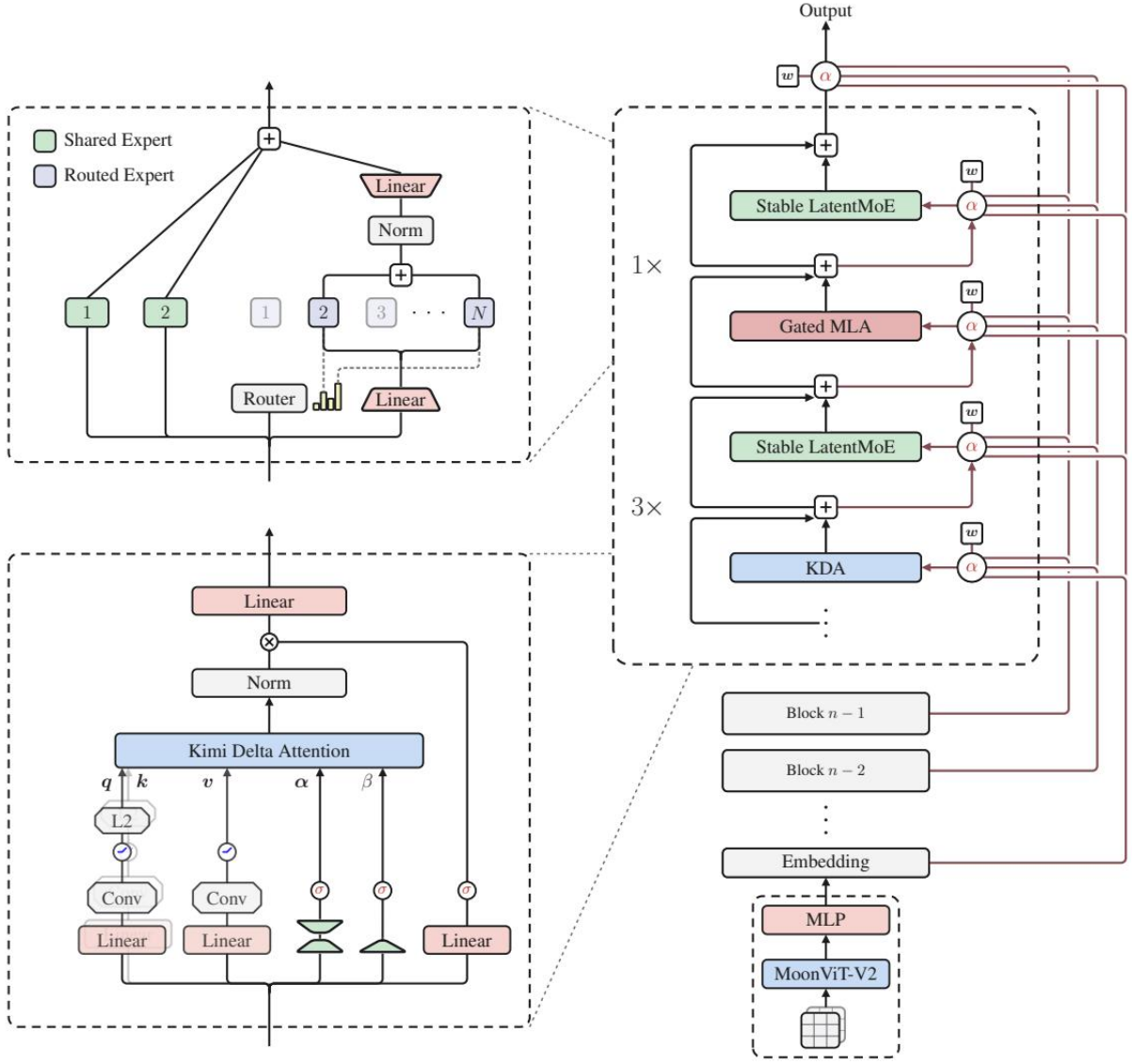


Figure 2: images/8aca94606516d3a31cab8f5e55c7b8572f6a9261b89cd6c191069fb3ef961a51.jpg

Figure 2: The Kimi K3 architecture, organized around token, channel, and layer mixing, with a native vision pathway at the input. Each block contains three Kimi Delta Attention (KDA) layers followed by one Gated MLA layer, with each attention layer paired with a Stable LatentMoE feed-forward network. Attention Residuals (AttnRes) use learned pseudo-queries (w) to derive attention weights (α) over the embedding and preceding block outputs, enabling selective information flow across depth. Top left: the Stable LatentMoE module with shared and routed experts. Bottom left: the KDA module. Bottom right: the native vision pathway.

图 2: Kimi K3 架构, 围绕 token、channel 和 layer 混合组织, 输入端具有原生的视觉路径。每个块包含三个 Kimi Delta Attention (KDA) 层, 后接一个 Gated MLA 层, 每个注意力层均与一个 Stable LatentMoE 前馈网络配对。注意力残差 (AttnRes) 使用学习到的伪查询 (w) 来推导注意力权重 (α), 从而在深度方向上实现选择性的信息流动。左上: Stable LatentMoE 模块, 包含共享和路由专家。左下: KDA 模块。右下: 原生的视觉路径。

2 Model Architecture

2 模型架构

The Kimi K3 architecture is designed to scale information flow along three complementary dimensions: sequence length, network depth, and model width. Along the sequence dimension, Hybrid Attention combines three Kimi Delta Attention (KDA) [63] layers with one Gated MLA layer in each block, providing an efficient mechanism for long-context token mixing while retaining selective high-capacity attention (§2.1). Along the depth dimension, Attention Residuals (AttnRes) [57] enable each module to selectively retrieve representations from the embedding, the current block, and preceding blocks, extending information access beyond conventional sequential residual accumulation (§2.2). Along the width dimension, each attention layer is followed by a Stable LatentMoE layer that performs sparse channel mixing, effectively activating 16 of 896 routed experts for each token (§2.3). For native vision, MoonViT-V2 encodes images and videos, and a lightweight projector maps the resulting visual features into the shared embedding space before backbone processing (§2.4). Together with Per-Head Muon (§2.5), these components provide a unified architecture for scaling information flow across tokens, layers, and channels. Combined with refined training and data recipes, they yield an approximately $2.5\times$ improvement in overall scaling efficiency over Kimi K2. Figure 2 provides an overview of the architecture.

Kimi K3 架构设计沿三个互补的维度扩展信息流：序列长度、网络深度和模型宽度。在序列维度上，混合注意力（Hybrid Attention）在每个块中结合了三个 Kimi Delta 注意力（KDA）[63] 层和一个门控 MLA 层，提供了一种高效的机制，用于长上下文标记混合，同时保留选择性高容量注意力 (§2.1)。在网络深度维度上，注意力残差（Attention Residuals, AttnRes）[57] 使每个模块能够有选择地从嵌入、当前块和先前块中检索表示，扩展了信息访问范围，超越了传统的顺序残差累积 (§2.2)。在网络宽度维度上，每个注意力层后接一个稳定的潜在 MoE 层，执行稀疏通道混合，有效激活每个标记的 16 个 896 个路由专家 (§2.3)。对于原生视觉，MoonViT-V2 编码图像和视频，并通过一个轻量级投影器将生成的视觉特征映射到共享嵌入空间，再进行主干处理 (§2.4)。与每头 Muon (§2.5) 相结合，这些组件提供了一种统一的架构，用于在标记、层和通道之间扩展信息流。结合精细的训练和数据配方，它们在整体扩展效率上相比 Kimi K2 提高了大约 $2.5\times$ 。图 2 提供了该架构的概览。

2.1 Hybrid Attention

2.1 混合注意力

Kimi K3 uses a layerwise hybrid of linear and global attention, combining KDA [63] with Gated MLA. Each block contains 3 KDA layers followed by 1 Gated MLA layer, giving a 3:1 mixing ratio. This pattern is repeated throughout the backbone. The two attention mechanisms are described separately below. An additional Gated MLA layer is placed at the end of the backbone, ensuring that the final layer always performs global attention.

Kimi K3 使用线性注意力和全局注意力的分层混合机制，结合 KDA [63] 与 Gated MLA。每个模块包含 3 个 KDA 层和 1 个 Gated MLA 层，比例为 3:1。这种模式在整个主干网络中重复使用。下面分别描述这两种注意力机制。在主干网络的末尾额外添加一个 Gated MLA 层，确保最后一个层始终执行全局注意力。

2.1.1 Kimi Delta Attention

2.1.1 Kimi Delta Attention

KDA extends the delta-rule recurrence [105, 138] with a channel-wise forget gate [63]. Consider a sequence of hidden states $x_t \in \mathbb{R}^d$, where t indexes the token position and d is the model hidden dimension. For clarity, we first describe a single attention head, with query and key vectors

$q_t, k_t \in \mathbb{R}^{d_k}$, value vector $v_t \in \mathbb{R}^{d_v}$, and recurrent state $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$. KDA applies channel-wise decay before the delta-rule update:

KDA 在 delta-rule 递归 [105, 138] 的基础上引入了一个通道级的遗忘门 [63]。考虑一个隐藏状态序列 $x_t \in \mathbb{R}^d$ ，其中 t 表示标记的位置， d 是模型的隐藏维度。为了清晰起见，我们首先描述一个单个注意力头，其中查询向量和键向量为 $q_t, k_t \in \mathbb{R}^{d_k}$ ，值向量为 $v_t \in \mathbb{R}^{d_v}$ ，递归状态为 $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$ 。KDA 在 delta-rule 更新之前应用通道级的衰减：

$$\mathbf{S}_t = (\mathbf{I} - \beta_t k_t k_t^\top) \text{Diag}(\alpha_t) \mathbf{S}_{t-1} + \beta_t k_t v_t^\top, \quad \tilde{o}_t = \mathbf{S}_t^\top q_t. \quad (1)$$

Here, $\alpha_t \in (0, 1)^{d_k}$ is the channel-wise one-step retention factor, and $\beta_t \in (0, 1)$ controls the delta-rule write strength. Following Kimi Linear [63], KDA parameterizes the per-head quantities as

这里， $\alpha_t \in (0, 1)^{d_k}$ 是通道级别的单步保留因子，而 $\beta_t \in (0, 1)$ 控制 delta 规则的写入强度。遵循 Kimi Linear [63]，KDA 将每个头的参数化量表示为

$$q_t^h, k_t^h = L_2\text{Norm}\left(\text{Swish}\left(\text{ShortConv}\left(\mathbf{W}_{q/k}^h x_t\right)\right)\right) \in \mathbb{R}^{d_k},$$

$$v_t^h = \text{Swish}(\text{ShortConv}(\mathbf{W}_v^h x_t)) \in \mathbb{R}^{d_v},$$

$$\beta_t^h = \text{Sigmoid}(\mathbf{W}_\beta^h x_t) \in (0, 1),$$

$$z_t^h = \mathbf{W}_\alpha^\uparrow \mathbf{W}_\alpha^\downarrow x_t + b_\alpha^h \in \mathbb{R}^{d_k}. \quad (2)$$

The query, key, and value projections apply ShortConv followed by Swish [138], and the query and key are further normalized with $L_2\text{Norm}$ [141]. The low-rank projection and head-specific bias $b_\alpha^h \in \mathbb{R}^{d_k}$ produce a fine-grained decay logit z_t^h for each key channel. The lower-bounded mapping from z_t^h to α_t^h is introduced after the chunkwise formulation below.

查询、键和值的投影应用了 ShortConv 后接 Swish [138]，查询和键进一步通过 $L_2\text{Norm}$ [141] 进行归一化。低秩投影和头特定偏置 $b_\alpha^h \in \mathbb{R}^{d_k}$ 为每个键通道生成细粒度的衰减对数几率 z_t^h 。在下面的分块公式之后，引入了从 z_t^h 到 α_t^h 的下界映射。

Chunkwise parallel form Following Kimi Linear [63], KDA is recurrent across chunks and parallel within each chunk. For a chunk size C , $\mathbf{X}_{[t]}$ stacks the token vectors in the t -th chunk for $\mathbf{X} \in \{\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{U}, \mathbf{W}\}$. The matrix $\mathbf{S}_{[t]} \in \mathbb{R}^{d_k \times d_v}$ denotes the recurrent state entering chunk t . For positions $1 \leq i \leq j \leq C$, define the channel-wise cumulative decay

分块并行形式依据 Kimi 线性 [63]，KDA 在分块之间进行递归，并在每个分块内部进行并行处理。对于分块大小 C ， $\mathbf{X}_{[t]}$ 将第 t 个分块中的 token 向量堆叠起来以形成 $\mathbf{X} \in \{\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{U}, \mathbf{W}\}$ 。矩阵 $\mathbf{S}_{[t]} \in \mathbb{R}^{d_k \times d_v}$ 表示进入第 t 个分块的递归状态。对于位置 $1 \leq i \leq j \leq C$ ，定义通道维度上的累积衰减

$$\gamma_{[t]}^{i \rightarrow j} := \prod_{r=i}^j \alpha_{[t]}^r, \quad \gamma_{[t]}^r := \gamma_{[t]}^{1 \rightarrow r}. \quad (3)$$

As in Kimi Linear, $\Gamma_{[t]}^{1 \rightarrow C} \in \mathbb{R}^{C \times d_k}$ stacks $\gamma_{[t]}^1, \dots, \gamma_{[t]}^C$ row-wise. The UT transform produces $\mathbf{U}_{[t]}$ and $\mathbf{W}_{[t]}$, from which we define the pseudo-value term $\tilde{\mathbf{V}}_{[t]} := \mathbf{U}_{[t]} - \mathbf{W}_{[t]} \mathbf{S}_{[t]}$. Given the incoming state $\mathbf{S}_{[t]}$, all outputs in chunk t are computed in parallel as

与 Kimi Linear 类似, $\Gamma_{[t]}^{1 \rightarrow C} \in \mathbb{R}^{C \times d_k}$ 按行堆叠 $\gamma_{[t]}^1, \dots, \gamma_{[t]}^C$ 。UT 变换产生 $\mathbf{U}_{[t]}$ 和 $\mathbf{W}_{[t]}$, 由此我们定义伪值项 $\tilde{\mathbf{V}}_{[t]} := \mathbf{U}_{[t]} - \mathbf{W}_{[t]} \mathbf{S}_{[t]}$ 。给定输入状态 $\mathbf{S}_{[t]}$, 所有在块 t 中的输出都并行计算为

$$\mathbf{A}_{[t]} = \text{Tril} \left[(\mathbf{Q}_{[t]} \odot \Gamma_{[t]}^{1 \rightarrow C}) (\mathbf{K}_{[t]} / \Gamma_{[t]}^{1 \rightarrow C})^\top \right],$$

$$\mathbf{O}_{[t]} = \underbrace{(\Gamma_{[t]}^{1 \rightarrow C} \odot \mathbf{Q}_{[t]}) \mathbf{S}_{[t]}}_{\text{inter - chunk}} + \underbrace{\mathbf{A}_{[t]} \tilde{\mathbf{V}}_{[t]}}_{\text{intra - chunk}}. \quad (4)$$

For a matrix $\mathbf{M}$, $\text{Tril}(\mathbf{M})$ sets all strictly upper-triangular entries to zero and retains the lower-triangular entries, including the diagonal. This mask enforces causal interactions within the chunk, and the diagonal is retained because each output reads the state after the current-token update. The first term in $\mathbf{O}_{[t]}$ carries information from preceding chunks, whereas the second term accounts for interactions within the current chunk. We refer readers to Kimi Linear [63] for the UT transform and the full derivation of the chunkwise form.

对于一个矩阵 $\mathbf{M}$, $\text{Tril}(\mathbf{M})$ 将所有严格上三角元素设置为零, 并保留下三角元素, 包括对角线。这种掩码强制在块内产生因果交互, 而保留对角线是因为每个输出在当前标记更新后读取状态。 $\mathbf{O}_{[t]}$ 中的第一个项携带来自前面块的信息, 而第二个项则考虑了当前块内的交互。读者可参考 Kimi Linear [63] 了解 UT 变换以及块形式的完整推导。

Lower-bounded decay Eq. 4 rescales the keys in each chunk by the reciprocal cumulative decay $1/\Gamma_{[t]}^{1 \rightarrow C}$. Because $\Gamma_{[t]}^{1 \rightarrow C}$ is a product of retention factors in $(0, 1)$, this reciprocal can grow without bound and overflow in finite precision [140, 63]. Kimi Linear controls this numerical range by computing relative decay in log space and dividing each chunk into secondary 16-token tiles [140, 63]. The off-diagonal tiles can then be computed with dense matrix multiplications on Tensor Cores directly. The diagonal tiles, in contrast, still require explicit position-pair computations, which remain the main intra-chunk bottleneck.

下界衰减公式 (式 4) 通过累积衰减的倒数 $1/\Gamma_{[t]}^{1 \rightarrow C}$ 对每个块中的键进行重新缩放。由于 $\Gamma_{[t]}^{1 \rightarrow C}$ 是 $(0, 1)$ 中保留因子的乘积, 因此该倒数可能会无限制增长并在有限精度下溢出 [140, 63]。Kimi Linear 通过在对数空间中计算相对衰减, 并将每个块划分为次级的 16 个标记的瓷砖, 从而控制这一数值范围 [140, 63]。这样, 非对角瓷砖就可以直接通过 Tensor Cores 上的密集矩阵乘法进行计算。相比之下, 对角瓷砖仍需要显式的位置对计算, 这仍然是块内计算的主要瓶颈。

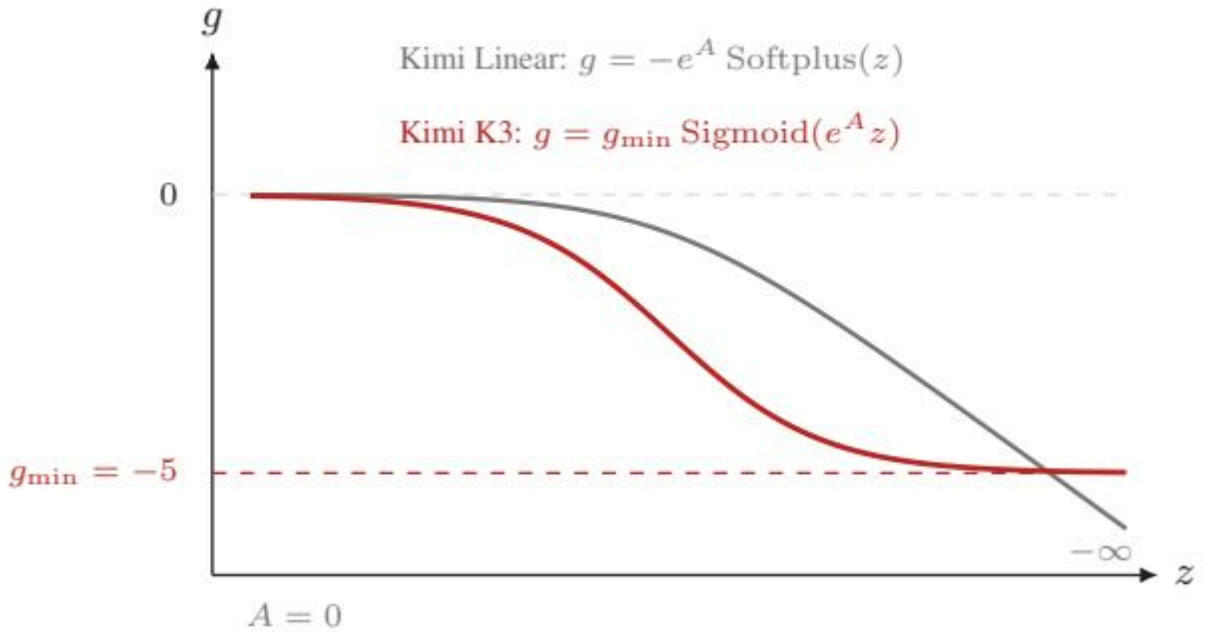


Figure 3: images/f7e37ef2224e2f76f3e7a674b5616386237299945c7d2fcae4d268543eb2fb36.jpg

(a) Log-decay parameterization.

(a) 对数衰减参数化。

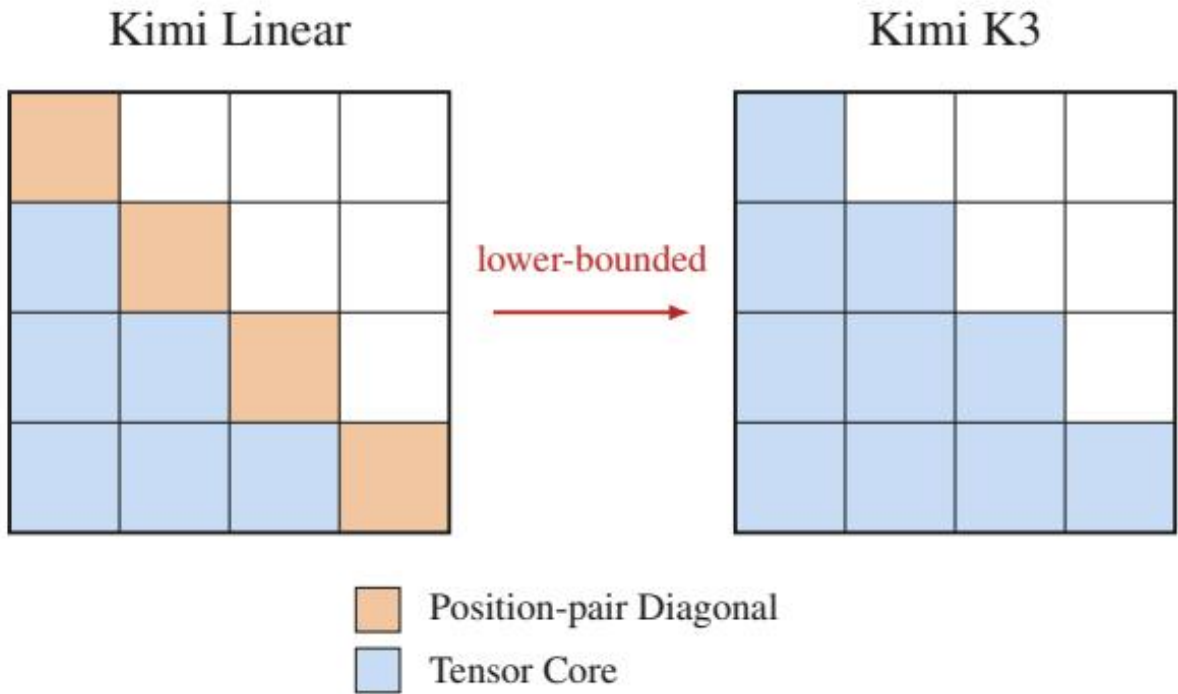


Figure 4: images/40e214b56ef4ab7f81105625e6a77ebed9c55a95603fdc1cbcd35ae3b3a2185c.jpg

(b) Diagonal-tile computation.

Figure 3: Lower-bounded decay and its effect on chunkwise KDA computation. (a) Kimi Linear uses an unbounded negative-Softplus mapping, whereas Kimi K3 bounds the log-decay with a scaled sigmoid; the curves show $A = 0$ and $g_{\min} = -5$. (b) Kimi Linear evaluates each diagonal tile with an explicit position-pair computation, while the bounded range in Kimi K3 allows all causal tiles to use dense Tensor Core matrix multiplications.

(b) 对角块计算。

图 3：下界衰减及其对分块 KDA 计算的影响。(a) Kimi Linear 使用无界的负 Softplus 映射，而 Kimi K3 则用缩放后的 Sigmoid 对对数衰减进行限制；曲线显示 $A = 0$ 和 $g_{\min} = -5$ 。(b) Kimi Linear 通过显式的位对位计算评估每个对角块，而 Kimi K3 中有限的范围允许所有因果块使用密集的 Tensor Core 矩阵乘法。

Kimi K3 addresses this bottleneck by changing the mapping from the decay logits z_t^h to the per-step log-decay g_t^h . Following GDN and Mamba-2, Kimi Linear uses the negative-Softplus mapping $g_t^h = -e^{A_h} \text{Softplus}(z_t^h) \in (-\infty, 0)^{d_k}$ [138, 24, 63]. Kimi K3 instead uses a scaled sigmoid to bound the log-decay from below:

Kimi K3 通过将衰减 logit z_t^h 的映射改为每步 log-decay g_t^h 来解决这一瓶颈。遵循 GDN 和 Mamba-2, Kimi Linear 使用负 Softplus 映射 $g_t^h = -e^{A_h} \text{Softplus}(z_t^h) \in (-\infty, 0)^{d_k}$ [138, 24, 63]。而 Kimi K3 则使用一个缩放的 Sigmoid 函数来对 log-decay 进行下界限制：

$$g_t^h = g_{\min} \text{Sigmoid}(e^{A_h} z_t^h) \in (g_{\min}, 0)^{d_k},$$

$$\alpha_t^h = \exp(g_t^h) \in (e^{g_{\min}}, 1)^{d_k}, \quad (5)$$

where A_h is a learnable per-head log-scale and $g_{\min} = -5$ is fixed. We initialize $A_h = 0$, and each bias b_α^h is initialized following [63, 24, 138]. With $g_{\min} = -5$, every retention factor satisfies $\alpha_{t,j}^h > e^{-5} \approx 6.7 \times 10^{-3}$, and the cumulative log-decay over a 16-token tile lies in $(-80, 0)$. The corresponding reciprocal rescaling factor is therefore smaller than e^{80} and remains within the BF16 dynamic range. This finite range allows both diagonal and off-diagonal tiles to use dense Tensor Core matrix multiplications, eliminating the position-pair diagonal path. This parameterization is closely related to the lower-bounded recurrence gates in prior work [97, 27, 91]. Fig. 3 illustrates the change in decay parameterization and its computational consequence.

其中 A_h 是每个头的可学习对数尺度， $g_{\min} = -5$ 是固定的。我们初始化 $A_h = 0$ ，每个偏置 b_α^h 的初始化方式参考 [63, 24, 138]。通过 $g_{\min} = -5$ ，每个保留因子满足 $\alpha_{t,j}^h > e^{-5} \approx 6.7 \times 10^{-3}$ ，并且在 16 个 token 的块上累积的对数衰减位于 $(-80, 0)$ 范围内。因此，相应的倒数重标定因子小于 e^{80} ，并保持在 BF16 动态范围内。这个有限的范围使得对角线和非对角线块都可以使用密集的 Tensor Core 矩阵乘法，从而消除了位置对角线路径。这种参数化方法与先前工作中下界递归门 [97, 27, 91] 密切相关。图 3 展示了衰减参数化的变化及其计算后果。

Full-rank gate Finally, Kimi K3 changes KDA’s output gate from the low-rank parameterization used by Kimi Linear [63] to an input-dependent full-rank projection. After applying head-wise RMSNorm [146] to the recurrent output, KDA applies data-dependent output gating [99]:

全秩门控最终，Kimi K3 将 KDA 的输出门控从 Kimi Linear [63] 使用的低秩参数化方法，更改为依赖输入的全秩投影。在对循环输出应用逐头 RMSNorm [146] 后，KDA 应用数据依赖的输出门控 [99]：

$$y_t = \mathbf{W}_o[\text{Sigmoid}(\mathbf{W}_g x_t) \odot \text{RMSNorm}(\tilde{o}_t)]. \quad (6)$$

2.1.2 Gated MLA

2.1.2 门控 MLA

Multi-head Latent Attention (MLA), introduced in DeepSeek-V2 [28], compresses the key-value representation of each token into a low-dimensional latent vector $c_t = W_c x_t$. Instead of caching full head-specific keys and values, MLA caches c_t and reconstructs the content keys and values through learned up-projections during attention computation. This factorization reduces the KV-cache footprint while retaining global token-to-token attention. MLA was subsequently adopted by Kimi K2 and Kimi K2.5 [58, 59], and Kimi K3 retains it in the periodic global-attention layers.

多头潜在注意力 (Multi-head Latent Attention, MLA), 在 DeepSeek-V2 [28] 中被引入, 将每个 token 的键-值表示压缩为一个低维的潜在向量 $c_t = W_c x_t$ 。与缓存完整的头特定键和值不同, MLA 缓存 c_t , 并在注意力计算过程中通过学习到的上投影重构内容键和值。这种分解在减少 KV 缓存占用空间的同时, 保留了全局的 token-to-token 注意力。MLA 随后被 Kimi K2 和 Kimi K2.5 [58, 59] 采用, 而 Kimi K3 在周期性全局注意力层中保留了这一机制。

Unlike Kimi K2 and Kimi K2.5, Kimi K3 follows the hybrid design of Kimi Linear [63] and applies No Position Encoding (NoPE) to all MLA layers. Consequently, no explicit positional encoding is applied to their queries or keys. The intervening KDA layers provide position-sensitive and recency-aware sequence mixing, while the MLA layers provide unrestricted global content interaction. This separation also avoids modifying positional-encoding parameters when extending the context length, such as retuning a RoPE frequency base or applying YaRN [92].

与 Kimi K2 和 Kimi K2.5 不同, Kimi K3 采用了 Kimi Linear [63] 的混合设计, 并对所有 MLA 层应用 No Position Encoding (NoPE)。因此, 它们的查询或键不会显式地应用位置编码。中间的 KDA 层提供位置敏感且关注近期的序列混合, 而 MLA 层提供不受限制的全局内容交互。这种分离也避免了在扩展上下文长度时修改位置编码参数, 例如重新调整 RoPE 频率基或应用 YaRN [92]。

In addition, Kimi K3 augments MLA with an input-dependent, channel-wise full-rank output gate. Let $\tilde{o}_t$ denote the ungated MLA output at position t ; the gated output is

此外, Kimi K3 在 MLAs 的基础上, 引入了一个依赖于输入的、按通道全秩的输出门。令 $\tilde{o}_t$ 表示位置 t 处未经过门控的 MLA 输出; 经过门控的输出为

$$y_t = \mathbf{W}_o [\text{Sigmoid}(\mathbf{W}_g x_t) \odot \tilde{o}_t]. \quad (7)$$

The gate projection $\mathbf{W}_g$ is full rank, matching the new parameterization used by KDA in Kimi K3. This gate allows each token to modulate the channels read from global attention [99].

门投影 $\mathbf{W}_g$ 是满秩的, 与 KDA 在 Kimi K3 中使用的新型参数化方法一致。该门允许每个 token 调制从全局注意力中读取的通道 [99]。

To correct the biased rounding error that arises in flash attention, we adopt the method of [98] and keep the attention output in FP32 during training. This choice doubles the on-chip footprint of the output tile; we therefore redesign the training kernel to overlap it with the KV staging buffers instead of the query tile, freeing shared memory for a deeper KV pipeline and higher training throughput.

为了纠正在 Flash Attention 中出现的偏倚舍入误差, 我们采用 [98] 的方法, 并在训练过程中将注意力输出保持在 FP32 格式。这一选择使输出瓦片的片上存储占用翻倍; 因此, 我们重新设计了训练内核, 使其与 KV 阶段缓冲区重叠, 而不是与查询瓦片重叠, 从而释放共享内存, 实现更深层的 KV 流水线和更高的训练吞吐量。

2.2 Attention Residuals

2.2 注意力残差

Standard residual connections [43] compress all prior information into a single state h_l over depth — a bottleneck reminiscent of RNNs over time. For sequence modeling, the Transformer replaced recurrence with attention [10, 125], allowing each position to selectively access all previous positions with data-dependent weights. Attention Residuals (AttnRes) [57] applies the same methodology to depth: each layer selectively retrieves representations from all preceding layers rather than accumulating them uniformly.

标准的残差连接 [43] 将所有先前的信息压缩到一个单一的状态 h_l 中，随着深度的增加，这种瓶颈类似于时间上的 RNN。对于序列建模，Transformer 用注意力机制取代了递归 [10, 125]，使得每个位置可以使用数据依赖的权重选择性地访问所有先前的位置。注意力残差 (AttnRes) [57] 将相同的方法应用于深度：每一层选择性地从所有先前的层中检索表示，而不是均匀地累积这些表示。

Full Attention Residuals For each layer l , we define a layer-specific learnable pseudo-query $q_l = w_l \in \mathbb{R}^d$ and keys and values

全注意力残差对于每个层 l ，我们定义一个特定于该层的可学习伪查询 $q_l = w_l \in \mathbb{R}^d$ 和键和值

$$k_i = v_i = \begin{cases} h_1 & i = 0 \\ f_i(h_i) & 1 \leq i \leq l-1 \end{cases} \quad (8)$$

where $f_i(h_i)$ is the output of layer i and h_1 is the token embedding. The attention weights follow a softmax kernel $\phi(q, k) = \exp(q^\top \text{RMSNorm}(k))$ [55, 146], where the RMSNorm prevents layers with large-magnitude outputs from dominating the weights:

其中 $f_i(h_i)$ 是第 i 层的输出， h_1 是 token embedding。注意力权重遵循一个 softmax 核函数 $\phi(q, k) = \exp(q^\top \text{RMSNorm}(k))$ [55, 146]，其中 RMSNorm 防止具有大幅输出的层主导权重：

$$\alpha_{i \rightarrow l} = \frac{\phi(q_l, k_i)}{\sum_{j=0}^{l-1} \phi(q_l, k_j)}, \quad h_l = \sum_{i=0}^{l-1} \alpha_{i \rightarrow l} \cdot v_i. \quad (9)$$

Since network depth is modest ($L < 100$), the $O(L^2d)$ arithmetic of this full form is affordable; the practical overhead is the $O(Ld)$ memory (and cross-stage communication under pipeline parallelism) for keeping all layer outputs alive.

由于网络深度适中 ($L < 100$)，这种完整形式的 $O(L^2d)$ 算术是可行的；实际的开销是 $O(Ld)$ 内存（以及在流水线并行下的跨阶段通信），用于保持所有层的输出。

Block Attention Residuals To reduce this overhead, we partition the L layers into N blocks of $S = L/N$ layers each. Within block n (layer indices B_n), layer outputs are reduced to a single representation by summation, $b_n = \sum_{j \in B_n} f_j(h_j)$, with b_n^i denoting the partial sum over the first i layers of the block; we set $b_0 = h_1$ so the token embedding is always included as a source. Across blocks, full attention is applied over only the N block-level representations: for the i -th layer in block n , the value matrix is

块注意力残差为了减少这种开销，我们将 L 层划分为 N 个块，每个块包含 $S = L/N$ 层。在块 n （层索引 B_n ）内，层输出通过求和被缩减为一个表示， $b_n = \sum_{j \in B_n} f_j(h_j)$ ，其中 b_n^i 表示块中前 i 层的部分

和；我们设置 $b_0 = h_1$ 以确保 token 嵌入始终作为源包含在内。在块之间，仅对 N 个块级表示应用完整注意力：对于块 n 中的第 i 层，值矩阵是

$$\mathbf{v} = \begin{cases} [b_0, b_1, \dots, b_{n-1}]^\top & \text{if } i = 1 (\text{first layer of block } n) \\ [b_0, b_1, \dots, b_{n-1}, b_n^{i-1}]^\top & \text{if } i \geq 2 (\text{subsequent layers}) \end{cases} \quad (10)$$

with keys and attention weights following Eq. 8 and Eq. 9. The final output layer then aggregates all N block representations. Under Block AttnRes, memory and communication overhead drop from $O(Ld)$ to $O(Nd)$, while this block structure also bounds the inference-time state, enabling the parallel inter-block results to be better merged with the sequential intra-block partial sums via online softmax [79], significantly reducing inference time cost.

使用键和注意力权重，遵循公式 8 和公式 9。最终的输出层则聚合所有 N 个块表示。在 Block AttnRes 中，内存和通信开销从 $O(Ld)$ 降低到 $O(Nd)$ ，同时，这种块结构还限制了推理时的状态，使得可以通过在线 softmax [79] 将并行的块间结果更好地与串行的块内部分和合并，显著减少了推理时间成本。

Empirically, $N \approx 8$ recovers most of the benefit across model scales [57]; for Kimi K3, we partition its layers into 8 blocks with 12-layer size, giving a partial final block and 9 total blocks when counting the embedding layer.

实证研究表明， $N \approx 8$ 在不同模型规模下都能恢复大部分收益 [57]；对于 Kimi K3，我们将其层分割为 8 个块，每个块大小为 12 层，计算嵌入层时，得到一个部分最终块和总共 9 个块。

2.3 Stable LatentMoE

2.3 稳定的潜在 MoE

Increasing both the expert pool and the number of active experts expands the space of expert specializations, but in a conventional MoE each selected expert receives the full d -dimensional token representation, so communication and expert-weight traffic grow with the routing multiplicity. LatentMoE [32] makes this expansion affordable by separating the full model width from the routed-expert width: shared experts retain a full-width path for common transformations, whereas specialized routed experts operate in a compact latent space of width ℓ . This enables Kimi K3 to scale channel mixing to 896 routed experts with 16 active experts per token, corresponding to a sparsity of 56.

增加专家池的规模以及活跃专家的数量，可以扩展专家专业化的空间，但在传统的 MoE 中，每个被选中的专家都会接收到完整的 d 维度的 token 表示，因此通信和专家权重的流量会随着路由的复杂度而增长。LatentMoE [32] 通过将完整的模型宽度与路由专家的宽度分离，使这种扩展变得可行：共享专家保留完整的宽度路径以进行常见的转换，而专门化的路由专家则在宽度为 ℓ 的紧凑潜在空间中运行。这使得 Kimi K3 能够将通道混合扩展到 896 个路由专家，每个 token 有 16 个活跃专家，对应稀疏度为 56。

This extreme sparsity amplifies two failure modes of the vanilla design. First, the routed path composes $W^\downarrow$, a gated multi-branch expert feed-forward network, and $W^\uparrow$ into a chain of nearly four consecutive matrix multiplications. This ill-conditioned structure, combined with the 2.8-trillion-parameter scale, produces exploding internal activations in the routed branch. Second, balancing the load of nearly 10^3 experts exceeds the regime in which existing auxiliary-loss-free

这种极端的稀疏性放大了原始设计的两种失效模式。首先，路由路径将 $W^\downarrow$ ，一个带门控的多分支专家前馈网络，和 $W^\uparrow$ 组合成一个几乎连续四个矩阵乘法的链式结构。这种病态条件的结构，结合 2.8 万亿参数的规模，导致路由分支中出现内部激活值爆炸。其次，平衡近 10^3 个专家的负载超出了现有无辅助损失方法的适用范围。

	Gate branch	Up branch	Curve
GLU [26]	$\sigma(x)$	x	
SwiGLU [107]	$x \cdot \sigma(x)$	x	
SiTU-GLU	$\beta_1 \tanh\left(\frac{x}{\beta_1}\right) \cdot \sigma(x)$	$\beta_2 \tanh\left(\frac{x}{\beta_2}\right)$	

	闸门分支	上分支	曲线
GLU [26]	$\sigma(x)$	x	
SwiGLU [107]	$x \cdot \sigma(x)$	x	
SiTU-GLU	$\beta_1 \tanh\left(\frac{x}{\beta_1}\right) \cdot \sigma(x)$	$\beta_2 \tanh\left(\frac{x}{\beta_2}\right)$	

Figure 4: Gate and up branches of GLU, SwiGLU, and SiTU-GLU, together with their scalar responses, where σ denotes the sigmoid function. Both branches receive the scalar input x , and all curves share the domain $x \in [-10, 100]$; the inset magnifies the near-origin region. SiTU-GLU, shown in red with $\beta_1 = 4$ and $\beta_2 = 25$, closely follows SwiGLU near the origin and approaches the bound $|f(x)| \leq \beta_1 \beta_2 = 100$ for large positive inputs, whereas SwiGLU remains unbounded.

图 4: GLU、SwiGLU 和 SiTU-GLU 的门控分支和上分支，以及它们的标量响应，其中 σ 表示 sigmoid 函数。两个分支都接收标量输入 x ，所有曲线共享定义域 $x \in [-10, 100]$ ；插图放大了近原点区域。SiTU-GLU（用 $\beta_1 = 4$ 和 $\beta_2 = 25$ 表示，红色显示）在原点附近紧随 SwiGLU，并且对于大正输入接近边界 $|f(x)| \leq \beta_1 \beta_2 = 100$ ，而 SwiGLU 则保持无界。

bias updates remain well behaved. Stable LatentMoE addresses these two failure modes with three components: an RMSNorm before the up-projection and Sigmoid Tanh Unit GLU (SiTU-GLU) to suppress activation explosion, and Quantile Balancing (QB) for load balancing.

bias updates remain well behaved. Stable LatentMoE addresses these two failure modes with three components: an RMSNorm before the up-projection and Sigmoid Tanh Unit GLU (SiTU-GLU) to suppress activation explosion, and Quantile Balancing (QB) for load balancing.

As illustrated in Fig. 2, the layer follows the shared- and routed-expert organization of DeepSeek-MoE [23]. For $x \in \mathbb{R}^d$, the shared experts process x directly, while the routed path projects it to $z = \mathbf{W}^\downarrow x \in \mathbb{R}^\ell$, dispatches z to the selected experts, and maps their weighted aggregate back to $\mathbb{R}^d$ through $\mathbf{W}^\uparrow$:

如图 2 所示，该层遵循 DeepSeekMoE [23] 的共享和路由专家结构。对于 $x \in \mathbb{R}^d$ ，共享专家直接处理 x ，而路由路径将其投影到 $z = \mathbf{W}^\downarrow x \in \mathbb{R}^\ell$ ，将 z 分配给选定的专家，并通过 $\mathbf{W}^\uparrow$ 将其加权聚合映射回 $\mathbb{R}^d$ ：

$$u = \sum_{i \in \mathcal{T}_k(x)} p_i E_i^{\text{routed}}(\mathbf{W}^\downarrow x),$$

$$y = \sum_{j=1}^{N_s} E_j^{\text{shared}}(x) + \mathbf{W}^\uparrow \text{RMSNorm}(u). \quad (11)$$

Here, $u \in \mathbb{R}^\ell$ is the aggregated routed representation, $E_j^{\text{shared}} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ and $E_i^{\text{routed}} : \mathbb{R}^\ell \rightarrow \mathbb{R}^\ell$ are the shared and routed expert feed-forward networks, and p_i is the router weight

defined by the Quantile Balancing rule below. Kimi K3 fixes the number of full-width shared experts to $N_s = 2$ in every layer.

在这里, $u \in R^\ell$ 是聚合的路由表示, $E_j^{shared} : R^d \rightarrow R^d$ 和 $E_i^{routed} : R^\ell \rightarrow R^\ell$ 是共享和路由的专家前馈网络, p_i 是由下面的分位数平衡规则定义的路由权重。Kimi K3 在每一层中将全宽共享专家的数量固定为 $N_s = 2$ 。

2.3.1 Normalized LatentMoE

2.3.1 标准化潜在 MoE

The original LatentMoE directly applies $W^\uparrow$ to the aggregated routed representation u , whose scale can vary with the selected experts and their routing weights. As shown in Eq. 11, Kimi K3 instead inserts RMSNorm [146] between expert aggregation and the up-projection. This normalization reduces the sensitivity of the routed branch to scale variation before it is combined with the full-width shared branch. Beyond stabilizing training, the additional RMSNorm consistently improves validation loss and downstream benchmarks.

原始的 LatentMoE 直接将 $W^\uparrow$ 应用于聚合后的路由表示 u , 其尺度会随着所选专家及其路由权重的不同而变化。如 Eq. 11 所示, Kimi K3 会在专家聚合和上投影之间插入 RMSNorm [146]。这种归一化方法降低了路由分支在与全宽度共享分支结合前对尺度变化的敏感性。除了稳定训练之外, 额外的 RMSNorm 一直有助于降低验证损失并提升下游基准表现。

2.3.2 Sigmoid Tanh Unit GLU

2.3.2 Sigmoid Tanh Unit GLU

Gated Linear Units (GLUs) modulate a linear value branch with a sigmoid-activated gate, computing $\text{Sigmoid}(W_g x) \odot W_u x$ [26]. SwiGLU replaces the sigmoid gate with $\text{Swish}(x) = x \text{Sigmoid}(x)$ and yields strong empirical performance in Transformers [107]. SwiGLU has subsequently become a widely adopted FFN design in large language models, while a complete account of its empirical effectiveness remains open.

门控线性单元 (GLUs) 通过一个使用 sigmoid 激活函数的门控机制来调节一个线性值分支, 计算 $\text{Sigmoid}(W_g x) \odot W_u x$ [26]。SwiGLU 用 $\text{Swish}(x) = x \text{Sigmoid}(x)$ 替代 sigmoid 门控机制, 在 Transformer 中表现出强大的实证性能 [107]。随后, SwiGLU 成为大型语言模型中广泛采用的前馈网络 (FFN) 设计, 但对其实证有效性的完整分析仍处于开放状态。

However, both multiplicative factors in SwiGLU are unbounded, so coincident large coordinates can produce activation outliers and increase overflow risk in low-precision arithmetic. The sigmoid gate of the original GLU avoids unbounded gate growth, but it does not retain the approximately linear positive regime of Swish. This motivates an activation that controls large-value growth while preserving the characteristic local and positive-side response of SwiGLU. Other recent efforts have explored alternative parameterizations of this trade-off [51].

然而, SwiGLU 中的两个乘法因子都是无界的, 因此, 坐标过大可能会导致激活值异常并增加低精度算术中的溢出风险。原始 GLU 中的 sigmoid 网络门可以避免无界门增长, 但它无法保留 Swish 的近似线性正区域。这促使我们设计一种激活函数, 它在控制大值增长的同时, 保留 SwiGLU 的局部性和正方向响应特性。其他近期的研究已经探讨了这种权衡的替代参数化方法 [51]。

To satisfy these requirements, we propose Sigmoid Tanh Unit GLU (SiTU-GLU). SiTU-GLU applies the smooth cap $\text{softcap}(x, \beta) = \beta \tanh(x/\beta)$ to the linear factor of the Swish gate and independently to the up branch:

为满足这些要求，我们提出了 Sigmoid Tanh Unit GLU (SiTU-GLU)。SiTU-GLU 将平滑的软帽 $(x, \beta) = \beta \tanh(x/\beta)$ 应用于 Swish 门的线性因子，并独立地应用于上分支：

$$\text{SiTU-GLU}(x) = \left[\beta_1 \tanh\left(\frac{\mathbf{W}_g x}{\beta_1}\right) \odot \text{Sigmoid}(\mathbf{W}_g x) \right] \odot \left[\beta_2 \tanh\left(\frac{\mathbf{W}_u x}{\beta_2}\right) \right], \quad (12)$$

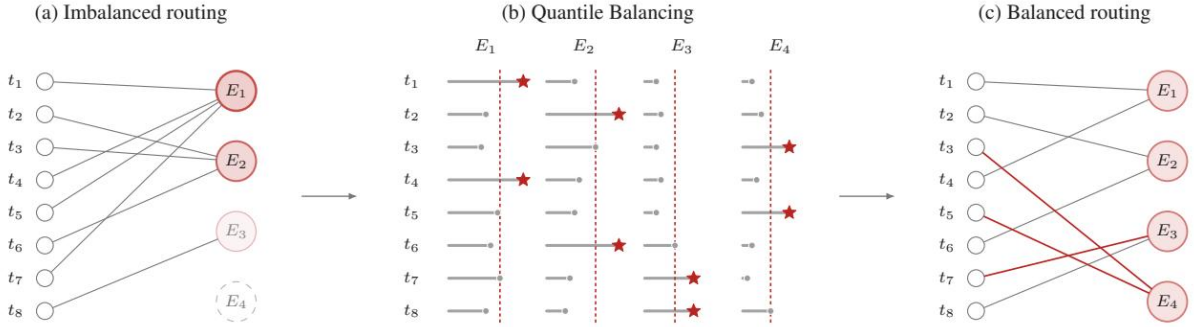


Figure 5: images/ef54f9ea7ade59973c91c308675b450c4bc2293d28d39079f2cea0a269454304.jpg

Figure 5: Illustration of Quantile Balancing with $m = 8$ tokens, $n = 4$ routed experts, and $k = 1$ selected expert per token. (a) Token-wise Top-k routing (tokens on the left, experts on the right) produces loads $(4, 3, 1, 0)$; darker circles indicate overheated experts, whereas faded and dashed circles indicate underutilized and dying experts, respectively. (b) Each gray bar is the margin of the currently biased score, $s_{i,j} + b_j^{(t)} - \alpha_i^{(t)}$, so the row-wise maxima reproduce the routing in (a). The dashed red line in each column is the bias adjustment $b_j^{(t)} - \hat{b}_j^{(t+1)}$, placed at the $(q+1)$ -th largest margin so that exactly $q = 2$ margins exceed it. The marker $\square$ denotes the row-wise Top-k choice after subtracting the column adjustments, i.e., the routing in (c). (c) The retained choices yield the balanced load $(2, 2, 2, 2)$; red edges denote assignments changed by QB.

图 5：使用 $m = 8$ 个 token， $n = 4$ 个路由专家，以及每个 token 选择 $k = 1$ 个选定专家的分位数平衡示意图。(a) 按 token 进行的 Top-k 路由（左侧为 token，右侧为专家）产生负载 $(4, 3, 1, 0)$ ；较深的圆圈表示过热的专家，而褪色和虚线圆圈分别表示未充分利用和即将死亡的专家。(b) 每个灰色条形图是当前偏倚评分的边际 $s_{i,j} + b_j^{(t)} - \alpha_i^{(t)}$ ，因此按行的最大值再现了 (a) 中的路由。每列中的虚线红色线是偏倚调整 $b_j^{(t)} - \hat{b}_j^{(t+1)}$ ，放置在 $(q+1)$ -th 最大的边际处，使得恰好 $q = 2$ 个边际超过它。标记 $\square$ 表示在减去列调整后按行选择的 Top-k 选择，即 (c) 中的路由。(c) 保留的选择产生平衡的负载 $(2, 2, 2, 2)$ ；红色边表示由 QB 更改的分配。

For Kimi K3, we set the soft-cap hyperparameters to $\beta_1 = 4$ for the gate branch and $\beta_2 = 25$ for the up branch. The scaled tanh is approximately linear near the origin and bounded at large magnitude, allowing SiTU-GLU to preserve the local response of SwiGLU while controlling both factors in the product. Fig. 4 compares the branch definitions and scalar responses of GLU, SwiGLU, and SiTU-GLU on a common slice.

对于 Kimi K3，我们为 gate 分支设置了软上限超参数 $\beta_1 = 4$ ，为 up 分支设置了 $\beta_2 = 25$ 。缩放后的 tanh 在原点附近近似线性，并且在大值时有界，这使得 SiTU-GLU 能够在保持 SwiGLU 局部响应的同时，控制乘积中的两个因子。图 4 比较了 GLU、SwiGLU 和 SiTU-GLU 在相同切片上的分支定义和标量响应。

§ B gives the local expansion, limiting case, formal output bound, and comparison with hard clamping.

§ B 给出了局部展开、极限情况、形式输出界限以及与硬钳位的比较。

2.3.3 Quantile Balancing

2.3.3 分位数平衡

Unlike auxiliary-loss-based routing [33], Kimi K3 adopts auxiliary-loss-free routing [30]. Load balancing is implemented by adding an expert-specific bias b_j to the router score used for Top- k selection. For token x_i , the router computes $s_i = \text{Sigmoid}(\mathbf{W}_r x_i)$ and applies

与基于辅助损失的路由 [33] 不同，Kimi K3 采用无辅助损失的路由 [30]。负载均衡通过向用于 Top- k 选择的路由得分添加特定专家的偏置 b_j 来实现。对于 token x_i ，路由器计算 $s_i = \text{Sigmoid}(\mathbf{W}_r x_i)$ 并应用

$$\mathcal{T}_i = \text{argtop}_k(s_i + b), \quad p_{i,j} = \frac{s_{i,j}}{\sum_{r \in \mathcal{T}_i} s_{i,r}}, \quad j \in \mathcal{T}_i. \quad (13)$$

Because b is omitted from $p_{i,j}$, it regulates dispatch without altering the mixture weights or the gradient-based optimization of the router. The original method updates b with the fixed-step rule $b_j^{(t+1)} = b_j^{(t)} + \gamma \text{sign}(\bar{\ell} - \ell_j^{(t)})$ [30], for which γ trades off slow adaptation against load oscillation. Maintaining balanced loads becomes more challenging as LatentMoE increases the routed expert pool to 896 per layer. Imbalanced routing slows expert-parallel training and may leave some experts poorly trained [47].

因为 b 被省略了 $p_{i,j}$ ，它在不改变混合权重或路由器的基于梯度的优化的情况下进行调度。原始方法使用固定步长规则 $b_j^{(t+1)} = b_j^{(t)} + \gamma \text{sign}(\bar{\ell} - \ell_j^{(t)})$ [30] 更新 b ，其中 γ 在缓慢适应和负载波动之间进行权衡。随着 LatentMoE 将每层的路由专家池增加到 896，保持负载平衡变得更加困难。不平衡的路由会减慢专家并行训练，并可能导致某些专家训练不足 [47]。

To address this limitation, we introduce Quantile Balancing (QB), which sets each expert bias from the router-score quantile that matches its target load [111]. Consider a training batch of m tokens routed to n experts with Top- k selection, so the target load is $q := mk/n$ tokens per expert. QB derives the next bias from a single forward pass. Routing replaces the Top- k selection with Top- $(k+1)$ on the biased score $s_i + b^{(t)}$: the first k entries are the routes actually taken, while the $(k+1)$ -th entry is the cutoff $\alpha_i^{(t)}$ that an expert must exceed to enter token i 's Top- k . Taking the cutoff from Top- $(k+1)$ routing avoids a separate token-side quantile. We then choose each expert bias so that expert j receives its target load: with the cutoffs fixed, the token count routed to expert j under a candidate bias $\hat{b}_j^{(t+1)}$ is

为了解决这一限制，我们引入了分位数平衡 (Quantile Balancing, QB)，它通过从与目标负载匹配的路由分数分位数中设置每个专家的偏差来实现 [111]。考虑一个包含 m 个标记的训练批次，这些标记通过 Top- k 选择路由到 n 个专家，因此每个专家的目标负载是 $q := mk/n$ 个标记。QB 通过一次前向传播推导出下一个偏差。路由将 Top- k 选择替换为在有偏分数 $s_i + b^{(t)}$ 上的 Top- $(k+1)$ ：前 k 个条目是实际采用的路由，而第 $(k+1)$ 个条目是专家必须超过的截止值 $\alpha_i^{(t)}$ ，以进入标记 i 的 Top- k 。从 Top- $(k+1)$ 路由中获取截止值，可以避免单独的标记侧分位数。然后，我们选择每个专家的偏差，使得专家 j 能够获得其目标负载：在截止值固定的情况下，候选偏差 $\hat{b}_j^{(t+1)}$ 下路由到专家 j 的标记数量为

$$\sum_{i=1}^m \mathbf{1} [s_{i,j} + \hat{b}_j^{(t+1)} > \alpha_i^{(t)}],$$

which is monotonically decreasing in the threshold $-\hat{b}_j^{(t+1)}$. Assuming no ties, setting this count

to q makes $-\hat{b}_j^{(t+1)}$ the $(q+1)$ -th largest margin $s_{i,j} - \alpha_i^{(t)}$, so that exactly q margins stay above the threshold. Since $q/m = k/n$, this is

在阈值 $-\hat{b}_j^{(t+1)}$ 下单调递减。假设没有并列情况，将此计数设置为 q ，则使 $-\hat{b}_j^{(t+1)}$ 成为 $(q+1)$ -th 最大的边距 $s_{i,j} - \alpha_i^{(t)}$ ，从而使得恰好 q 个边距高于阈值。由于 $q/m = k/n$ ，因此

the $(1 - k/n)$ -quantile of the margins across tokens, giving the QB update

$(1 - k/n)$ -分位数的边缘值，用于给出 QB 更新

$$\begin{aligned}\hat{b}_j^{(t+1)} &\leftarrow -\text{quantile}_{1-k/n}\left(s_{:,j} - \alpha^{(t)}\right), \\ b^{(t+1)} &\leftarrow \hat{b}^{(t+1)} - \text{mean}\left(\hat{b}^{(t+1)}\right) \mathbf{1}.\end{aligned}\tag{14}$$

The margins subtract the biased cutoff $\alpha_i^{(t)}$ from the raw score $s_{i,j}$, so the old bias enters the update only through the cutoffs, and the second line removes a common offset that leaves Top-k selection unchanged. For causality, the update takes effect only in the next step [30], i.e., a batch is never routed with a bias derived from itself. Fig. 5 illustrates the case $m = 8$, $n = 4$, and $k = 1$, where each expert receives the target load $q = 2$. The final bias is frozen at inference. The balanced-assignment derivation is given in § C.

边距从原始分数 $s_{i,j}$ 中减去了有偏的截断值 $\alpha_i^{(t)}$ ，因此旧的偏差仅通过截断值进入更新，第二行移除了一个共同的偏移量，而不会改变 Top-k 选择。对于因果关系，更新仅在下一步生效 [30]，即一个批次永远不会使用自身衍生出的偏差进行路由。图 5 展示了 $m = 8$ ， $n = 4$ ， $k = 1$ 的情况，其中每个专家接收的目标负载 $q = 2$ 。最终的偏差在推理时被固定。平衡分配的推导在第 C 节给出。

Histogram estimation At scale, the quantile in Eq. 14 spans the full global batch, whose margins number in the millions and are spread across ranks and accumulation steps, so gathering them for an exact quantile is not viable at training time. We instead read each expert’s quantile from a histogram of its margins: a single all-reduce sums the per-rank bin counts, and the quantile is recovered from the pooled counts. Because counts are additive, the histogram represents the pooled global batch regardless of how tokens are sharded, so the estimate reflects the whole-batch quantile up to the bin width, at a communication cost of only a few hundred bins per expert. This histogram estimator is the method we use in practice; we give more detailed descriptions of it and its error bound in § D.

直方图估计在大规模情况下，式 14 中的分位数覆盖了整个全局批次，其边缘数量达到数百万，并分布在不同的秩和累积步骤中，因此在训练时收集这些边缘以精确计算分位数是不可行的。我们转而从每个专家的边缘直方图中读取其分位数：一个单一的全减少操作将每个秩的直方图桶计数相加，然后从汇总的计数中恢复分位数。由于计数是可加的，无论令牌如何分片，直方图都代表了汇总后的全局批次，因此该估计值在仅需每个专家数百个桶的通信成本下，即可反映整个批次的分位数，精度达到桶宽的程度。这种直方图估计器是我们实际使用的方法；我们在 § D 中提供了该方法及其误差上限的更详细描述。

2.4 Native Vision

2.4 原生视觉

Kimi K3 is natively multimodal: text, images, and videos are processed by a single shared backbone within one context, with no post-hoc modality-alignment stage. This design is the architectural foundation of the long-horizon, vision-in-the-loop behavior described in §1. Rendered

outputs and the code that produced them live in the same token stream, the model can write code, inspect screenshots or video frames of the result, and iteratively refine visual artifacts—user interfaces, graphics, video—with no cross-model hand-off.

Kimi K3 是原生的多模态模型：文本、图像和视频在一个上下文中由一个共享的主干网络进行处理，无需后期的模态对齐阶段。这种设计是第 1 节中描述的长视野、视觉反馈行为的架构基础。渲染输出及其生成代码都存在于同一个 token 流中，模型可以编写代码，检查结果的截图或视频帧，并迭代地优化视觉成果——用户界面、图形、视频——而无需跨模型交接。

MoonViT-V2 A key departure from Kimi K2.5 is that we train Kimi K3 vision encoder, MoonViT-V2, entirely from scratch with next-token prediction. Prior practice, including Kimi K2.5 itself, initializes the vision encoder from a contrastively pre-trained model such as SigLIP, under the premise that pre-trained visual knowledge gives the model a head start. We depart from this practice primarily for training stability. When a pre-trained encoder is attached to the LLM, joint optimization becomes unstable: the SigLIP-initialized MoonViT-3D shows persistently higher gradient norms with frequent spikes, while MoonViT-V2 remains stable throughout training (Fig. 6). Training with next-token prediction also allows the encoder’s representations to be shaped directly by the language-modeling objective, rather than by a contrastive loss that favors global semantics over fine-grained textual and structural cues. Notably, we find MoonViT-V2 matches the SigLIP-initialized baseline across vision evaluations, indicating that contrastive pre-training is unnecessary as an initialization for multimodal language models at scale.

MoonViT-V2 与 Kimi K2.5 的关键区别在于，我们完全从头开始使用下一个 token 预测来训练 Kimi K3 视觉编码器 MoonViT-V2。之前的实践，包括 Kimi K2.5 本身，都是从对比预训练模型（如 SigLIP）初始化视觉编码器，前提是预训练的视觉知识可以为模型提供一个良好的起点。我们主要出于训练稳定性方面的考虑而偏离了这种做法。当将预训练编码器连接到语言模型（LLM）时，联合优化变得不稳定：以 SigLIP 初始化的 MoonViT-3D 在训练过程中表现出持续较高的梯度范数并频繁出现尖峰，而 MoonViT-V2 在整个训练过程中保持稳定（图 6）。使用下一个 token 预测进行训练还允许编码器的表示直接由语言建模目标塑造，而不是由倾向于优先考虑全局语义而非细粒度文本和结构线索的对比损失来塑造。值得注意的是，我们发现 MoonViT-V2 在视觉评估中与 SigLIP 初始化的基线表现相当，这表明在大规模多模态语言模型中，对比预训练作为初始化是不必要的。

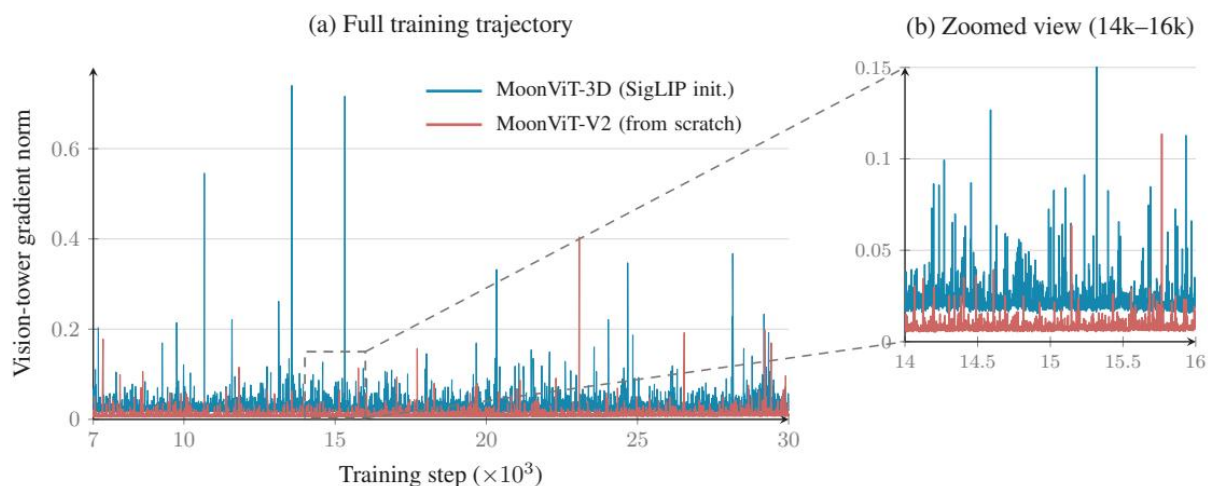


Figure 6: images/6519d88ebb582b620e1fa7ebfe54d16642f9f03a86c699ddf696c158df1d9e84.jpg

Figure 6: Vision-tower gradient norms in our pre-training ablations. Compared with the SigLIP-initialized MoonViT-3D, the from-scratch MoonViT-V2 maintains lower gradient norms with fewer spikes, indicating more stable optimization.

图 6：我们的预训练消融实验中的视觉塔梯度范数。与使用 SigLIP 初始化的 MoonViT-3D 相比，从头开始训练的 MoonViT-V2 保持了更低的梯度范数，并且尖峰更少，表明优化更加稳定。

Architecture This training recipe builds on a vision pathway that follows the overall design of Kimi K2.5 [59, 61]: visual inputs are first encoded by MoonViT-V2 and then mapped by a lightweight MLP projector into the LLM. MoonViT-V2 is a 27-layer vision transformer with roughly 0.4B parameters that adopts RMSNorm and removes all bias terms from its linear and attention projections, a design that further stabilizes the from-scratch optimization above. Images and videos are processed with fully shared parameters, as in MoonViT-3D: attention is factorized into intra-frame spatial and inter-frame temporal passes, and temporal pooling further compresses tokens along the time dimension. Before projection, a pixel-shuffle operation with 2×2 downsampling reduces the number of visual tokens by a factor of four, keeping inputs of up to 3584×3584 pixels affordable within the 1M-token context.

架构这个训练配方基于一个视觉路径，遵循 Kimi K2.5 的整体设计 [59, 61]：视觉输入首先由 MoonViT-V2 进行编码，然后通过一个轻量级的 MLP 投影器映射到 LLM 中。MoonViT-V2 是一个包含约 0.4B 参数的 27 层视觉 Transformer，它采用 RMSNorm，并在其线性和注意力投影中去除了所有的偏置项，这种设计进一步稳定了从头开始的优化过程。图像和视频使用完全共享的参数进行处理，如同 MoonViT-3D 一样：注意力被分解为帧内空间和帧间时间的传递，时间池化进一步沿时间维度压缩令牌。在投影之前，一个带有 2×2 下采样的像素洗牌操作将视觉令牌的数量减少四倍，使得最多 3584×3584 像素的输入在 1M-token 上下文中保持经济可行。

2.5 Per-Head Muon

2.5 每头缪子

Following Kimi K2, Kimi K3 adopts Muon [53] as the optimizer for its matrix parameters. For attention projections, we further refine it into a per-head variant: instead of applying Newton–Schulz orthogonalization to the full Q, K, and V projection matrices, we partition their momentum matrices along the head dimension and orthogonalize each head’s block separately. The intuition is that full-matrix orthogonalization treats all heads as a single coupled block, so heads with larger gradient or momentum scales dominate the shared update direction, while smaller-scale heads receive insufficiently normalized updates; per-head orthogonalization equalizes the update scale across heads. In practice, this design yields more balanced learning dynamics across heads and improves training stability at larger scales. It also slightly reduces optimizer overhead, as Newton–Schulz iterations on tall per-head blocks are cheaper than on the full projection matrix.

在 K2 之后，K3 采用 Muon [53] 作为其矩阵参数的优化器。对于注意力投影，我们进一步将其细化为每个头的变体：而不是对完整的 Q、K 和 V 投影矩阵应用 Newton–Schulz 正交化，我们沿着头的维度分割它们的动量矩阵，并分别对每个头的块进行正交化。其直觉是，完整的矩阵正交化将所有头视为一个耦合的块，因此具有较大梯度或动量尺度的头会主导共享的更新方向，而较小尺度的头则会接收到不足以归一化的更新；每个头的正交化则在各个头之间均衡了更新的尺度。在实践中，这种设计使得各个头之间的学习动态更加平衡，并在更大规模下提高了训练的稳定性。此外，它还略微减少了优化器的开销，因为对高瘦的每个头块进行 Newton–Schulz 迭代比对完整的投影矩阵更便宜。

3 Pre-Training

3 预训练

3.1 Pre-Training Data

3.1 预训练数据

Kimi K3 is pre-trained on a curated corpus spanning four primary text domains—Web Text, Code, Mathematics, and Knowledge—together with a large-scale vision corpus. The vision data covers

captions, interleaved image-text documents, OCR, perception, video, and visual coding data. Our data pipelines build on those developed for Kimi K2 [58] and refined in Kimi K2.5 [59].

Kimi K3 是在涵盖四个主要文本领域（网络文本、代码、数学和知识）的精选语料库上进行预训练的，同时还结合了大规模的视觉语料库。视觉数据包括描述、交错图像-文本文档、OCR、感知、视频和视觉编码数据。我们的数据管道基于 Kimi K2 [58] 中开发的管道，并在 Kimi K2.5 [59] 中进行了优化。

Text data Each domain is filtered by a combination of rule-based heuristics, classifier-based quality scoring, and deduplication, with domain-specific sampling rates determined by ablation studies on smaller models. Following the rephrasing recipe of Kimi K2 [58], we rephrase knowledge and mathematics corpora with style and perspective-diverse prompting, chunk-wise autoregressive generation, and fidelity verification against the source documents.

文本数据每个领域都通过基于规则的启发式方法、基于分类器的质量评分和去重进行过滤，领域特定的采样率由对较小模型的消融研究确定。遵循 Kimi K2 [58] 的重述配方，我们使用风格和视角多样化的提示对知识和数学语料库进行重述，采用分块自回归生成，并通过与源文档的保真度验证来确保准确性。

Vision data The vision corpus follows the taxonomy of Kimi K2.5 [59], combining open-source collections with in-house pipelines for filtering, synthesis, and deduplication. During training, coordinate supervision is provided in both absolute and normalized ([0,1]) formats, enabling precise and resolution-robust localization. In addition to classical text-captioned images, we substantially scale up programmatic multimodal data, coupling code snippets with their rendered visuals across domain-specific formats including SVG, 3D assets, Webpage, Game, and CAD schematics.

视觉数据视觉语料库遵循 Kimi K2.5 [59] 的分类体系，结合开源数据集与内部流程进行过滤、合成和去重。在训练过程中，坐标监督以绝对值和归一化 ([0,1]) 格式提供，从而实现精确且分辨率鲁棒的定位。除了传统的文本描述图像外，我们还大幅扩展了程序化多模态数据，将代码片段与特定领域格式（包括 SVG、3D 资产、网页、游戏和 CAD 图纸）下的渲染图像进行关联。

3.2 Scaling Law

3.2 标度律

Taken together, the architectural, data, and training improvements described in the previous sections define our new model family. Since these changes also alter the optimal training regime, we conduct dedicated scaling-law studies to retune key hyperparameters, including the batch size, learning rate, tokens-per-parameter ratio (TPP) and the model shape. Evaluated on held-out OOD validation data, the scaling law curves in (Fig. 7) show that these improvements collectively deliver an approximately $2.5\times$ gain in overall scaling efficiency over Kimi K2. Table 1 provides a detailed architectural comparison between Kimi K2 and Kimi K3, highlighting the structural changes that contribute to this improvement.

综合前几节中描述的架构、数据和训练方面的改进，定义了我们新的模型家族。由于这些更改还改变了最优的训练方案，我们进行了专门的扩展定律研究，重新调整了关键的超参数，包括批量大小、学习率、每参数令牌比（TPP）以及模型结构。在保留的异常数据集上评估后，图（Fig. 7）中的扩展定律曲线显示，这些改进总体上在整体扩展效率上相比 Kimi K2 实现了大约 $2.5\times$ 的提升。表 1 提供了 Kimi K2 和 Kimi K3 之间详细的架构对比，突出了促成这一改进的结构变化。

Our scaling-law study consistently favors cosine decay over Warmup Stable Decay (WSD) [46], leading us to adopt cosine decay as the default learning rate schedule. We compare cosine decay and WSD under a fixed minimum learning rate. Although prior work has reported that WSD can match or even outperform cosine decay, we observe that the two schedules exhibit markedly different optimal hyperparameters. Even under the same model size and training-token budget, their optimal peak learning rates and batch sizes differ substantially. As a result, comparing the two schedules using a shared set of hyperparameters may unfairly favor one simply because those hyperparameters are better aligned with it. To ensure a fair comparison, we conduct an independent scaling-law search for each schedule. Under their respective optimal hyperparameter

settings, cosine decay consistently achieves a lower final loss than WSD.

我们的缩放定律研究始终倾向于使用余弦衰减而不是 Warmup Stable Decay (WSD) [46]，因此我们采用余弦衰减作为默认的学习率调度策略。我们在固定最小学习率的情况下比较了余弦衰减和 WSD。尽管先前的研究报告称 WSD 可以匹配甚至优于余弦衰减，但我们观察到这两种调度策略具有明显不同的最佳超参数。即使在相同的模型规模和训练 token 预算下，它们的最佳峰值学习率和批量大小也有显著差异。因此，使用共享的超参数集来比较这两种调度策略可能会因为这些超参数更符合其中一种而不公平地偏袒它。为了确保公平比较，我们为每种调度策略分别进行独立的缩放定律搜索。在各自的最佳超参数设置下，余弦衰减始终比 WSD 实现更低的最终损失。

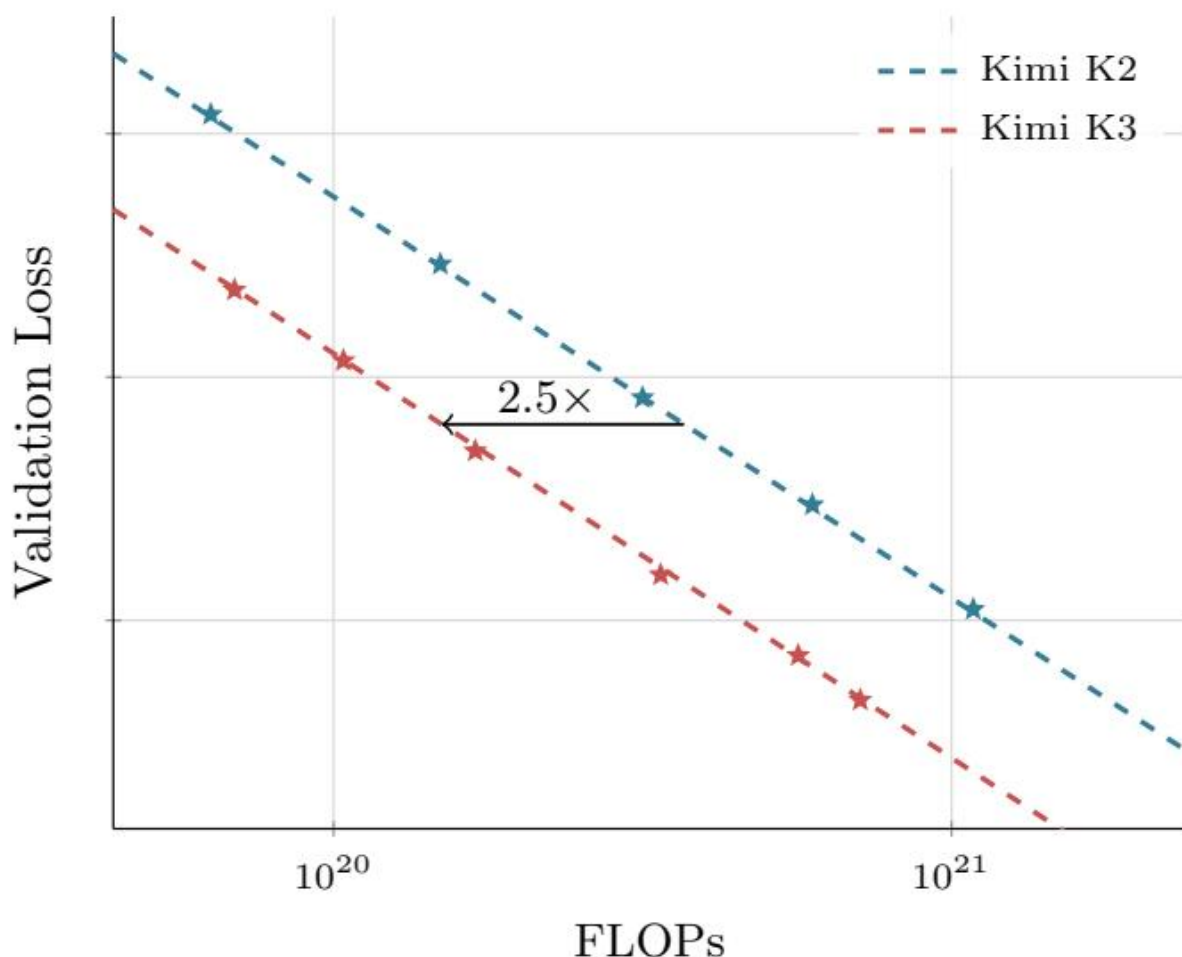


Figure 7: images/e2344c09c9ed0d187fcb218dda3f4ffe68d806c243ec46b72192e5421c190c81.jpg

Figure 7: Fitted scaling-law curves for Kimi K2 and Kimi K3. Kimi K3 achieves $2.5\times$ gain in scaling efficiency over Kimi K2.

图 7: Kimi K2 和 Kimi K3 的拟合标度律曲线。Kimi K3 在扩展效率上比 Kimi K2 实现了 $2.5\times$ 的提升。

Table 1: Architectural comparison between Kimi K2 and Kimi K3.

表 1: Kimi K2 与 Kimi K3 的架构对比。

	Kimi K2	Kimi K3	Δ
Architecture	MoE	MoE	-
#Layers	61	93	$\uparrow 52\%$

Total Parameters	1.04T	2.78T	□167%
Activated Parameters	32.6B	104.2B	□220%
Hidden Dimension	7,168	7,168	=
Latent MoE Dimension	-	3584 (0.5×)	-
MoE Hidden Dimension per Expert	2,048	3,072	□50%
Routed Experts	384	896	□133%
Experts Active per Token	8	16	□100%
Shared Experts	1	2	□100%
Attention Heads	64	96	□50%
Number of Dense Layers	1	1	=
Vocabulary Size	160K	160K	=
Training Context Length	128K	1M	8×
Attention Mechanism	MLA	Hybrid KDA-MLA	-
Activation Function	SwiGLU	SiTU-GLU	-
Attention-Layer Composition	61 MLA	69 KDA + 24 MLA	-
Number of MTP Layers	1 layer	1 layer	=
Total Parameters of ViT	-	401M	-
#ViT Layers	-	27 layers	-
Patch Size of ViT	-	14	-
#Attention Heads of ViT	-	12	-

	Kimi K2	Kimi K3	Δ
架构	MoE	MoE	-
# 图层	61	93	□52%
总参数数	1.04T	2.78T	□167%
激活参数	32.6B	104.2B	□220%
隐藏维度	7,168	7,168	=
潜在的 MoE 维度	-	3584 (0.5×)	-
每个专家的 MoE 隐藏维度	2,048	3,072	□50%
路由专家	384	896	□133%
按代币划分的专家活跃度	8	16	□100%
共享专家	1	2	□100%
注意头部	64	96	□50%
密集层的数量	1	1	=
词汇量	160K	160K	=
训练上下文长度	128K	1M	8×
注意力机制	MLA	混合 KDA-MLA	-
激活函数	SwiGLU	SiTU-GLU	-
注意力层结构	61 MLA	69 KDA + 24 MLA	-
MTP 层的数量	1 层	1 层	=

ViT 的总参数量	-	401M	-
#ViT 层	-	27 层	-
ViT 的图像块大小	-	14	-
# 注意 ViT 的头部	-	12	-

3.3 Training Recipe

3.3 训练方案

Kimi K3 adopts a native multimodal training strategy in which language and vision are jointly optimized from the start of training, rather than grafting a vision encoder onto a pre-trained language model through a post-hoc alignment stage. Under this paradigm, visual and textual tokens are interleaved within a single next-token prediction objective, enabling the shared backbone to learn unified multimodal representations from the outset.

Kimi K3 采用了一种原生的多模态训练策略，在训练开始时就对语言和视觉进行联合优化，而不是通过后处理对齐阶段将视觉编码器嫁接到预训练的语言模型上。在这一范式下，视觉和文本标记在一个单一的下一个标记预测目标中交错，使共享的主干网络能够从一开始就学习统一的多模态表示。

We optimize the model using the Per-Head Muon optimizer (§ 2.5) together with the weight-clipping mechanism introduced in Kimi K2, while adopting QB (§ 2.3.3) for MoE load balancing. We use a cosine learning rate schedule with a 1% linear warmup. Weight decay is set to 0.1 throughout.

我们使用 Per-Head Muon 优化器 (§ 2.5) 以及 Kimi K2 中引入的权重裁剪机制来优化模型，同时采用 QB (§ 2.3.3) 进行 MoE 负载均衡。我们使用余弦学习率调度，并采用 1% 的线性预热。权重衰减在整个训练过程中设置为 0.1。

Our pre-training begins with a context length of 8k tokens, which is later extended to 64k tokens in a subsequent training phase.

我们的预训练最初使用 8k 个 token 的上下文长度，随后在后续的训练阶段扩展到 64k 个 token。

3.4 Long-Context Extension

3.4 长上下文扩展

Positional encoding Kimi K3 uses no explicit positional embedding (NoPE), and instead encodes positional information implicitly through the recurrent gating and decay mechanism of KDA. As a result, the model extrapolates directly to 1M-token contexts without any positional-encoding modification, such as RoPE rescaling or interpolation [92].

位置编码 Kimi K3 不使用显式的 positional embedding (NoPE)，而是通过 KDA 的递归门控和衰减机制隐式地编码位置信息。因此，该模型可以直接扩展到 1M-token 的上下文，而无需任何位置编码的修改，例如 RoPE 缩放或插值 [92]。

Long-context data Long documents and videos from natural sources contain a substantial amount of low-quality content, including near-duplicates, binary blobs, truncated files, video clips, and invalid machine-generated logs. We therefore process them through a dedicated cleaning pipeline that combines exact and fuzzy deduplication, supplemented by perceptual hashing over frames for video, together with heuristic and classifier-based quality filtering, and structural validation. Because genuinely long and coherent documents and videos are

scarce relative to short text, we upsample them so that the long-context distribution is not overwhelmed by short sequences during cooldown. Length alone, however, does not confer long-range capability. To address this, we synthesize additional long-context data by carefully permuting and concatenating multimodal documents and sub-tasks, so that the embedded tasks can be solved only by attending to information scattered across the full 1M-token context. This trains the attention mechanism at the intended scale and prevents it from degenerating into local patterns.

长上下文数据长文档和视频来自自然来源，包含大量低质量内容，包括近似重复内容、二进制块、截断文件、视频片段和无效的机器生成日志。因此，我们通过一个专用的清理流程来处理这些数据，该流程结合了精确和模糊去重，辅以视频帧的感知哈希，同时结合启发式和基于分类器的质量过滤以及结构验证。由于真正长且连贯的文档和视频相对于短文本来说非常稀少，我们对其进行上采样，以确保在冷却期间长上下文分布不会被短序列所淹没。然而，仅凭长度本身并不能带来长距离的能力。为了解决这个问题，我们通过仔细排列和连接多模态文档和子任务来合成额外的长上下文数据，使得嵌入的任务只能通过关注整个 100 万标记上下文中分散的信息来解决。这在预期的规模上训练注意力机制，并防止其退化为局部模式。

Progressive context extension Kimi K3 supports a context window of up to 1 million tokens. We achieve this through extending the context window progressively as training proceeds, following a four-stage curriculum. The window grows from 8K to 64K tokens during pre-training, and from 256K to 1M tokens during the cooldown phase. Concentrating the costly long-sequence computation within a small fraction of the overall training budget keeps the curriculum economical while still allowing the model to adapt gradually to increasingly long-range dependencies. The sequence-dimension partitioning that makes million-token training tractable for the KDA layers is described in §5.1.2.

渐进式上下文扩展 Kimi K3 支持高达 100 万 token 的上下文窗口。我们通过在训练过程中逐步扩展上下文窗口来实现这一点，遵循一个四阶段的课程安排。在预训练阶段，窗口从 8K 扩展到 64K token；在冷却阶段，窗口从 256K 扩展到 1M token。将昂贵的长序列计算集中在整体训练预算中的一小部分，使课程安排更加经济，同时仍允许模型逐步适应更长的依赖关系。关于使百万 token 训练在 KDA 层中可行的序列维度划分方法，请参见 §5.1.2。

4 Post-Training

4 后训练

4.1 Method

4.1 方法

Our post-training pipeline follows a three-stage paradigm: initializing baseline agent capabilities via supervised fine-tuning (SFT), developing specialized domain experts at varying reasoning effort via Reinforcement Learning (RL), and consolidating these domain-specific policies into a single model using Multi-Teacher On-Policy Distillation (MOPD).

我们的训练后管道遵循一个三阶段范式：通过监督微调（SFT）初始化基线代理能力，通过强化学习（RL）开发不同推理努力水平的专用领域专家，并使用多教师在线策略蒸馏（MOPD）将这些领域特定策略整合到一个模型中。

4.1.1 Supervised Fine-Tuning

4.1.1 监督微调

The SFT stage establishes a high-quality cold-start policy for the subsequent RL stage. Building on the SFT pipeline of previous Kimi models [58, 59], we expand the SFT dataset for Kimi

K3, substantially broadening its coverage of complex agentic tasks. Specifically, we synthesize data trajectories using domain-specialized models from the prior Kimi series, followed by multi-stage verification and human-in-the-loop annotation. To represent these complex agentic trajectories consistently, we serialize all data with our XHTML-based chat template (eXtensible Token Markup Language; see § F for details). Collectively, these steps yield a large-scale instruction dataset that endows Kimi K3 with adaptive reasoning, precise tool calling, and robust execution in long-horizon agentic scenarios. In addition, we apply quantization-aware training (QAT) from the SFT stage onward, with MXFP4 weights and MXFP8 activations (§4.1.4).

SFT 阶段为后续的 RL 阶段建立了一个高质量的冷启动策略。基于之前 Kimi 模型的 SFT 流程 [58, 59]，我们扩展了 Kimi K3 的 SFT 数据集，大幅提升了其在复杂代理任务方面的覆盖范围。具体而言，我们使用之前 Kimi 系列中的领域专用模型合成数据轨迹，随后进行多阶段验证和人类参与的标注。为了保持这些复杂代理轨迹的一致性，我们使用基于 XHTML 的聊天模板（可扩展标记语言；详见第 F 节）对所有数据进行序列化。这些步骤共同产生了一个大规模的指令数据集，使 Kimi K3 具备在长时地平线代理场景中进行自适应推理、精确工具调用和稳健执行的能力。此外，我们从 SFT 阶段开始应用量化感知训练 (QAT)，使用 MXFP4 权重和 MXFP8 激活 (§4.1.4)。

4.1.2 Reinforcement Learning

4.1.2 强化学习

While SFT provides a solid cold-start foundation, RL is critical to unlocking higher-order reasoning and execution capabilities. Rather than training specialized RL models for individual tasks, we scale RL across three broad domains, each encompassing a wide spectrum of sub-tasks, and train a single expert for each domain at every reasoning effort level: (i) general tasks, spanning general experience, vision, reasoning, faithfulness, search capabilities, and knowledge work tasks; (ii) general agents, spanning long-horizon assistant tasks, deep research, and paragraph-level writing; and (iii) coding agents, spanning software engineering (SWE), coding experience, kernel tasks, and web development. As shown in Figure 8, scaling RL FLOPs consistently improves a variety of capabilities across knowledge, reasoning, vision, general agent, and coding. Crossing these three domain experts with three reasoning effort levels in {low, high, max} yields a total of nine expert models.

虽然 SFT 为冷启动提供了坚实的基础，但 RL 对于解锁更高层次的推理和执行能力至关重要。我们并未为每个单独任务训练专门的 RL 模型，而是将 RL 扩展到三个广泛的领域，每个领域涵盖了一系列子任务，并在每个推理努力级别上为每个领域训练一个专家：(i) 通用任务，涵盖一般经验、视觉、推理、忠实性、搜索能力以及知识型工作任务；(ii) 通用代理，涵盖长视野助手任务、深入研究和段落级写作；(iii) 编码代理，涵盖软件工程 (SWE)、编码经验、内核任务和网络开发。如图 8 所示，扩展 RL 的 FLOPs 持续提升知识、推理、视觉、通用代理和编码等多种能力。将这三个领域专家与三个推理努力级别 {低、高、最大} 进行组合，总共得到了九个专家模型。

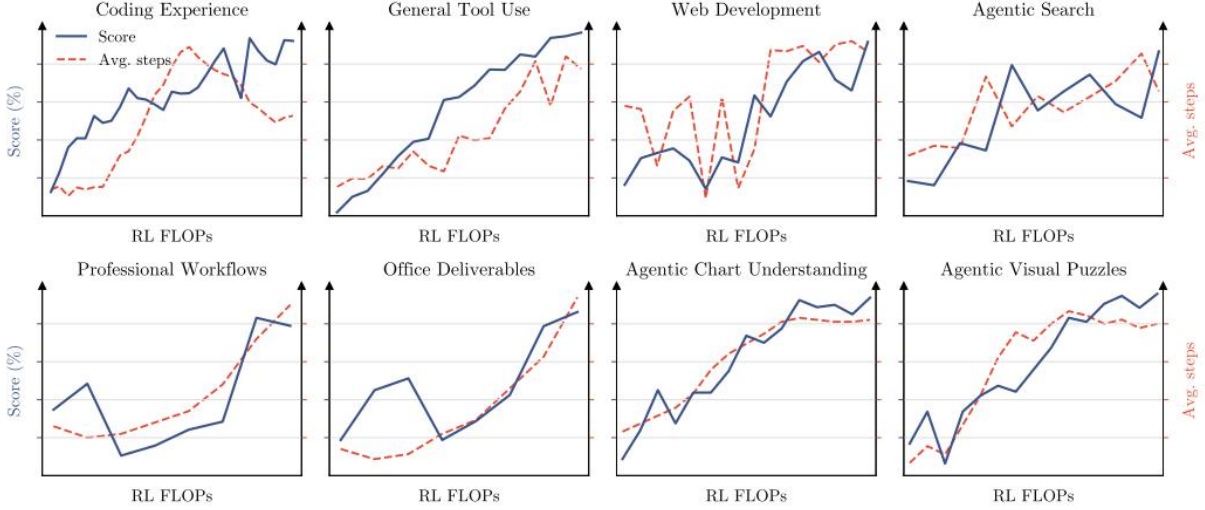


Figure 8: images/701dec29ad6c5f01d30ef473221bae67b891b477743e30265df374ef4ecfe1c5.jpg

Figure 8: Scores and the average assistant steps across a variety of public and in-house evaluations during RL. By scaling RL FLOPs, tool-call steps scale up consistently, accompanied by a comprehensive improvement in the model’s overall capability.

图 8：在强化学习（RL）过程中，跨多种公开和内部评估的得分以及平均助手步骤。通过扩展 RL FLOPs，工具调用步骤会持续增加，同时模型的整体能力也得到全面提升。

Algorithm To mitigate the long-tail latency that intensifies in long-horizon tasks, we extend the partial rollout scheme from our synchronous RL framework [118, 59]. During the rollout phase of each iteration, we sample K completions for each of N prompts, maintaining an active workload of $N \times K$ trajectories. Rather than waiting for all rollouts to terminate, the generation phase pauses as soon as a fraction $\lambda \in (0, 1)$ of trajectories completes (i.e., $\lambda N K$), allowing policy optimization to proceed without execution stragglers. Paused rollouts are enqueued and prioritized for resumption at the start of the next iteration, powered by our sandbox infrastructure (§ 5.3.2). Once all K responses for a prompt complete, they are immediately dispatched for policy optimization, which follows the algorithm in Kimi K2.5 [59]. Under our partial rollout scheme, an individual long-horizon trajectory naturally spans multiple iterations, introducing data staleness that threatens training stability. Our policy optimization algorithm inherently tolerates such an extreme off-policy regime through a per-token regularization. By constraining policy updates within a localized neighborhood, this regularization enables the algorithm to robustly handle highly stale data and sustain training stability.

算法为缓解在长期任务中加剧的长尾延迟问题，我们从同步强化学习框架 [118, 59] 中扩展了部分 rollout 方案。在每次迭代的 rollout 阶段，我们为每个提示词 N 采样 K 个完成结果，保持 $N \times K$ 条轨迹的活跃工作负载。而不是等待所有 rollout 都完成，生成阶段会在一定比例 $\lambda \in (0, 1)$ 的轨迹完成时暂停（即 $\lambda N K$ ），从而允许策略优化在没有执行延迟的情况下继续进行。暂停的 rollout 会被加入队列，并在下一次迭代开始时优先恢复，由我们的沙盒基础设施 (§ 5.3.2) 提供支持。一旦某个提示词的所有 K 个响应完成，它们会立即被派发进行策略优化，该优化过程遵循 Kimi K2.5 [59] 中的算法。在我们的部分 rollout 方案中，单个长期轨迹自然跨越多个迭代，引入数据陈旧问题，这会威胁训练的稳定性的。我们的策略优化算法通过每 token 的正则化，内在容忍这种极端的离策略模式。通过将策略更新限制在局部邻域内，这种正则化使得算法能够稳健地处理高度陈旧的数据并维持训练的稳定性的。

Reasoning Effort RL To fine-tune reasoning effort while maximizing token efficiency, we implement a per-problem budget control mechanism during RL [59]. We associate each problem x with an initial token budget $b_0(x)$ estimated from the cold-start model, and override the task reward with -1 for trajectories whose total token budget $T(y)$ exceeds a scaled threshold $\tau \cdot b_0(x)$. For general tasks, $T(y)$ measures the number of thinking tokens, whereas for agentic tasks,

$T(y)$ accounts for the cumulative output tokens, including both reasoning traces and tool-call arguments. Training follows a stage-wise curriculum over the budget multiplier τ . We first train a max-budget variant with a relatively large τ , while still capping the maximum budget to suppress excessive overthinking. We then anneal τ to smaller values to obtain the high- and low-effort expert models. The adjustment of τ is configured per domain under human-in-the-loop guidance. Trajectories produced by the resulting experts at all reasoning levels are jointly collected for supervised fine-tuning and multi-teacher on-policy distillation.

推理努力强化学习 (Reasoning Effort RL) 为了在最大化 token 效率的同时微调推理努力, 我们在强化学习 [59] 过程中实现了一个基于问题的预算控制机制。我们将每个问题 x 与一个初始 token 预算 $b_0(x)$ 关联, 该预算从冷启动模型中估计得出, 并且对于总 token 预算 $T(y)$ 超过一个缩放阈值 $\tau \cdot b_0(x)$ 的轨迹, 我们将任务奖励覆盖为 -1。对于一般任务, $T(y)$ 衡量思考 token 的数量, 而对于代理任务, $T(y)$ 考虑累积输出 token, 包括推理轨迹和工具调用参数。训练过程遵循预算乘数 τ 的分阶段课程。我们首先训练一个最大预算变体, 使用相对较大的 τ , 同时仍然限制最大预算以抑制过度思考。然后我们将 τ 逐渐减小到较小的值, 以获得高努力和低努力专家模型。 τ 的调整在人类指导下根据领域进行配置。所有推理层级下由所得专家生成的轨迹共同收集, 用于监督微调和多教师在线蒸馏。

Agentic Generative Reward Model For non-verifiable general tasks, we adopt an Agentic Generative Reward Model (GRM), retaining the tournament-style group reward with binary comparisons as in Kimi K2.5 [58, 59]. Beyond generic agentic capabilities for enhanced judgment, the agentic judge is required to follow a mandatory protocol: (1) read the outcome, product, or text output; (2) generate a rubric; (3) score each candidate against the rubric; and (4) record the rubric-assigned scores in a scorepad. To mitigate reward hacking toward increasingly verbose outputs, we apply a budget-based verbosity control analogous to the reasoning-effort control above: given an initial verbosity ℓ_0 estimated from the cold-start model and a multiplier σ , a candidate whose output length exceeds $\sigma \cdot \ell_0$ automatically loses the binary comparison.

对于不可验证的一般任务, 我们采用一种代理生成奖励模型 (GRM), 保留了类似于 Kimi K2.5 的竞赛式群体奖励机制, 通过二元比较进行评估 [58, 59]。除了增强判断力的一般代理能力外, 代理裁判员还需遵循强制性协议: (1) 阅读结果、产品或文本输出; (2) 生成评分标准; (3) 根据评分标准对每个候选者进行评分; (4) 在评分簿中记录评分标准所赋予的分数。为缓解奖励黑客行为, 使输出更加冗长, 我们采用基于预算的冗长度控制机制, 类似于上述的推理努力控制: 给定初始冗长度 ℓ_0 由冷启动模型估计, 以及一个乘数 σ , 如果候选者的输出长度超过 $\sigma \cdot \ell_0$, 则自动在二元比较中被淘汰。

4.1.3 Multi-Teacher On-Policy Distillation

4.1.3 多教师在线策略蒸馏

We adopt Multi-Teacher On-Policy Distillation (MOPD) to consolidate these domain-specialized capabilities across varying reasoning efforts into a unified model [75, 134, 29]. During training, for a given domain d and a sampled reasoning effort level $e \in \{low, high, max\}$, optimization is guided by the corresponding teacher model $\pi_{teacher}^{(d,e)}$ among the nine experts. Given an input query x and the prefix response $y_{<t}$, the per-token OPD reward evaluated on y_t between

我们采用多教师在线策略蒸馏 (MOPD) 将这些领域专用能力整合到一个统一的模型 [75, 134, 29] 中。在训练过程中, 对于给定的领域 d 和一个采样的推理努力水平 $e \in \{low, high, max\}$, 优化由九个专家中的相应教师模型 $\pi_{teacher}^{(d,e)}$ 引导。给定一个输入查询 x 和前缀响应 $y_{<t}$, 在 y_t 上评估的每 token OPD 奖励在之间

the teacher $\pi_{teacher}^{(d,e)}$ and the student π_θ is defined as:

老师 $\pi_{teacher}^{(d,e)}$ 和学生 π_θ 被定义为:

$$r_{\text{opd}}^d(y_t \mid e, x, y_{<t}) = \text{clip} \left(\text{sg} \left(\log \frac{\pi_{\text{teacher}}^{(d,e)}(y_t \mid x, y_{<t})}{\pi_{\theta}(y_t \mid e, x, y_{<t})} \right), -R_{\max}, R_{\max} \right), \quad (15)$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operator, and $R_{\max} > 0$ is a clipping threshold to constrain extreme advantage signals, thereby stabilizing RL training. This dense reward signal seamlessly integrates into our RL framework, naturally enabling infrastructure-level optimizations such as partial rollout training for long-horizon tasks. While we also experimented with more fine-grained top-k distillation objectives, we observed no clear advantage in either convergence speed or final performance in our setting.

其中 $\text{sg}(\cdot)$ 表示停止梯度操作符，而 $R_{\max} > 0$ 是一个裁剪阈值，用于限制极端优势信号，从而稳定强化学习（RL）训练。这种密集的奖励信号可以无缝地融入我们的 RL 框架中，自然地实现诸如长时间跨度任务的部分 rollout 训练等基础设施级优化。尽管我们还尝试了更细粒度的 top-k 蒸馏目标，但在我们的设置中，我们并未观察到在收敛速度或最终性能方面有明显的优势。

4.1.4 Deployment-Aware Post-Training

4.1.4 部署感知的后训练

MXFP4 Quantization-Aware Post-Training To reduce memory footprint and serving cost at deployment, we quantize the MoE expert weights — which dominate the model’s parameter memory — to MXFP4 [103], with activations computed in MXFP8, while all non-expert components (attention projections, latent MoE projections, shared experts, and MoE routers) remain in higher precision. We perform quantization-aware training (QAT) [49] throughout the entire post-training stage, covering both SFT and RL, so that the model adapts to quantization-induced precision loss. During RL, rollout and training share the same quantization scheme — eliminating the train-inference mismatch.

MXFP4 量化感知的微调训练 为了在部署时减少内存占用和推理成本，我们将 MoE 专家权重（这些权重占用了模型参数内存的大部分）量化为 MXFP4 [103]，而激活值则使用 MXFP8 计算，所有非专家组件（注意力投影、潜在 MoE 投影、共享专家和 MoE 路由器）则保持更高的精度。我们在整个微调阶段进行量化感知训练（QAT）[49]，涵盖 SFT 和 RL，使模型适应量化导致的精度损失。在 RL 过程中，rollout 和训练使用相同的量化方案——消除了训练与推理之间的不匹配。

Draft Model Fine-Tuning Optimizing inference efficiency is crucial for serving complex, long-horizon agentic models. Kimi K3 is pre-trained with a multi-token-prediction (MTP) layer that mirrors the structure of a backbone block. As the draft model of EAGLE-3 [71] comprises a single decoder layer whose structure matches the MTP layer, we fine-tune the pre-trained MTP layer into an EAGLE-3-style draft model, with the target model frozen and only the draft layer and its feature-fusion projection updated. Following the training-time test protocol of EAGLE-3, the draft is unrolled for seven steps during training; beyond the first step, where the target-side features of the newest position are unavailable, the draft consumes its own outputs from earlier steps, mirroring the recurrent drafting procedure at inference.

优化推理效率 对于服务复杂、长周期的代理模型至关重要。Kimi K3 采用多标记预测（MTP）层进行预训练，该层的结构与主干块的结构相匹配。由于 EAGLE-3 的草稿模型 [71] 包含一个结构与 MTP 层相匹配的单个解码器层，我们对预训练的 MTP 层进行微调，使其成为 EAGLE-3 风格的草稿模型，目标模型保持冻结状态，仅更新草稿层及其特征融合投影。遵循 EAGLE-3 的训练时测试协议，在训练过程中草稿模型展开七步；在第一步之后，由于最新位置的目标侧特征不可用，草稿模型会消耗其自身早期步骤的输出，从而在推理过程中模拟递归起草过程。

The draft input fuses low-, mid-, and high-level features of the target model, taken from the outputs of the 1st, 4th, and final AttnRes blocks, respectively (§ 2.2). These features are concatenated and projected to the hidden size by a bias-free matrix W_{E3} , initialized as $\begin{bmatrix} 0 & 0 & I \end{bmatrix}$ so that the fused representation coincides at initialization with the high-level feature h_h — the input on

which the MTP layer was pre-trained — and gradually learns to incorporate the low- and mid-level features during fine-tuning.

草稿输入融合了目标模型的低级、中级和高级特征，这些特征分别取自第 1 个、第 4 个和最后一个 AttnRes 块的输出 (§ 2.2)。这些特征被连接起来，并通过一个无偏矩阵 W_{E3} 投影到隐藏大小，该矩阵初始化为 $\begin{bmatrix} 0 & 0 & I \end{bmatrix}$ ，使得融合表示在初始化时与高级特征 h_h 一致——即 MTP 层预训练所使用的输入——并在微调过程中逐渐学习融入低级和中级特征。

The speedup of speculative decoding is governed by the per-token acceptance rate $\sum_{x \in \mathcal{V}} \min(p(x), q(x))$ under lossless speculative sampling, where p and q denote the next-token distributions of the target and draft models. Since minimizing the conventional KL-divergence surrogate does not guarantee maximizing this rate for a capacity-limited draft model, we directly optimize the likelihood-based LK loss [104], the negative logarithm of the acceptance rate itself,

推测解码的速度提升由无损推测采样下的每 token 接受率 $\sum_{x \in \mathcal{V}} \min(p(x), q(x))$ 决定，其中 p 和 q 分别表示目标模型和草稿模型的下一个 token 分布。由于最小化传统的 KL 散度替代目标并不能保证在容量受限的草稿模型中最大化该接受率，因此我们直接优化基于似然的 LK 损失 [104]，即接受率本身的负对数。

$$\mathcal{L}_{\text{LK}} = -\log \sum_{x \in \mathcal{V}} \min(p(x), q(x)), \quad (16)$$

with p and q evaluated at temperature 1 and no auxiliary ground-truth cross-entropy term. Draft fine-tuning follows the post-training QAT configuration (§ 4.1.4), with MoE expert weights in MXFP4 and their input activations in MXFP8, while non-expert modules remain in higher precision.

在温度为 1 的情况下评估 p 和 q ，并且不包含辅助真实值交叉熵项。微调草案遵循后训练 QAT 配置 (§ 4.1.4)，其中 MoE 专家权重使用 MXFP4，其输入激活使用 MXFP8，而非专家模块则保持更高的精度。

4.2 RL Task Synthesis and Agentic Environments

4.2 RL 任务合成与代理环境

The effectiveness of our RL framework relies heavily on rich, diverse, and robustly verifiable environments. To support scalable training across complex long-horizon tasks, we design a series of specialized white-box environments and task synthesis paradigms.

我们的强化学习框架的效果在很大程度上依赖于丰富、多样且可稳健验证的环境。为了支持在复杂长时任务中的可扩展训练，我们设计了一系列专门的白盒环境和任务合成范式。

4.2.1 Unified White-Box RL Environment

4.2.1 统一的白盒强化学习环境

Training with a single fixed agent harness can cause a model to overfit to a particular tool schema, system prompt, context management mechanism, or interaction protocol. To address this, we develop a unified white-box RL environment that represents an agent harness as a collection of configurable, composable modules, including tool interfaces, system prompts, context management strategies, skills, memories, subagents, and other components. Composing these modules through configuration, the environment can instantiate mainstream harnesses such as Kimi Code [56], Claude Code [15], Codex [20], OpenClaw [86], and Hermes [44], as well as entirely new ones. During RL training, we dynamically

使用单一固定代理框架进行训练可能导致模型过度适应特定的工具架构、系统提示、上下文管理机制或交互协议。为了解决这个问题，我们开发了一个统一的白盒强化学习环境，该环境将代理框架表示为一组可配置、可组合的模块集合，包括工具接口、系统提示、上下文管理策略、技能、记忆、子代理以及其他组件。通过配置这些模块，环境可以实例化主流框架，如 Kimi Code [56]、Claude Code [15]、Codex [20]、OpenClaw [86] 和 Hermes [44]，以及全新的框架。在强化学习训练过程中，我们动态地

construct different harness configurations for different task groups, exposing Kimi K3 to diverse combinations of these modules rather than the conventions of any single harness. The same abstraction also readily supports RL across various task domains, providing a scalable foundation for training more general-purpose agents.

为不同的任务组构建不同的 harness 配置，将 Kimi K3 暴露在这些模块的各种组合中，而不是任何单一 harness 的惯例。同样的抽象也能够轻松支持在各种任务领域中的强化学习（RL），为训练更具通用性的代理提供可扩展的基础。

4.2.2 Knowledge-Graph-Guided Task Synthesis

4.2.2 知识图引导的任务综合

Motivation and overview The quality and diversity of post-training tasks are largely determined by their source materials. Retrieval guided by fine-grained concepts surfaces specialized and underrepresented knowledge, while sampling across diverse concepts broadens domain coverage. To control both granularity and coverage at scale, we build a self-evolving, hierarchically organized knowledge graph that agents continuously expand through web-scale exploration across knowledge-intensive and coding domains. Figure 9 illustrates the task synthesis pipeline.

动机与概述任务的质量和多样性在很大程度上取决于其来源材料。基于细粒度概念的检索能够揭示专门化和较少被关注的知识，而跨多样概念的采样则能扩大领域覆盖范围。为了在大规模范围内同时控制粒度和覆盖范围，我们构建了一个自我演进、层级组织的知识图谱，智能体通过在知识密集型和编程领域的大规模网络探索不断扩展该图谱。图 9 展示了任务合成的流程。

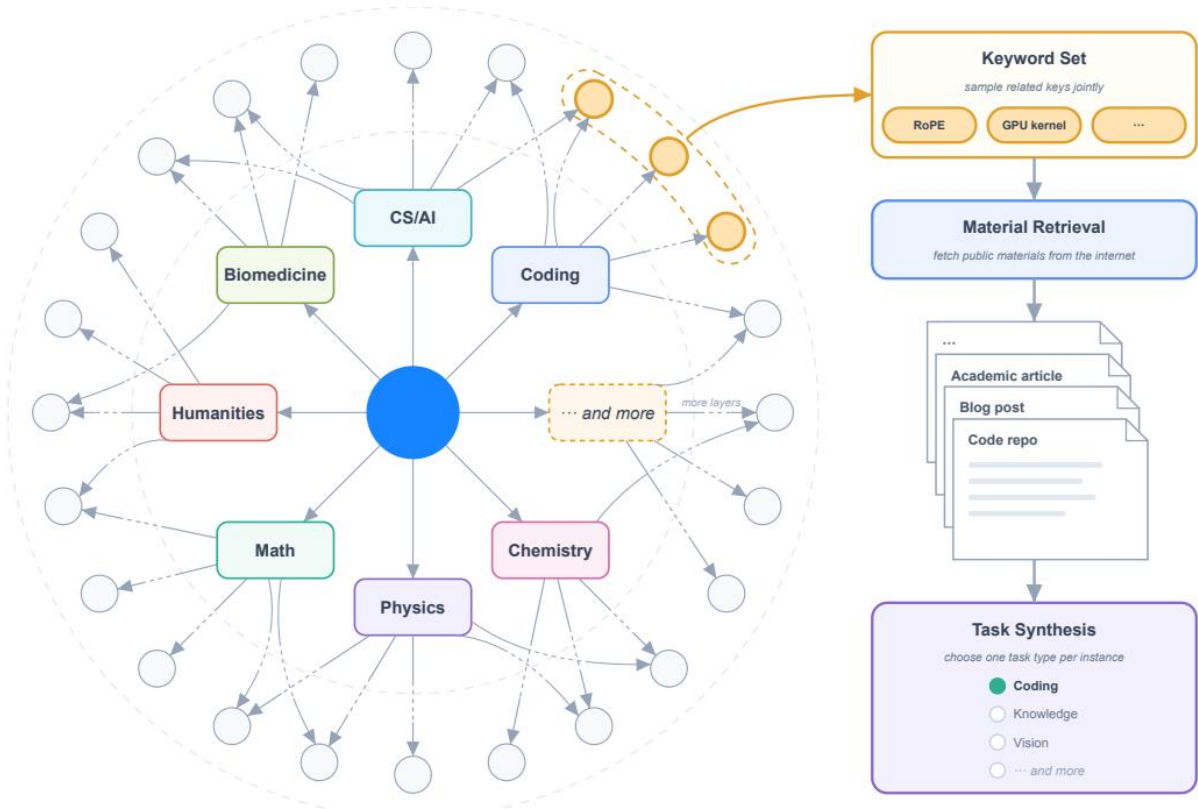


Figure 9: images/eead63b491ccda6eaea2b2207753019f30ac3f5d7aa8c1e50fc4c306a46e49e9.jpg

Figure 9: Overview of knowledge-graph-guided task synthesis. The hierarchically organized knowledge graph represents concepts at multiple levels, ranging from broad domains to fine-grained concepts. Related nodes are sampled to form a keyword set that guides the retrieval of publicly available source materials. For each synthesis instance, the system selects a task type and uses the retrieved materials to synthesize a corresponding task.

图 9：知识图谱引导的任务合成概述。层次化组织的知识图谱表示了不同层次的概念，从广泛领域到细粒度概念。相关节点被采样以形成关键词集，从而引导检索公开可用的源材料。对于每个合成实例，系统选择一个任务类型，并利用检索到的材料合成相应任务。

Agentic knowledge graph construction We construct the knowledge graph as a directed acyclic graph through recursive, agent-driven expansion. The expansion process begins with a predefined set of coarse-grained seed nodes. An agent instance is then assigned to each node and performs multiple web searches to investigate the corresponding concept. Before adding new nodes, the agent explores the existing graph to identify equivalent or related concepts, reuse existing nodes where appropriate, and minimize duplication. Edges are always directed from the coarser concept to the finer one, regardless of which endpoint the agent discovers first. Newly added nodes are subsequently assigned to agents for further exploration. A branch stops expanding when the assigned agent determines that the current concept is sufficiently atomic.

代理知识图谱构建我们通过递归、以代理驱动的方式扩展，构建知识图谱为有向无环图。扩展过程从预定义的一组粗粒度种子节点开始。然后为每个节点分配一个代理实例，并执行多次网络搜索以调查相应的概念。在添加新节点之前，代理会探索现有的图，以识别等价或相关的概念，在适当的情况下重用现有节点，并尽量减少重复。边始终从较粗的概念指向更细的概念，无论代理先发现哪个端点。新添加的节点随后被分配给代理进行进一步探索。当分配的代理确定当前概念已经足够原子化时，该分支停止扩展。

Material retrieval and task synthesis To target a desired distribution across domains and task types, the system samples nodes at varying levels of granularity, either individually or in related

combinations. Keywords derived from the sampled nodes are combined with contextual information from their ancestors in the knowledge graph to formulate web queries. The retrieved real-world materials are assembled so that a synthesis agent produces training tasks of various task types.

材料检索与任务合成为了在不同领域和任务类型之间实现期望的分布，系统以不同粒度级别采样节点，无论是单独采样还是在相关组合中采样。从采样节点中提取的关键词与知识图谱中祖先节点的上下文信息相结合，以生成网络查询。检索到的实际材料被组装起来，以便合成代理生成各种类型的任务训练任务。

4.2.3 Verifiable Problems in Agentic Environments

4.2.3 代理环境中的可验证问题

We train Kimi K3 on verifiable problems in agentic environments; representative examples include multi-step complex information searching, where the model plans its research, gathers evidence from the web step by step, and produces

我们在可验证的问题上训练 Kimi K3，在代理环境中；代表性例子包括多步骤的复杂信息搜索，其中模型计划其研究，逐步从网络上收集证据，并产生

a verifiable answer; the real day-to-day work of professionals, such as investment banking, data analysis, and legal practice, where the model decomposes a complex request, operates domain tools in a sandbox, and completes a deliverable over dozens to hundreds of steps; and multi-step verifiable visual reasoning over STEM problems, visual puzzles, and chart understanding. Each visual-reasoning trajectory is generated in an agent environment equipped with a Python interpreter in an isolated sandbox: the model iteratively writes and executes code to crop, zoom, or transform the input image, perform precise computation, or verify intermediate results, and receives the execution outputs — including generated images — as new observations over multiple interaction steps. As the model learns to perform more image operations and collect more observations, its performance on complex visual reasoning tasks steadily improves.

一个可验证的答案；专业人士日常工作的实际内容，例如投资银行、数据分析和法律实务，其中模型会分解复杂请求，在沙箱中操作领域工具，并在几十到几百个步骤中完成交付成果；以及在 STEM 问题、视觉谜题和图表理解上的多步骤可验证视觉推理。每条视觉推理轨迹都是在一个配备 Python 解释器的隔离沙箱代理环境中生成的：模型会迭代地编写和执行代码，以裁剪、缩放或转换输入图像，进行精确计算或验证中间结果，并在多次交互步骤中接收执行输出——包括生成的图像——作为新的观察结果。随着模型学习执行更多的图像操作并收集更多的观察结果，其在复杂视觉推理任务上的表现会稳步提高。

4.2.4 Kernel Optimization Tasks

4.2.4 内核优化任务

To strengthen Kimi K3's GPU kernel optimization capabilities, we build a large-scale suite of kernel tasks ranging from single-operator kernels to fused mega-kernels, sourced from high-quality GitHub repositories such as Flash Linear Attention [139]. The suite spans diverse GPU programming approaches, such as CUDA, Triton, CuTe DSL, Gluon, ThunderKittens [110], and TileLang [129], and covers widely used GPU architectures and numerical formats including BF16, FP8, and FP4. Rewards evaluate both correctness and performance: each kernel provides a PyTorch reference implementation, and solutions exceeding a predefined numerical error threshold receive zero reward. Performance is scored against an expert implementation, where matching it yields a reward of 0.5 and approaching the hardware roofline increases the reward toward 1. To ensure that rewards reflect genuine optimization, we develop a hacking-detection system that penalizes reward-hacking strategies such as CUDA graph replay, input caching, and precision reduction, and we continuously extend it with new safeguards as new hacking strategies are observed during Kimi K3's development.

为增强 Kimi K3 的 GPU 内核优化能力，我们构建了一个大规模的内核任务套件，涵盖了从单操作符内核到融合的大内核，这些任务来源于高质量的 GitHub 仓库，例如 Flash Linear Attention [139]。该套件涵盖了多种多样的 GPU 编程方法，包括 CUDA、Triton、CuTe DSL、Gluon、ThunderKittens [110] 和 TileLang [129]，并覆盖了广泛使用的 GPU 架构和数值格式，如 BF16、FP8 和 FP4。奖励机制同时评估正确性和性能：每个内核都提供一个 PyTorch 参考实现，解决方案如果超过预定义的数值误差阈值，则获得零奖励。性能则与专家实现进行比较，匹配专家实现可获得 0.5 的奖励，接近硬件峰值性能则可使奖励逐步增加至 1。为确保奖励真正反映优化效果，我们开发了一个反作弊系统，对诸如 CUDA 图回放、输入缓存和精度降低等奖励作弊策略进行惩罚，并在 Kimi K3 开发过程中不断根据新发现的作弊策略扩展新的保护机制。

4.2.5 Personal Assistant Tasks

4.2.5 个人助理任务

For long-horizon personal assistant tasks, we develop realistic mock implementations of widely used applications, such as Gmail, Notion, Slack, and Canvas. They preserve the core semantics of their real-world counterparts while enabling reproducible, large-scale interaction without external APIs or rate limits. Building on these mock applications, we design complex tasks inspired by real-world professional workflows in scenarios like human resources, legal services, and finance. In each task, the agent operates in a persistent, evolving environment over multiple simulated days and encounters dozens of interdependent events distributed across applications. A single rollout may involve up to thousands of tool calls and millions of context tokens. Each event carries its own evaluation criterion, assessed by deterministic rules or LLM-based evaluators. The initial workspace is constructed by agents that autonomously search the web for reference materials and transform them into a coherent, task-relevant environment. We also extend our RL framework to support such living environments, modeling complex event streams and the induced world-state transitions.

对于长期目标的个人助理任务，我们开发了广泛使用的应用程序（如 Gmail、Notion、Slack 和 Canvas）的现实模拟实现。这些模拟应用保留了其真实世界对应物的核心语义，同时能够在没有外部 API 或速率限制的情况下实现可重复、大规模的交互。基于这些模拟应用，我们设计了复杂的任务，这些任务受到人力资源、法律服务和金融等现实专业工作流程的启发。在每个任务中，代理在一个持久且不断变化的环境中运行，跨越多个模拟天数，并遇到分布在不同应用上的数十个相互依赖的事件。一次部署可能涉及多达数千次工具调用和数百万个上下文标记。每个事件都有自己的评估标准，由确定性规则或基于 LLM 的评估器进行评估。初始工作空间由代理自主搜索网络参考材料，并将其转化为一个连贯且与任务相关的环境。我们还扩展了我们的强化学习框架，以支持此类动态环境，模拟复杂事件流以及由此引发的世界状态转换。

4.2.6 Autonomous Execution Tasks

4.2.6 自主执行任务

We introduce Autonomous Execution Tasks (AET), an environment paradigm that trains long-horizon agent intelligence through verify-in-the-loop optimization. Each task specifies an initial state, a constrained goal, a tool-based action space, execution budgets, and an independent verifier. Agents see only the objective, context, constraints, and verification interfaces, without reference trajectories or predefined procedures, and must autonomously perform task decomposition, tool selection, planning, error recovery, and termination. Rewards are grounded in the verifier’s evaluation of the final environment state rather than the agent’s self-reported completion. We design multiple types of verifiers that support diverse environments, including black-box system replication (Figure 10), quantitative factor discovery, and tax auditing. In each environment, agents iteratively submit solutions, receive verifier feedback, and refine their strategies, training a general loop of hypothesizing, acting, analyzing feedback, and adapting. Reward hacking is mitigated by isolating agents from verifiers, pairing public verifiers that offer diagnostic feedback with hidden verifiers that evaluate held-out scenarios, and applying penalty-based rewards under limited submission budgets.

我们引入了自主执行任务（AET），这是一种环境范式，通过循环验证优化来训练具有长期目标的智能体。每个任务都指定了初始状态、受限目标、基于工具的动作空间、执行预算以及独立验证器。智能体只能看到目标、上下文、约束条件和验证接口，而不参考轨迹或预定义的流程，必须自主地进行任务分解、工具选择、规划、错误恢复和终止。奖励基于验证器对最终环境状态的评估，而不是智能体自我报告的完成情况。我们设计了多种类型的验证器，以支持不同的环境，包括黑盒系统复制（图 10）、量化因子发现和税务审计。在每个环境中，智能体反复提交解决方案，接收验证器反馈，并改进其策略，从而训练出一个通用的循环过程：假设、行动、分析反馈和适应。通过将智能体与验证器隔离，结合提供诊断反馈的公开验证器和评估未见过场景的隐藏验证器，并在有限提交预算下应用基于惩罚的奖励，从而缓解奖励黑客问题。

4.2.7 Web Development Tasks

4.2.7 网站开发任务

We construct a diverse suite of expert-curated web development tasks covering typical scenarios. Inputs range from one-line scene descriptions to multi-paragraph specifications; artifacts span websites, interactive games, 3D/WebGL scenes, data visualization, SVGs, and full-stack applications. Every task runs in a containerized sandbox and is rolled out under diverse agent scaffolds rather than a single fixed harness, to promote cross-scaffold generalization. Rewards consist of two components: deterministic checks and model judging by an internal reward model. Deterministic checks functionally test application behavior, and score structural and pixel-level similarity for tasks that replicate a reference. The reward is zeroed when a project fails to build, runs with errors, or fakes rather than implements the artifact. Model judging uses other models to perform source code inspection or to look at and interact with the output artifact.

我们构建了一套多样化的专家整理的网页开发任务，涵盖典型的场景。输入内容从单行场景描述到多段落规格说明不等；产出物包括网站、交互式游戏、3D/WebGL 场景、数据可视化、SVG 以及全栈应用。每个任务都在容器化的沙箱中运行，并在多种代理框架下部署，而非单一固定的框架，以促进跨框架的泛化能力。奖励由两个部分组成：确定性检查和由内部奖励模型进行的模型评估。确定性检查功能上测试应用程序的行为，并对复制参考内容的任务进行结构和像素级别的相似性评分。当项目无法构建、运行时出现错误或伪造而非实现产出物时，奖励将被设为零。模型评估使用其他模型对源代码进行检查，或查看并交互产出物。

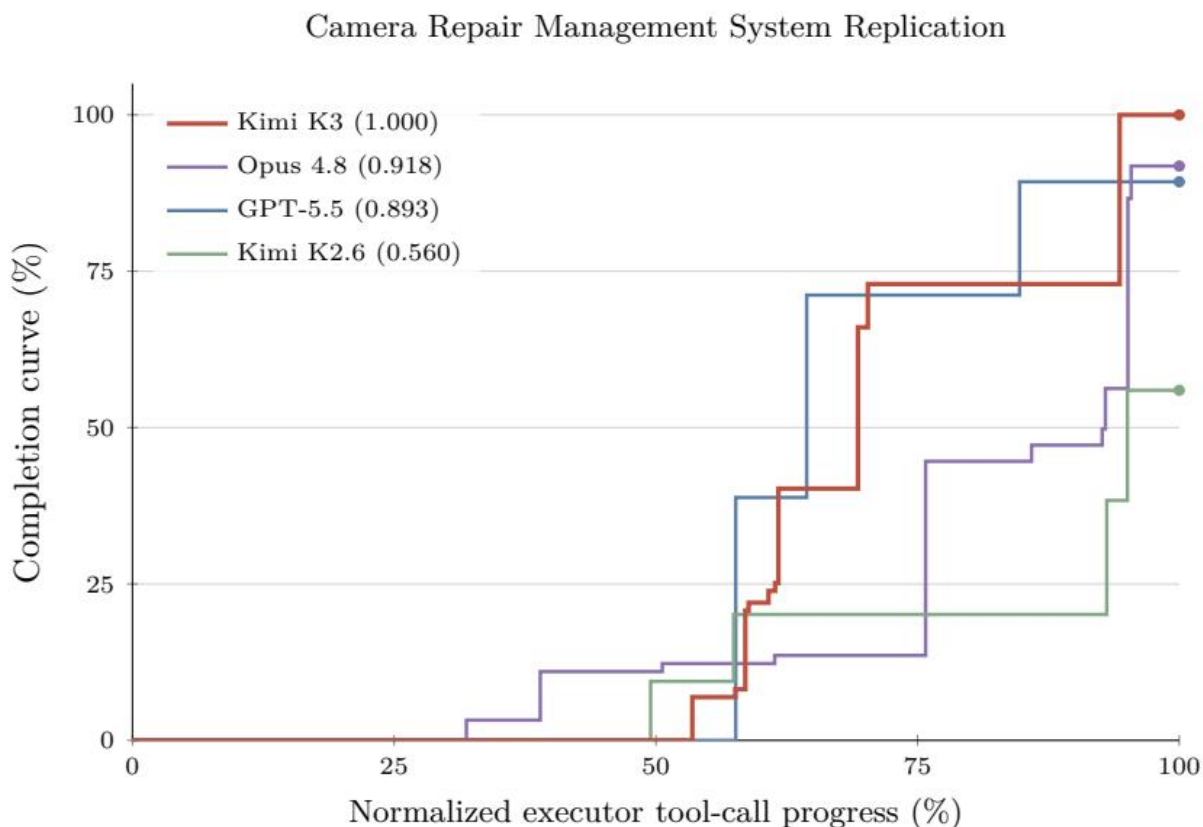


Figure 10: images/f20624e135d50a45e852057becab174885022bb6df8e589126c8bfedfdf914cf.jpg

Figure 10: Completion curves on Camera Repair Management System, a black-box system replication task in which the agent reconstructs a hidden 3D-camera repair system as a web application through oracle queries. Completion denotes verifier-assessed task progress.

图 10：完成曲线在相机维修管理系统中，这是一个黑盒系统复制任务，代理通过查询或acles 重建隐藏的 3D 相机维修系统作为网络应用程序。完成度表示验证者评估的任务进度。

5 Infrastructure

5 基础设施

Kimi K3 combines three system challenges rarely encountered in a single model: hybrid KDA attention, 3T-class sparse multimodal training and inference, and million-token agentic workloads. Our infrastructure is co-designed with these challenges across the model lifecycle. At the architecture level, high-performance KDA kernels and Context Parallelism make the recurrent formulation efficient within and across devices, in both training and inference. During pretraining, balanced expert execution, reduced memory footprint, and communication-overlapped scheduling sustain high utilization at scale. During 1M-token agentic RL, hierarchical state management and resumable sandbox execution preserve long trajectories across iterations. Finally, state-aware KDA prefix caching, specialized inference kernels, and cache- and budget-aware scheduling translate these efficiencies into predictable production serving.

Kimi K3 结合了三个在单个模型中很少遇到的系统挑战：混合 KDA 注意力、3T 类稀疏多模态训练和推理，以及百万标记的代理工作负载。我们的基础设施在整个模型生命周期中与这些挑战共同设计。在架构层面，高性能的 KDA 内核和上下文并行性使得在训练和推理过程中，设备内部和跨设备的递归公式高效运行。在预训练阶段，平衡的专家执行、减少的内存占用以及通信重叠调度在大规模情况下保持高利用率。

在 1M 标记的代理强化学习中，分层状态管理与可恢复的沙盒执行能够在多次迭代中保持长轨迹。最后，状态感知的 KDA 前缀缓存、专用的推理内核以及缓存和预算感知的调度将这些效率转化为可预测的生产服务。

5.1 Algorithm-System Co-Design for KDA

5.1 KDA 的算法-系统协同设计

KDA replaces the growing key-value cache of softmax attention with a fixed-size recurrent state $S \in R^{d_k \times d_v}$ (§2.1.1), whose serial update poses challenges in parallel execution, in exchange for a fixed-size state that is cheap to transfer and reuse. The designs below address the first property and exploit the second at two levels of execution, with fused kernels within a device and KDA Context Parallelism across devices.

KDA 用一个固定大小的循环状态 $S \in R^{d_k \times d_v}$ (§2.1.1) 替代了 softmax 注意力中增长的键值缓存，该状态的串行更新在并行执行中存在挑战，以换取一个易于传输和重用的固定大小状态。下面的设计解决了第一个特性，并在两个执行层面利用了第二个特性：在设备内使用融合内核，在跨设备中使用 KDA 上下文并行性。

5.1.1 KDA Kernels across Regimes

5.1.1 不同制度下的 KDA 内核

The serial dependence of the KDA state is at odds with the GPU's preference for wide, uniform parallelism, and it manifests as a different bottleneck in each execution regime. We design a dedicated kernel for each regime.

KDA 状态的串行依赖与 GPU 对广泛、统一并行性的偏好相冲突，并且在每种执行模式下表现为不同的瓶颈。我们为每种模式设计专用的内核。

Chunkwise kernel for training and prefill The chunkwise form of KDA is parallel within each chunk but serial across chunks, since the recurrent state must propagate from chunk to chunk. Executed naively, these two phases alternate, leaving the SMs idle during the serial propagation. We therefore develop FlashKDA [14], a CUTLASS-based chunkwise kernel that overlaps intra-chunk computation with cross-chunk state propagation. The kernel decomposes the work into token-parallel stages and a head-parallel recurrence, each scheduled and tuned independently, and substantially outperforms the Triton reference implementation. FlashKDA serves both training and inference prefill and is auto-dispatched as a backend of flash-linear-attention [139].

分块式内核用于训练和预填充。KDA 的分块形式在每个分块内是并行的，但在分块之间是串行的，因为递归状态必须从一个分块传播到下一个分块。如果直接执行，这两个阶段会交替进行，导致在串行传播期间 SM 空闲。因此，我们开发了 FlashKDA [14]，这是一个基于 CUTLASS 的分块式内核，它将块内计算与块间状态传播重叠。该内核将工作分解为并行于 token 的阶段和并行于头的递归，每个部分独立地进行调度和调优，并且显著优于 Triton 参考实现。FlashKDA 既用于训练也用于推理预填充，并作为 flash-linear-attention [139] 的后端自动调度。

Intra-device context parallelism for long-context prefill Tensor parallelism partitions heads across devices but never shortens the recurrence, so under pure TP deployment, prefilling an ultra-long sequence leaves most SMs idle when each rank holds only a few heads. The key observation is that the state transition of each segment can be evaluated

设备内上下文并行性用于长上下文预填充，张量并行性将头部分布在不同设备上，但不会缩短递归过程，因此在纯 TP 部署下，当每个 rank 仅持有少量头时，预填充超长序列会导致大多数 SM 空闲。关键的观察是，每个段的状态转移可以被评估

independently of the incoming state and composed exactly afterward. An automatic SM-level context-parallel (CP) planner [142, 139] therefore partitions the sequence across the SMs of a single rank, evaluates the segment transitions in parallel, and merges them to recover each segment’s exact initial state. In contrast to the cross-device KCP of §5.1.2, this parallelism is entirely intra-device and incurs no cross-device communication.

与输入状态无关，并且在之后精确地组成。因此，一个自动的 SM 级上下文并行（CP）规划器 [142, 139] 将序列划分到单个等级的 SMs 上，并行地评估各个段的转移，并将它们合并以恢复每个段的精确初始状态。与第 5.1.2 节中跨设备的 KCP 不同，这种并行性完全是在单个设备内进行的，并且不会产生跨设备的通信。

KDA decoding presents challenges distinct from those encountered during training and prefill. We discuss these challenges in detail in §5.4.2.

KDA 解码带来了与训练和预填充阶段不同的挑战。我们在 §5.4.2 中详细讨论了这些挑战。

5.1.2 KDA Context Parallelism

5.1.2 KDA 上下文并行性

The communication overhead of context parallelism differs fundamentally between softmax and linear attention. Softmax attention requires ranks to exchange key-value blocks whose size grows with the sequence length [72]. Linear attention instead carries the preceding context in a fixed-size recurrent state $S \in R^{d_k \times d_v}$. Prior context-parallel methods exploit the additive recurrence of vanilla linear attention by computing, on each rank, the state that the local tokens generate from $S = 0$ and summing these local states over the preceding ranks to recover the incoming state [114, 113].

上下文并行性的通信开销在 softmax 和线性注意力之间存在根本性的差异。softmax 注意力要求各 ranks 交换键值块，其大小随着序列长度增长 [72]。而线性注意力则通过一个固定大小的循环状态 $S \in R^{d_k \times d_v}$ 来携带先前的上下文。先前的上下文并行方法通过利用普通线性注意力的加法循环特性，在每个 ranks 上计算从 $S = 0$ 开始的局部 token 生成的状态，并将这些局部状态在先前的 ranks 上求和以恢复输入状态 [114, 113]。

This direct summation, however, is insufficient for KDA. Recall from Eq. 1 that KDA updates its state as $\mathbf{S}_t = \mathbf{M}_t \mathbf{S}_{t-1} + \beta_t k_t v_t^\top$, where $\mathbf{M}_t := (\mathbf{I} - \beta_t k_t k_t^\top) \text{Diag}(\alpha_t)$. KDA’s delta rule applies the token-dependent matrix $\mathbf{M}_t$ to the incoming state before adding the current write. Consequently, the effect of a local sequence segment depends on the state entering that segment and cannot be determined from the state computed with $\mathbf{S} = \mathbf{0}$ alone.

然而，这种直接求和对于 KDA 来说是不够的。回想一下公式 1，KDA 更新其状态为 $\mathbf{S}_t = \mathbf{M}_t \mathbf{S}_{t-1} + \beta_t k_t v_t^\top$ ，其中 $\mathbf{M}_t := (\mathbf{I} - \beta_t k_t k_t^\top) \text{Diag}(\alpha_t)$ 。KDA 的 delta 规则在将当前写入添加之前，应用了依赖于令牌的矩阵 $\mathbf{M}_t$ 到输入状态。因此，一个局部序列片段的效果取决于进入该片段的状态，而不能仅通过使用 $\mathbf{S} = \mathbf{0}$ 计算出的状态来确定。

To preserve this dependence, we introduce KDA Context Parallelism (KCP), which decomposes the effect of each segment into two locally computable quantities, a cumulative transition acting on the incoming state and a state generated locally from zero. Following the chunkwise notation of §2.1.1, we write $\mathbf{S}_{[i]}^t$ for the recurrent state within the segment of rank i after t local tokens, so that $\mathbf{S}_{[i]}^{T_i}$ denotes the state leaving rank i and entering rank $i + 1$. We write $\tilde{\mathbf{S}}_{[i]}^t$ for the state of the same recurrence started instead from $\mathbf{S} = \mathbf{0}$. For an arbitrary state entering the $(i + 1)$ -th of P context-parallel ranks, the state after t local tokens is

为了保持这种依赖性，我们引入了 KDA 上下文并行性（KCP），它将每个段落的效果分解为两个可以局部计算的量，一个是作用于输入状态的累积转移，另一个是从零生成的局部状态。按照第 2.1.1 节的分块表

示法，我们用 $\mathbf{s}_{[i]}^t$ 表示在 t 个局部标记之后，段落等级为 i 内的循环状态，因此 $\mathbf{s}_{[i]}^{T_i}$ 表示从等级 i 离开并进入等级 $i + 1$ 的状态。我们用 $\tilde{\mathbf{s}}_{[i]}^t$ 表示从 $\mathbf{s} = \mathbf{0}$ 开始的相同循环状态。对于任意一个进入 P 个上下文并行等级中的第 $(i + 1)$ 个的状态，在 t 个局部标记之后的状态是

$$\mathbf{M}_{[i+1]}^{t \leftarrow 1} := \prod_{r \leftarrow 1}^t \mathbf{M}_r \in \mathbb{R}^{d_k \times d_k}, \quad \mathbf{s}_{[i+1]}^t = \tilde{\mathbf{s}}_{[i+1]}^t + \mathbf{M}_{[i+1]}^{t \leftarrow 1} \mathbf{s}_{[i]}^{T_i} \quad (17)$$

$$= \tilde{\mathbf{s}}_{[i+1]}^t + \mathbf{M}_{[i+1]}^{t \leftarrow 1} \sum_{j=1}^i \left(\prod_{l \leftarrow j+1}^i \mathbf{M}_{[l]}^{T_l \leftarrow 1} \right) \tilde{\mathbf{s}}_{[j]}^{T_j} \in \mathbb{R}^{d_k \times d_v}.$$

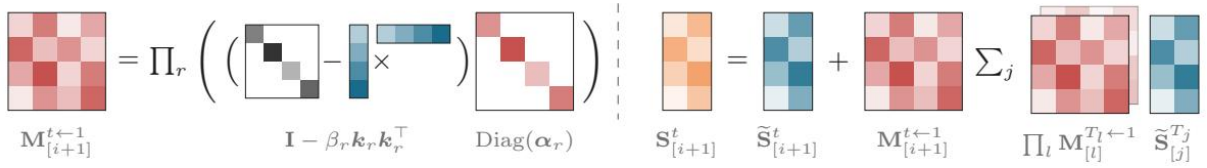


Figure 11: images/e709f1b3237353aea8e3df777955a4f4f9505f16a7147e9217db051b0d3d82ff.jpg

where $M_{[i+1]}^{t \leftarrow 1}$ denotes the cumulative transition of the first t local tokens. The first term contains the state generated by the local tokens, whereas the second term propagates the context from preceding ranks through the local KDA updates. At $t = T_{i+1}$, both quantities $M_{[i+1]}^{T_{i+1} \leftarrow 1}$ and $\tilde{S}_{[i+1]}^{T_{i+1}}$ can be computed using only the local tokens, before $S_{[i]}^{T_i}$ is available, and are the fragments each rank exchanges with the others.

其中 $M_{[i+1]}^{t \leftarrow 1}$ 表示前 t 个局部 token 的累积转移。第一项包含由局部 token 生成的状态，而第二项则通过局部 KDA 更新将上下文从先前的排名传播开来。在 $t = T_{i+1}$ 处，这两个量 $M_{[i+1]}^{T_{i+1} \leftarrow 1}$ 和 $\tilde{S}_{[i+1]}^{T_{i+1}}$ 可以在 $S_{[i]}^{T_i}$ 可用之前仅使用局部 token 计算得出，并且是每个排名与其他排名交换的片段。

The summation in Eq. 17 shows that every state is composed purely from locally computed fragments. These rank-level updates compose associatively, so the incoming state of each rank can be recovered by a prefix scan [77]. Each rank first computes $\mathbf{M}_{[i]}^{T_i \leftarrow 1}$ and $\tilde{\mathbf{S}}_{[i]}^{T_i}$ locally, then exchanges both tensors with one all-gather [139]. After the all-gather, rank $i + 1$ reconstructs $\mathbf{s}_{[i]}^{T_i}$ by processing preceding fragments of the same document in order, starting from $\mathbf{s} = \mathbf{0}$ and applying $\mathbf{s} \leftarrow \mathbf{M}_{[j]}^{T_j \leftarrow 1} \mathbf{s} + \tilde{\mathbf{s}}_{[j]}^{T_j}$ at each fragment. Therefore, KCP requires only a fixed-size all-gather for recurrent-state synchronization and achieves linear compute scaling.

式 17 中的求和表明，每个状态纯粹由本地计算的片段组成。这些等级级别的更新具有结合性，因此可以通过前缀扫描 [77] 恢复每个等级的输入状态。每个等级首先本地计算 $\mathbf{M}_{[i]}^{T_i \leftarrow 1}$ 和 $\tilde{\mathbf{S}}_{[i]}^{T_i}$ ，然后通过一次全收集交换这两个张量 [139]。全收集之后，等级 $i + 1$ 通过按顺序处理同一文档的先前片段，从 $\mathbf{s} = \mathbf{0}$ 开始，并在每个片段上应用 $\mathbf{s} \leftarrow \mathbf{M}_{[j]}^{T_j \leftarrow 1} \mathbf{s} + \tilde{\mathbf{s}}_{[j]}^{T_j}$ ，从而重建 $\mathbf{s}_{[i]}^{T_i}$ 。因此，KCP 只需进行固定大小的全收集即可实现循环状态同步，并实现线性计算扩展。

5.2 Infra for 3T-class Pre-Training

5.2 适用于 3T 级预训练的基础设施

Kimi K3 pre-training combines Pipeline Parallelism (PP) with virtual stages (VP) [48, 81], Expert Parallelism (EP) [66], ZeRO-1 Data Parallelism [100], Pipeline ZeRO-2 gradient sharding [145], and Context Parallelism (CP, §5.1.2) [50].

Kimi K3 预训练结合了流水线并行 (PP) 与虚拟阶段 (VP) [48, 81]、专家并行 (EP) [66]、ZeRO-1 数据并行 [100]、流水线 ZeRO-2 梯度分片 [145]，以及上下文并行 (CP, 第 5.1.2 节) [50]。

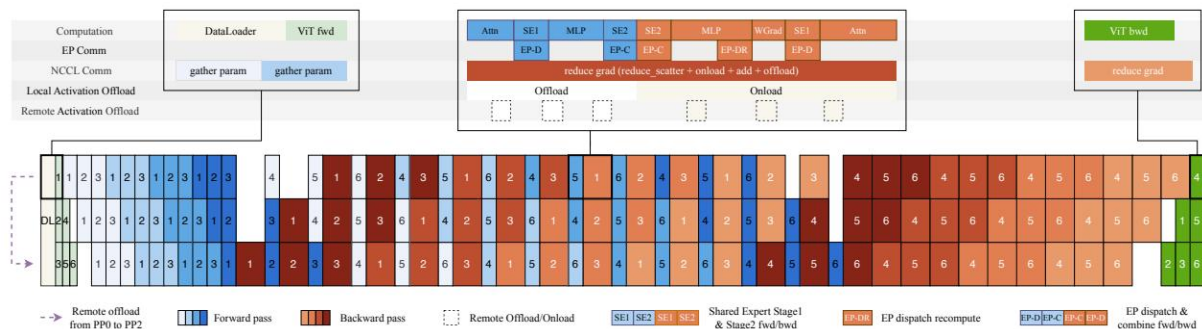


Figure 11: Computation, communication and offloading overlapped in different PP phases.

图 11：计算、通信和卸载在不同 PP 阶段中的重叠。

The MoE layers employ shared experts replicated across EP ranks, and the all-to-all communication for expert dispatch and combine is overlapped with computation to hide its latency.

MoE 层使用在 EP 等级上复制的共享专家，并且专家调度 and 组合的全互通信与计算重叠，以隐藏其延迟。

Natively multimodal pre-training at the 3T-class poses three critical problems: (i) token loads are imbalanced across EP ranks; (ii) activations, gradients, and optimizer states exceed the memory budget; and (iii) the vision encoder’s highly variable computation is exposed on the critical path. The following subsections address these problems in turn: perfectly balanced expert-parallel MoE training (§5.2.1), memory-efficient training (§5.2.2), and multimodal encoder optimization (§5.2.3). Fig. 11 illustrates the resulting execution schedule.

在 3T 级的原生多模态预训练中存在三个关键问题：(i) 在 EP 等级之间，token 负载不平衡；(ii) 激活值、梯度和优化器状态超过了内存预算；(iii) 视觉编码器的高变计算在关键路径上暴露。以下小节依次解决这些问题：完全平衡的专家并行 MoE 训练 (§5.2.1)、内存高效的训练 (§5.2.2) 以及多模态编码器优化 (§5.2.3)。图 11 展示了由此产生的执行计划。

5.2.1 Perfectly Balanced Expert-Parallel MoE Training

5.2.1 完美平衡的专家并行 MoE 训练

In conventional EP schemes, token loads are imbalanced across ranks. The resulting computational imbalance degrades training throughput, and the dynamically varying shapes of routed-expert activations cause substantial memory fragmentation. We therefore propose MoonEP³, an EP scheme that achieves perfect load balance with dynamic redundant experts. MoonEP preserves the overall computation flow of conventional schemes such as DeepEP [147] and additionally introduces online planning and migration of redundant experts. In the forward pass, we plan the redundant experts from the router outputs of the current micro-batch and layer and prefetch them before the routed-expert computation. In the backward pass, we stage their gradients in a local reduce buffer and, once the computation completes, reduce them back to the gradient buffers of their home ranks.

在传统的 EP 方案中, token 负载在不同 rank 之间是不平衡的。这种计算不平衡会降低训练吞吐量, 而路由专家激活的动态变化形状会导致显著的内存碎片化。因此, 我们提出 MoonEP³, 一种能够通过动态冗余专家实现完美负载均衡的 EP 方案。MoonEP 保留了传统方案 (如 DeepEP [147]) 的整体计算流程, 并进一步引入了冗余专家的在线规划和迁移。在前向传递过程中, 我们根据当前微批次和层的路由器输出来规划冗余专家, 并在路由专家计算之前进行预取。在反向传递过程中, 我们将它们的梯度暂存在一个本地减少缓冲区中, 并在计算完成后将其减少回它们所属 rank 的梯度缓冲区中。

Perfect balance with bounded redundant experts MoonEP requires every rank to receive exactly $S \times K$ tokens, where S is the sequence length and K is the number of experts selected per token, so that all ranks perform identical amounts of computation. The key question is how many redundant experts suffice to guarantee such a balance. Let E be the number of experts and R the EP size. We prove that a balanced plan always exists with at most E/R redundant experts per rank and that this bound is essentially tight (§ E). Reserving E/R redundant-expert slots per rank therefore guarantees that planning always admits a feasible solution, so training is never interrupted. In contrast, prior work such as ECHO [137] and UltraEP [132] presets the number of redundant experts or imposes a per-rank token cap. Training is then forced to stop whenever no feasible plan exists within the cap, and the cap itself requires manual tuning while still leaving residual imbalance.

完美平衡与有限冗余专家的 MoonEP 需要每个 rank 接收恰好 $S \times K$ 个 tokens, 其中 S 是序列长度, K 是每个 token 选择的专家数量, 从而使得所有 rank 进行相同量的计算。关键问题是需要多少冗余专家才能保证这种平衡。设 E 为专家数量, R 为 EP 的大小。我们证明, 每个 rank 最多使用 E/R 个冗余专家即可保证存在一个平衡的计划, 并且这个界限本质上是紧的 (§ E)。因此, 每个 rank 预留 E/R 个冗余专家的插槽, 就可保证计划总是存在可行解, 从而训练永远不会中断。相比之下, 先前的工作如 ECHO [137] 和 UltraEP [132] 则预设了冗余专家的数量或对每个 rank 设置 token 的上限。当在上限内不存在可行计划时, 训练将被迫停止, 而该上限本身需要手动调整, 仍然存在残余的不平衡。

Online planning Computing the exact optimum at every training step is prohibitively expensive. We therefore compute exact solutions offline with integer linear programming (ILP) for representative cases as references and design a GPU planning kernel that is near-optimal, incurs negligible overhead, and always respects the E/R upper bound.

在线规划计算每个训练步骤的精确最优解成本过高。因此, 我们针对代表性情况使用整数线性规划 (ILP) 离线计算精确解作为参考, 并设计了一个接近最优的 GPU 规划内核, 其开销可以忽略不计, 并且始终遵守 E/R 上限。

Zero-copy communication Perfect balance also simplifies the communication path. We implement a fused permute/unpermute operator in which the planning kernel precomputes the destination of every token, so tokens are sent directly to their expert-grouped positions on remote ranks, and views of the communication buffer are returned directly to the computation, eliminating intermediate copies. Under worst-case imbalance, supporting the same copy-free data path in DeepEP requires a communication buffer of size $S \times K \times R$, whereas MoonEP requires only a fixed $S \times K$ buffer owing to the perfect balance.

零拷贝通信完美的平衡也简化了通信路径。我们实现了一个融合的排列/反排列操作符, 在该操作符中, 规划内核预先计算每个 token 的目标位置, 因此 token 可以直接发送到远程 rank 上对应的专家组位置, 并且通信缓冲区的视图可以直接返回给计算, 从而消除了中间拷贝。在最坏情况下的不平衡情况下, 要在 DeepEP 中支持相同的无拷贝数据路径需要一个大小为 $S \times K \times R$ 的通信缓冲区, 而由于完美平衡, MoonEP 只需要一个固定的 $S \times K$ 缓冲区。

Sync-free execution with static shapes In conventional MoE implementations, the per-expert token counts vary across steps and layers, and the host must synchronize with the device at every layer to obtain the actual computation

无同步执行与静态形状在传统的 MoE 实现中, 每个专家的 token 计数在步骤和层之间会变化, 主机必须在每一层与设备同步以获取实际的计算

shapes before launching the expert computation, stalling the pipeline between layers. With per-

fect balance, every rank receives exactly $S \times K$ tokens and the computation shapes of all layers are statically known. This eliminates the per-layer MoE host synchronization and alleviates the host-side kernel-launch overhead.

在启动专家计算之前，形状会进行调整，使层之间的流水线暂停。在完美平衡的情况下，每个 rank 接收恰好 $S \times K$ 个 token，并且所有层的计算形状都是静态已知的。这消除了每层 MoE 主机的同步，并减轻了主机端内核启动的开销。

Expert-GEMM scheduling and overlap Even with the aggregate load perfectly balanced across ranks, the per-expert token counts within each rank remain skewed, and a fixed-order, workload-oblivious schedule turns this skew into an imbalanced makespan across SM workers. We therefore schedule the routed-expert GEMM with a workload-aware scheduler that adapts its parameters to the current token distribution before launch and keeps them fixed during execution. A lightweight heuristic selects these parameters using an analytical cost model of hardware metrics, with key coefficients calibrated through offline autotuning. For the shared experts, we dispatch their GEMMs to a separate stream so that they overlap with other kernels.

专家-GEMM 调度与重叠即使在各节点上的总负载完全平衡的情况下，每个节点内每个专家的 token 数量仍存在偏差，而固定顺序、不考虑工作量的调度方式会将这种偏差转化为 SM 工作者之间的不平衡执行时间。因此，我们使用一个基于工作量的调度器来调度路由专家的 GEMM，在启动前根据当前的 token 分布调整其参数，并在执行过程中保持这些参数不变。一个轻量级的启发式方法利用硬件指标的分析成本模型来选择这些参数，关键系数通过离线自动调优进行校准。对于共享专家，我们将它们的 GEMM 分配到一个单独的流中，以便与其他内核重叠。

5.2.2 Memory-Efficient Training

5.2.2 高效内存训练

Unified activation manager We design a unified storage abstraction for activations, in which every tensor saved for the backward pass is associated with a pluggable storage backend. Recomputation, quantization, and offload/remote-offload are merely storage policies under this abstraction and can be freely composed at tensor granularity; policies are declared via lightweight annotations on tensors, fully decoupled from the model code. Recomputation is performed at function granularity, which supports cross-layer recomputation. In our implementation, all GPU memory is allocated on the main compute stream and managed within a single memory pool, avoiding multi-stream fragmentation and host-bound overhead; activations are prefetched back at layer granularity and overlapped with computation, introducing negligible extra overhead. In Kimi K3, most activations use block-wise FP8 quantization [58, 30] combined with offload/remote-offload, and element-wise operators are configured with recomputation.

统一激活管理器我们设计了一个统一的激活存储抽象，其中每个用于反向传播的张量都与一个可插拔的存储后端相关联。重计算、量化以及卸载/远程卸载在这一抽象下仅仅是存储策略，并且可以在张量粒度上自由组合；策略通过轻量级注解声明在张量上，完全与模型代码解耦。重计算在函数粒度上进行，支持跨层重计算。在我们的实现中，所有 GPU 内存都在主计算流上分配，并在单一内存池中管理，避免了多流碎片化和主机绑定的开销；激活在层粒度上被预取回来，并与计算重叠，引入的额外开销可以忽略不计。在 Kimi K3 中，大多数激活使用块式 FP8 量化 [58, 30] 结合卸载/远程卸载，并且元素级运算配置了重计算。

Memory-efficient MoE In the native MoE implementation, the gradient computation of permuted probs depends on the forward output output. Inspired by SonicMoE [41], we rewrite this gradient through a mathematical transformation into a form that depends only on the intermediate activation act_output and the upstream gradient doutput, eliminating the backward dependency on output at the cost of an additional lightweight element-wise computation. Furthermore, in the forward pass of the group GEMM, we save only the input of the dispatch operation; during the backward pass, the input of the group GEMM is recovered by recomputing dispatch. As shown in Fig. 11, the communication introduced by this recomputation can be overlapped with part of the group-GEMM backward computation, eliminating this portion of activation storage at a negligible cost.

内存高效的 MoE

在原生的 MoE 实现中，排列概率的梯度计算依赖于前向传播的输出 output。受 SonicMoE [41] 的启发，我们通过数学变换将这种梯度重新表示为仅依赖于中间激活值 act_output 和上游梯度 doutput 的形式，从而消除了对 output 的反向依赖，代价是增加了一个额外的轻量级逐元素计算。此外，在组 GEMM 的前向传播中，我们仅保存调度操作的输入；在反向传播中，通过重新计算调度操作的输入来恢复组 GEMM 的输入。如图 11 所示，这种重新计算引入的通信可以与部分组 GEMM 反向计算重叠，以微小的成本消除这部分激活值的存储。

Memory-efficient Attention residual For the attention residual, we design a companion optimization based on Block AttnRes. The block representation is generated once at the boundary layer and shared by all subsequent layers, residing directly on the GPU. The AttnRes computation is entirely wrapped with checkpointing, so the activation saved for the backward pass at each layer is identical to that of the standard residual architecture. For pipeline parallelism, we adopt cache-based pipeline communication [57], in which only newly generated blocks are incrementally transferred between stages and released as soon as the micro-batch finishes, reaching the theoretical lower bound on memory footprint.

内存高效的注意力残差对于注意力残差，我们基于 Block AttnRes 设计了一种配套的优化方法。块表示在边界层生成一次，并被所有后续层共享，直接存储在 GPU 上。AttnRes 的计算完全通过检查点机制进行封装，因此每一层在反向传播过程中保存的激活值与标准残差结构相同。对于流水线并行，我们采用基于缓存的流水线通信 [57]，其中仅新生成的块在各个阶段之间逐步传输，并在微批次完成时立即释放，从而达到内存占用的理论下限。

Balancing activations across PP ranks Under interleaved 1F1B pipeline parallelism, activations are unevenly distributed across PP ranks due to pipeline warmup, and the number of resident activations decreases as the PP rank increases. To avoid out-of-memory (OOM) errors, we remotely offload activations to the memory of other PP ranks using the Mooncake Transfer Engine [96], achieving balanced activation memory across PP ranks.

在交错的 1F1B 流水线并行中，由于流水线预热，激活值在 PP 等级之间分布不均，且随着 PP 等级的增加，驻留的激活值数量会减少。为了避免内存不足 (OOM) 错误，我们使用 Mooncake 传输引擎 [96] 将激活值远程卸载到其他 PP 等级的内存中，从而在 PP 等级之间实现激活值内存的均衡。

Pipeline ZeRO-2 gradient sharding and offloading Beyond activations, we use Pipeline ZeRO-2 gradient sharding [145] to shard gradients across data-parallel (DP) ranks. Furthermore, we store the sharded gradients in CPU memory to reduce peak GPU memory usage, while keeping the double grad buffer on the GPU. After gradients are reduced across DP ranks into the double grad buffer, they are accumulated into the CPU shards.

Pipeline ZeRO-2 梯度分片和卸载在激活值之外，我们使用 Pipeline ZeRO-2 梯度分片 [145] 将梯度分片到数据并行 (DP) 进程中。此外，我们将分片后的梯度存储在 CPU 内存中，以减少 GPU 内存的峰值使用，同时保留双梯度缓冲区在 GPU 上。在 DP 进程中将梯度归约到双梯度缓冲区后，它们会被累积到 CPU 分片中。

P2P-based Muon orthogonalization The distributed optimizer shards parameters evenly across DP ranks, whereas the Newton-Schulz orthogonalization in Muon requires the full parameter matrix, necessitating a communication step to gather complete parameters before each update. The naive approach performs an all-gather over the entire parameter buffer on every rank [73], which incurs a substantial memory footprint on top of making communication the primary bottleneck at scale. Instead, each rank retrieves only the shards of its locally owned parameters via peer-to-peer (P2P) communication with the corresponding owner ranks, eliminating the full-parameter buffer and reducing both memory usage and communication volume. Communication and computation are further pipelined at the granularity of model-chunk buffers, hiding the communication overhead.

基于 P2P 的 Muon 正交化方法。分布式优化器将参数均匀地分布在 DP 等级上，而 Muon 中的 Newton-Schulz 正交化需要完整的参数矩阵，因此在每次更新之前都需要通过通信步骤收集完整的参数。朴素的方法在每个等级上对整个参数缓冲区执行全部收集操作 [73]，这不仅使通信成为大规模下的主要瓶颈，还带来了显著的内存开销。相反，每个等级通过与相应所有者等级进行点对点 (P2P) 通信，仅获取其本地

拥有的参数的分片，从而消除了完整的参数缓冲区，减少了内存使用和通信量。此外，通信和计算在模型块缓冲区的粒度上进一步流水线化，从而隐藏了通信开销。

5.2.3 Multimodal Encoder Optimization

5.2.3 多模态编码器优化

Dynamic CP in multimodal encoder In long-context multimodal training, large images and long videos substantially increase the computation time of the vision encoder and cause significant load imbalance across devices. To address this, we extend context parallelism to such large samples. A single large image is partitioned along the patch dimension across multiple devices, and attention is computed by gathering key-value pairs (gather-KV) across CP ranks. In addition, we divide each CP group into several sub-CP groups and distribute multiple large images across them in a load-balanced manner, preventing the communication fraction from growing with scale. This reduces both the encoder latency of large visual samples and the cross-device load imbalance, allowing the remaining encoder computation to be hidden in pipeline bubbles.

多模态编码器中的动态上下文并行 (Dynamic CP) 在长上下文多模态训练中，大尺寸图像和长视频会显著增加视觉编码器的计算时间，并导致设备间的负载不平衡。为了解决这个问题，我们将上下文并行扩展到这些大尺寸样本。一个大的图像会沿着补丁维度在多个设备之间进行划分，并通过在 CP 秩 (CP ranks) 上收集键-值对 (gather-KV) 来计算注意力。此外，我们将每个 CP 组划分为若干个子 CP 组，并以负载均衡的方式将多个大图像分布到这些子 CP 组中，从而防止通信开销随着规模的增加而增长。这减少了大视觉样本的编码器延迟和跨设备的负载不平衡，使得剩余的编码器计算可以隐藏在流水线气泡中。

Encoder computation in PP bubbles In Kimi K2.5, we introduced the Decoupled Encoder Process (DEP) [59], which splits ViT and text training into separate stages and balances vision forward and backward passes across PP stages. We observe that, under the interleaved 1F1B pipeline schedule, the text forward passes of the first PP micro-batches are all scheduled at the very beginning, while the text backward passes of the last PP micro-batches finish only at the very end. We therefore further decompose the ViT computation. The ViT forward passes of the first PP micro-batches are executed synchronously upfront, the remaining forward passes are scheduled into pipeline bubbles, and the backward passes are handled analogously. As a result, most of the ViT computation is hidden within pipeline bubbles, largely eliminating the effective overhead of the vision encoder.

PP 气泡中的编码器计算在 Kimi K2.5 中，我们引入了解耦编码器流程 (DEP) [59]，该流程将 ViT 和文本训练拆分为单独的阶段，并在 PP 阶段之间平衡视觉前向和后向传递。我们观察到，在交错的 1F1B 流水线调度下，第一个 PP 微批次的文本前向传递都被安排在非常开始的位置，而最后一个 PP 微批次的文本后向传递则只在非常结束时完成。因此，我们进一步将 ViT 计算分解。第一个 PP 微批次的 ViT 前向传递被同步地提前执行，其余的前向传递被安排到流水线气泡中，而后向传递则类似地处理。结果，大部分的 ViT 计算隐藏在流水线气泡中，大大消除了视觉编码器的有效开销。

5.3 Infra for 1M Agentic RL

5.3 100 万智能体强化学习的基础设施

Scaling agentic RL for a model as large as Kimi K3 to million-token contexts under a bounded compute budget makes resource efficiency a first-order goal. This motivates two complementary efforts: 1) efficient training and rollout, including KV-cache management, request scheduling, and training-state placement; 2) high-performance resumable sandboxes for long-horizon interaction.

在计算预算受限的情况下，将代理强化学习扩展到像 Kimi K3 这样大的模型，并支持百万标记的上下文，使资源效率成为首要目标。这促使我们采取两项互补的努力：1) 高效的训练和部署，包括键值缓存管理、请求调度和训练状态放置；2) 高性能的可恢复沙箱，用于长时域交互。

5.3.1 Long-context RL infrastructure

5.3.1 长上下文强化学习基础设施

We adopt co-located RL training [58] to keep each 1M-context Kimi K3 RL experiment within a few hundred GPUs, and use partial rollouts [118] to reduce tail latency from ultra-long trajectories. This design achieves good hardware utilization, but introduces a memory usage contention between rollout KV-cache that needs to be persisted for the next iteration, and the memory needed for training. This challenge becomes more severe in long-context RL.

我们采用共置强化学习训练 [58]，以确保每个 1M 上下文的 Kimi K3 强化学习实验能够在几百块 GPU 上运行，并使用部分 rollout [118] 来减少超长轨迹带来的尾部延迟。这种设计实现了良好的硬件利用率，但引入了 rollout KV 缓存（需要持久化以供下一轮使用）与训练所需内存之间的内存使用竞争。这一挑战在长上下文强化学习中变得更加严重。

External KV cache pool At 1M-context multi-step rollout, a prefix KV-cache miss is extremely expensive. Partial rollout exacerbates this issue at the beginning of each iteration, due to many unfinished long prefill requests from the previous iteration arriving at the same time. Speculative decoding further accelerates request turnover within relatively fixed tool-call intervals, increasing prefix-block churn. These issues can trigger preemption and lower the cache hit rate, which is critical for long-context RL.

外部 KV 缓存池在 1M 上下文多步部署中，前缀 KV 缓存未命中成本极高。部分部署在每次迭代开始时加剧这一问题，因为上一次迭代中许多未完成的长预填充请求会同时到达。推测解码进一步加快了在相对固定工具调用间隔内的请求周转速度，增加了前缀块的频繁变化。这些问题可能会触发抢占并降低缓存命中率，这对于长上下文强化学习至关重要。

We therefore decouple prefix retention from GPU residency with a write-back design. Active decoding blocks remain in GPU KV cache, while reusable idle prefixes are written back to an external KV cache pool in CPU DRAM only when it is evicted from GPU, and is prefetched back before the next reuse. KDA states are offloaded and prefetched together with the corresponding MLA KV cache blocks, keeping their lifecycles aligned. Compared with a write-through strategy, this policy incurs CPU DRAM usage and transfer bandwidth only for prefixes that leave the active decode path, avoiding redundant CPU copies of blocks that are still resident and active on GPU.

因此，我们采用一种写回设计，将前缀保留与 GPU 驻留解耦。活跃的解码块保留在 GPU 的 KV 缓存中，而可重用的空闲前缀在被驱逐出 GPU 时写回至 CPU DRAM 中的外部 KV 缓存池，并在下次重用前被预取回。KDA 状态与相应的 MLA KV 缓存块一起被卸载和预取，从而保持其生命周期的一致性。与写通策略相比，该策略仅对离开活跃解码路径的前缀产生 CPU DRAM 使用和传输带宽，避免了对仍驻留在 GPU 并处于活跃状态的块进行冗余的 CPU 复制。

To provide sufficient DRAM for the external pool, we offload training states (model weights and optimizer states) to NVMe after a training iteration finishes. After a rollout iteration, the pool is released to avoid contention with training workloads.

为了为外部池提供足够的 DRAM，我们在每次训练迭代完成后将训练状态（模型权重和优化器状态）卸载到 NVMe。在一次 rollout 迭代之后，释放池以避免与训练负载发生争用。

Rollout auto-throttling scheduler In multi-step rollout, contexts grow progressively as the trajectory advances, making fixed concurrency based on the full-trajectory average length both hard to estimate and overly conservative early on. Conversely, setting concurrency too high creates KV cache pressure in later stages and can trigger preemption. We therefore design an auto-throttling mechanism at the LLM request scheduling layer, using runtime signals such as active request count, queued request count, and KV cache utilization to dynamically control how many requests are sent to the inference engine. This keeps early rollout well utilized while reducing concurrency as KV cache pressure rises, avoiding both under-saturation and overload without manual tuning.

发布自动限流调度器在多步骤发布过程中，随着轨迹的推进，上下文会逐步增长，因此基于完整轨迹平均长度的固定并发度既难以估计，在早期阶段也过于保守。相反，如果设置的并发度太高，则会在后期阶段造成 KV 缓存压力，并可能触发抢占。因此，我们设计了一种自动限流机制，在 LLM 请求调度层使用运行时信号（如活跃请求数量、排队请求数量和 KV 缓存利用率）动态控制发送到推理引擎的请求数量。这确保了早期发布得到充分利用，同时随着 KV 缓存压力的增加而降低并发度，避免了手动调优的需要，从而防止资源不足和过载。

Gradient-buffer reuse for non-policy model forwarding RL loss computation often requires forward-only non-policy models, such as reference models, whose weights are too large to keep resident on GPU. We keep these weights in CPU memory and materialize them only when needed, backing their parameter tensors with the policy model's FP32 gradient-buffer storage. This reuses existing GPU memory without extra allocation or fragmentation, and remains safe because the buffers are overwritten when real gradients are later computed.

非策略模型前向传递的梯度缓冲区重用，通常需要仅前向的非策略模型，例如参考模型，其权重太大，无法驻留在 GPU 上。我们将在 CPU 内存中保存这些权重，并在需要时才进行实例化，利用策略模型的 FP32 梯度缓冲区存储来支撑这些参数张量。这种方法可以重用现有的 GPU 内存，无需额外分配或碎片化，并且是安全的，因为当后续计算真实梯度时，缓冲区会被覆盖。

With ZeRO-2 gradient sharding and offloading (§ 5.2.2), each GPU retains gradient buffers for only two VPP chunks in Kimi K3 RL training. We stream reference weights into these slots chunk by chunk: one slot is used for the current forward computation while the other prefetches the next chunk, hiding copy overhead without increasing GPU memory.

通过 ZeRO-2 梯度分片和卸载 (§ 5.2.2)，在 Kimi K3 RL 训练中，每个 GPU 仅保留两个 VPP 块的梯度缓冲区。我们按块将参考权重流式传输到这些插槽中：一个插槽用于当前的前向计算，另一个插槽则预取下一个块，从而隐藏了复制开销，而不会增加 GPU 内存。

5.3.2 Sandbox Infrastructure

5.3.2 沙盒基础设施

We employ multiple sandbox runtimes to support the diverse requirements of Kimi K3 post-training and evaluation, including a traditional container-based runtime, a GPU sandbox runtime, and, most notably, a new microVM-based sandbox runtime called AgentENV.

我们采用多种沙箱运行时来支持 Kimi K3 在训练和评估阶段的多样化需求，包括传统的基于容器的运行时、GPU 沙箱运行时，以及最值得注意的基于微虚拟机的新沙箱运行时 AgentENV。

AgentENV⁴, developed in collaboration with our partners, is a sandbox system specifically designed for agentic AI workloads. It is built around three core design goals:

AgentENV⁴，由我们与合作伙伴共同开发，是一个专为代理型 AI 工作负载设计的沙箱系统。它围绕三个核心设计目标构建：

- High-fidelity isolated sandbox runtime As agents become more capable and tasks more difficult, they tend to explore more aggressively and may even attempt reward hacking. On the one hand, this poses unique security challenges: in our early experiments with traditional container-based sandbox runtimes, we observed several kernel panics and deadlocks caused by unintended agent operations. On the other hand, we want to permit as much exploration as possible so as not to constrain agent capability, and complex tasks require a sandbox close to a real-world environment — for example, agents should be able to mount disks, run containers, or even launch virtual machines at will. By running isolated microVMs with Firecracker [3], AgentENV provides a level of isolation and fidelity that container-based runtimes cannot match.

- **Flexible sandbox life-cycles for agentic RL** At the low level, AgentENV supports incremental checkpointing and resuming of sandbox states, where only memory pages dirtied since the last checkpoint are saved during checkpointing, achieving checkpoint and resume latencies as low as 133 ms and 49 ms, respectively. On top of this, AgentENV provides three high-level operations that help improve agentic RL efficiency. (a) Pause and Resume: a paused sandbox consumes no memory or CPU resources; a sandbox can therefore be paused while the agent is waiting for the model's inference result, which can account for as much as 98% of the sandbox lifetime. (b) Fork: fork creates a new sandbox from the exact state of the original one while keeping the original running, which is useful for reward judging without side effects. (c) Snapshot: snapshots of a sandbox can be saved at regular intervals for error recovery.
- **High efficiency and high density** In our workloads, tens of thousands of sandboxes, each with a unique set of images, may need to be created within seconds. We adopt OverlayBD [68] as the image format, together with a custom ublk driver implementation, storage-layer sharing, and P2P transport, achieving sub-second launch latency at large scale. We further reduce memory usage with copy-on-write memory and page-cache optimizations, achieving a memory overcommit ratio of up to $6.5\times$ in real workloads.
- **高保真隔离的沙箱运行时环境**随着智能体的能力增强和任务的复杂性增加，它们往往会更加积极地进行探索，甚至可能尝试奖励操纵。一方面，这带来了独特的安全挑战：在我们早期使用传统容器型沙箱运行时环境的实验中，我们观察到由于智能体的意外操作导致的几次内核崩溃和死锁。另一方面，我们希望尽可能允许智能体进行探索，以免限制其能力，而复杂任务需要一个接近现实环境的沙箱——例如，智能体应该能够随意挂载磁盘、运行容器，甚至启动虚拟机。通过使用 Firecracker [3] 运行隔离的微虚拟机，AgentENV 提供了容器型运行时环境无法达到的隔离程度和保真度。
- **灵活的沙箱生命周期**用于智能体强化学习在底层，AgentENV 支持沙箱状态的增量检查点和恢复，其中在检查点过程中，只保存自上次检查点以来被修改的内存页，从而实现检查点和恢复延迟低至 133 毫秒和 49 毫秒。在此基础上，AgentENV 提供了三种高级操作，有助于提高智能体强化学习的效率。(a) 暂停和恢复：暂停的沙箱不消耗任何内存或 CPU 资源；因此，当智能体等待模型推理结果时，可以暂停沙箱，这可能占到沙箱生命周期的 98%。(b) 分叉：分叉从原始沙箱的确切状态创建一个新沙箱，同时保持原始沙箱继续运行，这对于奖励判断而不会产生副作用非常有用。(c) 快照：可以在固定间隔保存沙箱的快照以用于错误恢复。
- **高效和高密度**在我们的工作负载中，数万个沙箱，每个沙箱都有独特的镜像集，可能需要在几秒钟内创建。我们采用 OverlayBD [68] 作为镜像格式，结合自定义的 ublk 驱动程序实现、存储层共享和点对点传输，实现了大规模下的亚秒级启动延迟。我们进一步通过写时复制内存和页缓存优化来减少内存使用，在实际工作负载中实现了高达 $6.5\times$ 的内存过提交比例。

Throughout Kimi K3's training and evaluation, a total of 51,219,741 sandboxes across 1,505,678 images were created.

在整个 Kimi K3 的训练和评估过程中，共创建了 1,505,678 张图像，总计 51,219,741 个沙盒。

5.4 Inference and Online Serving

5.4 推理与在线服务

Serving Kimi K3 exposes the same challenges from the production side: the hybrid KDA-MLA architecture maintains two fundamentally different caches that must be managed jointly at million-token contexts, its new modules and highly sparse experts demand kernels tailored to each, and production traffic mixes requests whose per-request cost spans three orders of magnitude. The designs below address these challenges at three levels. At the engine level, a KDA-aware prefix cache packs the fixed-size recurrent state into the same paged pool as the MLA KV cache and keeps long prefixes reusable across requests. At the device level, dedicated kernels for KDA decoding, Block AttnRes, and the sparse latent MoE minimize per-token latency and memory traffic.

At the fleet level, cache-aware affinity scheduling and budget-based admission control translate these efficiencies into predictable serving.

为 Kimi K3 提供服务时，生产端面临相同的挑战：混合的 KDA-MLA 架构维护两个本质上不同的缓存，必须在百万标记上下文中联合管理；其新的模块和高度稀疏的专家需要针对每个模块量身定制的内核；生产流量混合了每个请求的成本跨越三个数量级的请求。以下设计在三个层面解决了这些挑战。在引擎层面，一种 KDA-aware 的前缀缓存将固定大小的递归状态打包到与 MLA KV 缓存相同的分页池中，并在请求之间保持长前缀的可重用性。在设备层面，为 KDA 解码、Block AttnRes 和稀疏潜在 MoE 专门设计的内核最小化了每标记的延迟和内存流量。在集群层面，基于缓存的亲性和调度及基于预算的准入控制将这些效率转化为可预测的服务。

5.4.1 KDA-Aware Prefix Cache Management

5.4.1 KDA-Aware Prefix Cache Management

The hybrid architecture in Kimi K3 complicates prefix caching: the KDA recurrent state and the MLA KV cache differ fundamentally in size and lifetime, yet a cached prefix is reusable only when both can be restored together at the same

Kimi K3 中的混合架构使前缀缓存变得复杂：KDA 循环状态和 MLA 的 KV 缓存在大小和生命周期上存在根本性差异，但只有在两者都能同时恢复的情况下，缓存的前缀才能被重新使用

boundary. We therefore design a KDA-aware prefix cache that manages the two cache types jointly—from a unified paged layout to fine-grained prefix reuse and consistency under concurrent scheduling—keeping million-token prefixes cheap to retain and reusable across requests.

边界。因此，我们设计了一个 KDA-aware 的前缀缓存，联合管理两种缓存类型——从统一的分页布局到在并发调度下细粒度的前缀重用和一致性，保持百万标记的前缀廉价地保留，并可在不同请求间重用。

Unified cache layout for hybrid KDA-MLA attention Each Kimi K3 block consists of three KDA layers and one Gated MLA layer, whose caches differ fundamentally. The MLA KV cache grows with sequence length and is paged per token, whereas the KDA recurrent state is fixed in size with a single copy per request. Maintaining a separate manager for each would duplicate the allocation, eviction, and transfer logic. We therefore pack KDA states into the same paged block pool as MLA KV, unifying pages to the same byte size so that both page types share one implementation of allocation, reference counting, and eviction. Within a page, the states of all heads are stored contiguously head by head, so that each head's byte stream is self-contained and serves as the minimal unit of cross-node transfer. Under prefill/decode disaggregation, when prefill and decode nodes adopt different TP degrees, re-layout is performed on the transfer path with zero GPU-side reshuffling. This asymmetry proved useful during development: any type-confused access yields garbage rather than plausible data — a zero-overhead sanity check on the pooled layout.

混合 KDA-MLA 注意力的统一缓存布局每个 Kimi K3 块由三个 KDA 层和一个门控 MLA 层组成，其缓存存在根本性差异。MLA 的 KV 缓存会随着序列长度增长而扩展，并按 token 进行分页，而 KDA 的循环状态则固定大小，每个请求仅保留一份副本。为每个缓存单独维护管理器会导致内存分配、驱逐和传输逻辑的重复。因此，我们将 KDA 状态打包到与 MLA KV 共享的同一分页块池中，统一页面大小，使得两种页面类型共享相同的内存分配、引用计数和驱逐实现。在页面内，所有头的状态按头顺序连续存储，因此每个头的字节流是自包含的，并作为跨节点传输的最小单位。在预填充/解码分离的情况下，当预填充和解码节点采用不同的 TP 度数时，传输路径上的重新布局在 GPU 端无需重新排序即可完成。这种不对称性在开发过程中证明是有用的：任何类型混淆的访问都会产生垃圾数据而非合理数据——这为池化布局提供了一个零开销的合理性检查。

KDA prefix cache optimization Block-hash-based prefix caching reuses the KV cache at the granularity of one physical block: only complete blocks are hashed, so only block-aligned prefixes are reusable.

KDA 前缀缓存优化基于块哈希的前缀缓存在物理块的粒度上重用 KV 缓存：仅对完整块进行哈希，因此只有对齐块的前缀才可重用。

This coupling breaks down in Kimi K3. Block-hash matching requires one block size shared by all layers, and a prefix hit is reusable only if the KDA state at the hit boundary has been persisted. A KDA layer maintains a single large recurrent state per sequence rather than per-token entries, so state snapshots are affordable only at sparse boundaries; the shared block size is therefore forced to 1024–6144 tokens—and, since hashing is tied to the storage block, the hash granularity as well, although MLA’s per-token entries alone would tolerate much finer blocks. At such a coarse granularity caching is nearly useless: requests shorter than one block can never be reused, and chunked prefill exports no cacheable prefix until it crosses a full block boundary.

这种耦合在 Kimi K3 中失效了。块哈希匹配需要一个由所有层共享的块大小，且只有在命中边界处的 KDA 状态已被持久化的情况下，才能重复使用前缀命中。KDA 层为每个序列维护一个单一的大规模递归状态，而不是每个 token 的条目，因此状态快照只能在稀疏边界处进行，因此共享的块大小被强制设置为 1024–6144 个 token。由于哈希与存储块相关联，因此哈希粒度也受到限制，尽管 MLA 的每个 token 条目本身可以容忍更精细的块。在如此粗粒度的情况下，缓存几乎毫无用处：长度短于一个块的请求永远无法被重复使用，并且分块预填充直到跨越完整的块边界之前都不会导出可缓存的前缀。

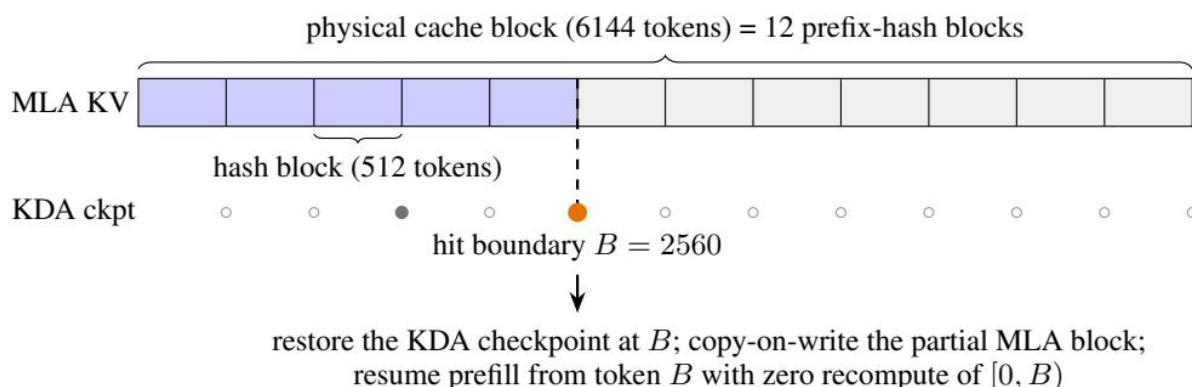


Figure 12: images/49c8d421c598bcf49b1368bd6dcd641385d48e5f8aba591d3b1e26087df26a5a.jpg

Figure 12: Fine-grained prefix caching within a physical cache block. A 6144-token physical block contains twelve 512-token hash blocks, with cached MLA blocks shown in blue and empty blocks in light gray. The markers below show the KDA checkpoint status at each hash boundary. An open circle (○) denotes a boundary without a stored checkpoint, a gray dot (●) denotes a persisted KDA checkpoint, and an orange dot (●) marks the checkpoint hit at $B = 2560$. Persisted checkpoints are sparse and typically coincide with conversation-turn boundaries. The request reuses the five MLA hash blocks and the KDA checkpoint at B , then resumes prefill without recomputing $[0, B)$.

图 12：物理缓存块内的细粒度前缀缓存。一个包含 6144 个 token 的物理块包含十二个每个包含 512 个 token 的哈希块，其中缓存的 MLA 块以蓝色显示，空块以浅灰色显示。下方的标记显示了每个哈希边界处的 KDA 检查点状态。一个开放的圆圈 (○) 表示没有存储检查点的边界，一个灰色点 (●) 表示已持久化的 KDA 检查点，一个橙色点 (●) 表示在 $B = 2560$ 处的检查点命中。已持久化的检查点较为稀疏，通常与对话回合边界一致。请求重用了五个 MLA 哈希块和 B 处的 KDA 检查点，然后在无需重新计算 $[0, B)$ 的情况下恢复预填充。

We therefore decouple the two granularities. Prefix hashing runs on fine hash blocks (e.g., 512 tokens) inside MLA pages, while the physical block remains the coarse allocation unit. Alignment runs the other way for KDA: checkpoints of the recurrent state are saved only at (a sparse subset of) MLA’s hash endpoints—the only positions a lookup can ever reference.

因此，我们将这两种粒度解耦。前缀哈希在 MLA 页面内的细粒度哈希块（例如 512 个 token）上运行，而物理块则保持为粗粒度的分配单位。对于 KDA，对齐方式则相反：递归状态的检查点仅在 MLA 的哈希

端点（即查找操作所能引用的唯一位置）上保存——这是一组稀疏的子集。

During prefill, a partially filled MLA page is registered in the prefix-cache index under the chained hash of its last complete hash block, where each hash covers all preceding hash blocks so that matching an endpoint certifies the whole prefix up to it; the registered endpoint advances as the page fills. Meanwhile, after each forward pass, the KDA kernel persists the recurrent state at the last hash-aligned position processed. Checkpoints are large, so intermediate checkpoints superseded as the request advances are recycled, while those at conversation-turn boundaries are retained for cross-request reuse. Cached checkpoints are read-only snapshots: a hit restores the state by copying it into the request's private running state before the next forward pass, and new checkpoints are written to fresh slots, so a checkpoint visible to other requests is never mutated in place.

在预填充过程中，部分填充的 MLA 页面在前缀缓存索引中以最后一个完整哈希块的链式哈希进行注册，其中每个哈希覆盖所有先前的哈希块，从而匹配端点证书可以验证该端点之前的整个前缀；随着页面填充，注册的端点也会不断前进。同时，在每次正向遍历之后，KDA 内核会在处理的最后一个哈希对齐位置持久化递归状态。检查点较大，因此随着请求的推进，中间的检查点会被覆盖并回收，而位于对话回合边界处的检查点则保留用于跨请求重用。缓存的检查点是只读快照：命中时，通过将其复制到请求的私有运行状态中，在下次正向遍历之前恢复状态；新的检查点写入到新的槽位中，因此一个对其他请求可见的检查点永远不会就地修改。

Lookup proceeds in two stages (Fig. 12). The MLA stage matches whole physical blocks by chained hash and, at the first missing block, falls back to the hash endpoints inside it, so partially filled pages remain hittable. The KDA stage then requires a checkpoint at the candidate boundary in every KDA cache group, each of which maintains an

查找过程分为两个阶段（图 12）。MLA 阶段通过链式哈希匹配完整的物理块，在第一个缺失块时回退到其内部的哈希端点，因此部分填充的页面仍可被访问。然后 KDA 阶段需要在每个 KDA 缓存组的候选边界处进行检查点，每个缓存组都维护一个

independent recurrent state. The hit is the longest boundary satisfying both stages—always a multiple of the hash block, and never required to be a multiple of the physical block. In Fig. 12, a request whose first 2800 tokens match the cached prefix hits at $B = 2560 = 5 \times 512$, deep inside a 6144-token physical block, and resumes prefill from token B instead of recomputing $[0, B)$.

独立的递归状态。命中是指满足两个阶段的最长边界——总是哈希块的整数倍，且不需要是物理块的整数倍。在图 12 中，一个请求的前 2800 个 token 与缓存的前缀匹配，在 $B = 2560 = 5 \times 512$ 处命中，位于一个 6144 个 token 的物理块内部深处，并从 B 处重新开始预填充，而不是重新计算 $[0, B)$ 。

Consistency under concurrent scheduling The remaining design points are each dictated by a concrete failure mode of sharing partially filled blocks, in a setting where a hit block is at once a shared cache entry and the growth point of a private request, and where the MLA and KDA cache groups must agree on every hit boundary. First, all cache groups draw blocks from one shared free list, so allocating a private copy for one group could evict a block that another group has just hit; every hit block is therefore pinned across all groups before anything is allocated. Second, the copy into the private block executes on the GPU immediately before the forward pass, so a block allocated or registered within the current scheduling step would still hand the previous owner's bytes to a reader; such blocks are excluded from matching until their copies land. Third, a checkpoint can restore a request only if it exists in every KDA group, so evicting one group's checkpoint atomically invalidates its siblings — a checkpoint is either hittable in every group or in none. With these mechanisms, every registered state always corresponds to exactly its declared token prefix, and prefix caching for hybrid KDA-MLA models reaches the same generality as for full-attention models: any shared prefix is reusable at any 512-token boundary, independently of request length, chunking, or scheduling interleaving.

并发调度下的一致性

剩余的设计点均由共享部分填充块的具体失效模式所决定。在一种场景下，命中块同时是共享缓存条目和私有请求的增长点，而 MLA 和 KDA 缓存组必须就每个命中边界达成一致。首先，所有缓存组都从同一个

共享空闲列表中获取块，因此为一个组分配私有副本可能会驱逐另一个组刚刚命中的块；因此，在任何分配之前，所有命中块都会被固定在所有组中。其次，复制到私有块的操作在前向传递之前立即在 GPU 上执行，因此在当前调度步骤中分配或注册的块仍会将先前所有者的字节交给读取器；此类块在它们的副本落地之前会被排除在匹配之外。第三，检查点只能在每个 KDA 组中都存在的情况下才能恢复请求，因此驱逐一个组的检查点会原子性地使它的兄弟检查点失效——一个检查点要么在所有组中都是可命中的，要么在没有任何组中可命中的。通过这些机制，每个注册的状态始终与它声明的令牌前缀完全对应，并且混合 KDA-MLA 模型的前缀缓存达到了与全注意力模型相同的通用性：任何共享前缀都可以在任何 512 个令牌边界处被重用，无论请求长度、分块方式或调度交错如何。

5.4.2 High-Performance Kernels

5.4.2 高性能内核

Kimi K3 introduces several new architectural modules: KDA (§2.1.1), Block AttnRes (§2.2), and Stable LatentMoE (§2.3). We optimize the kernel implementation for each.

Kimi K3 引入了几个新的架构模块：KDA (§2.1.1)、Block AttnRes (§2.2) 和 Stable LatentMoE (§2.3)。我们对每个模块的内核实现进行了优化。

KDA Compared with KDA prefill (§5.1), KDA decoding presents a distinct set of challenges: the primary bottleneck shifts from exploiting parallelism to efficiently managing the evolving recurrent state, which is updated in place at every decoding step. This in-place update becomes problematic in MTP-based speculative decoding: if verification rejects a subset of the drafted tokens, the state has already advanced beyond the last accepted token and cannot be trivially rolled back. Maintaining a state snapshot for each draft position would enable rollback, but would also multiply state traffic — a cost that dominates at the large batch sizes typical of online serving.

与 KDA 预填充 (§5.1) 相比，KDA 解码呈现出一组不同的挑战：主要瓶颈从利用并行性转移到高效管理不断演化的递归状态，该状态在每次解码步骤中都会就地更新。这种就地更新在基于 MTP 的推测解码中会带来问题：如果验证拒绝了部分草稿标记，状态已经超出了最后一个被接受的标记，无法轻易回滚。为每个草稿位置维护一个状态快照可以实现回滚，但也会增加状态流量——这一成本在在线服务中常见的大批次规模下占主导地位。

The state after any accepted draft prefix, however, is fully determined by the projected inputs of the draft tokens, which are far smaller than the state itself. We therefore cache only these projected inputs, rebuild the states of accepted tokens on-chip, and write back the states of the verified and bonus tokens, a design independently proposed in the concurrent work ReplaySSM [25]. The replayed tokens, the bonus token, and the next draft window share one recurrent loop inside a single fused kernel covering short convolution, input normalization, gating, the KDA recurrence, and output normalization. Verification latency grows sub-linearly with the number of tokens verified and remains below that of state-caching baselines. Because the projection caches never leave the decode stage, prefix caching and prefill-decode disaggregation operate on the same payload as in non-speculative serving.

然而，任何接受的草案前缀之后的状态，完全由草案标记的投影输入决定，这些投影输入远小于状态本身。因此，我们仅缓存这些投影输入，在芯片上重建接受标记的状态，并将已验证和奖励标记的状态写回，这是一种在并发工作 ReplaySSM [25] 中独立提出的架构设计。被重放的标记、奖励标记以及下一个草案窗口共享一个递归循环，该循环位于一个融合内核中，涵盖短卷积、输入归一化、门控、KDA 递归以及输出归一化。验证延迟随着验证标记数量的增加而以亚线性方式增长，并且保持在状态缓存基线以下。由于投影缓存从未离开解码阶段，前缀缓存和预填充-解码分离操作与非推测服务中的负载相同。

Block AttnRes Block AttnRes [57] follows a two-phase schedule: a batched inter-block pass reads the cached block representations once per block, after which each layer folds in the intra-block partial sum through an online-softmax merge [79]. Memory access accounts for a substantial fraction of the cost of these kernels in both prefill and decoding, so our optimizations in both stages focus primarily on memory efficiency.

Block AttnRes Block AttnRes [57] 采用两阶段调度：批量的块间传递在每个块中读取一次缓存的块表示，之后每个层通过在线-softmax 合并 [79] 将块内部分和折叠进来。内存访问在预填充和解码过程中都占这些内核成本的很大一部分，因此我们在两个阶段的优化主要集中在内存效率上。

For prefill, materializing the block representations on every tensor-parallel (TP) rank would incur substantial redundant memory consumption. We therefore adopt sequence parallelism (SP) for activations: the TP all-reduce is decomposed into a reduce-scatter and an all-gather, with the intra-block kernel inserted between the two collectives, operating on the sequence-sharded hidden states so that the block representations of each token are materialized on exactly one rank. This eliminates the additional memory consumption and reduces the I/O overheads of Block AttnRes during prefill.

对于预填充 (prefill) 阶段，如果在每个张量并行 (TP) rank 上都材料化块表示，将会导致大量的冗余内存消耗。因此，我们采用序列并行 (SP) 来处理激活值：将 TP 的 all-reduce 分解为一个 reduce-scatter 和一个 all-gather，并在两个集体操作之间插入一个块内核，该内核作用于序列分片的隐藏状态，使得每个 token 的块表示仅在一个 rank 上被材料化。这消除了额外的内存消耗，并减少了 Block AttnRes 在预填充期间的 I/O 开销。

For decoding, we launch the inter-block kernel on a side stream so that it overlaps with independent computation on the main stream. The intra-block kernel is instead streamlined through fusion: the merging of the AttnRes output with its partial-sum update, together with the subsequent RMSNorm, is fused into the preceding TP all-reduce, eliminating a dedicated kernel for the intra-block phase. Together, these optimizations hide the latency of the inter-block pass and reduce the memory traffic of the intra-block phase.

对于解码，我们在侧流上启动块间内核，使其与主流上的独立计算重叠。块内核则通过融合进行优化：将 AttnRes 输出与其部分和更新合并，并与随后的 RMSNorm 合并到前面的 TP 全减少操作中，从而消除了块内阶段专用的内核。这些优化共同隐藏了块间传递的延迟，并减少了块内阶段的内存流量。

Stable LatentMoE Stable LatentMoE increases both the total number of experts and the number of activated experts per token. The resulting growth in both the expert space and the per-token expert count raises scheduling and coordination overheads, making it difficult for conventional MoE kernels to sustain high hardware utilization. These challenges motivate dedicated kernel optimizations for this module.

Stable LatentMoE Stable LatentMoE 增加了专家总数以及每个 token 激活的专家数量。这种在专家空间和每个 token 专家数量上的增长增加了调度和协调的开销，使得传统的 MoE 内核难以维持高硬件利用率。这些挑战促使对该模块进行专门的内核优化。

To mitigate the overhead of the latent GEMMs, we adopt three optimizations. First, we fuse the latent down-projection with the MoE router into a single GEMM. Second, we shard latent weight matrices across ranks and fuse the output all-gather into the GEMM epilogue using multitem store instructions. Finally, we overlap the resulting communication with other operators, such as the shared-expert computation. Together, these optimizations eliminate redundant weight traffic and duplicated computation, while hiding the communication latency behind computation.

为了解决潜在 GEMMs 的开销问题，我们采用了三种优化方法。首先，我们将潜在的下投影与 MoE 路由器融合为一个 GEMM。其次，我们将潜在权重矩阵分片到不同的秩上，并利用多内存存储指令将输出的 all-gather 融合到 GEMM 的后处理阶段。最后，我们将由此产生的通信与其他操作（如共享专家计算）重叠。这些优化共同消除了冗余的权重流量和重复的计算，同时将通信延迟隐藏在计算之下。

For routed experts, at small batch sizes, the group GEMMs reduce to memory-bound streaming of weight matrices — a regime for which conventional tile-centric kernels are poorly suited due to their compute-oriented design and preprocessing overheads. We instead build the MoE decoding kernel upon the token-centric design of WarpDecode [12], in which each warp is responsible for one output neuron and streams the associated weights directly from memory. To further increase parallelism, we subdivide each warp into finer-grained lane teams, each processing a disjoint subset of experts, followed by a warp-wide reduction of the partial results. In addition, the weight layout is permuted offline at a one-time preprocessing cost, substantially reducing the

runtime dequantization overhead.

对于路由专家来说，在小批量大小的情况下，组 GEMMs 会减少为内存受限的权重矩阵流式处理——这与传统以瓦片为中心的内核设计不匹配，因为它们的计算导向设计和预处理开销使得其在该模式下表现不佳。相反，我们基于 WarpDecode [12] 的以 token 为中心的设计构建 MoE 解码内核，其中每个 warp 负责一个输出神经元，并直接从内存中流式传输相关权重。为了进一步提高并行性，我们将每个 warp 细分为更细粒度的通道团队，每个团队处理一组不相交的专家，随后对部分结果进行全 warp 的归约。此外，权重布局在一次预处理开销下进行离线排列，显著降低了运行时的反量化开销。

5.4.3 Fleet-Level Scheduling

5.4.3 队列级调度

Beyond a single serving instance, the challenge shifts from per-request efficiency to predictability: a prefix-cache miss costs orders of magnitude more than a hit, and a burst of million-token requests can starve short ones. We propose two fleet-level scheduling policies to address this: cache-aware affinity scheduling routes each session to the cluster holding its prefix cache while bounding the cost of cluster failures, and budget-based admission control grants each request class its own resource budget so that bursty long-context traffic cannot degrade system-wide SLOs.

除了单个服务实例之外，挑战从每个请求的效率转移到了可预测性：一个前缀缓存未命中所造成的成本比命中高几个数量级，而大量百万标记请求的突发流量可能会使短请求饥饿。我们提出了两种舰队级别的调度策略来解决这个问题：缓存感知的亲和性调度将每个会话路由到持有其前缀缓存的集群，同时限制集群故障的成本；基于预算的准入控制为每个请求类别分配其自己的资源预算，从而防止突发的长上下文流量损害系统范围内的 SLO。

Cache-aware affinity scheduling At 1M context, a typical coding input carries a prefix of 400K tokens but requires a prefill increment of only 4K tokens, so a prefix-cache hit avoids re-prefilling the entire prefix and is orders of magnitude cheaper than a miss. We therefore route each request to the cluster that holds its prefix cache, as moving the cache to another cluster would require transferring it over inter-cluster links far slower than the intra-cluster fabric. This cache-aware affinity, however, binds each session to a single cluster, whose failure would interrupt all sessions bound to it. Consistent hashing therefore pins each session to two clusters, a primary that serves its traffic and a pre-assigned secondary that takes over when the primary fails. The secondary holds none of the session's prefix cache and must re-prefill it upon failover. Since consistent hashing distributes the secondary assignments of different sessions uniformly across the fleet, this re-prefill work is divided among many clusters rather than concentrated on one. Cache locality is thus preserved in the common case, while the impact of any single cluster failure remains bounded.

基于缓存感知的亲和性调度在 1M 上下文的情况下，典型的编码输入携带 400K 个 token 的前缀，但只需要 4K 个 token 的预填充增量，因此前缀缓存命中可以避免重新预填充整个前缀，其成本比未命中低几个数量级。因此，我们将每个请求路由到存储其前缀缓存的集群，因为将缓存移动到另一个集群需要通过跨集群链路传输，而跨集群链路的速度远慢于集群内部的网络。然而，这种基于缓存的亲和性会将每个会话绑定到一个单一的集群，该集群的故障将中断所有绑定到它的会话。因此，一致性哈希将每个会话绑定到两个集群，一个主要集群处理其流量，一个预先分配的次要集群在主要集群发生故障时接管。次要集群不存储会话的前缀缓存，因此在故障转移时必须重新预填充该缓存。由于一致性哈希将不同会话的次要集群分配均匀地分布在整个集群舰队中，因此这种重新预填充的工作被分散到多个集群，而不是集中在单一集群。因此，在常见情况下，缓存局部性得以保留，而任何单个集群故障的影响也保持在可控范围内。

Budget-based admission control Production traffic mixes short requests under 2K tokens with ultra-long requests up to 1M tokens, so the per-request cost spans roughly three orders of magnitude and the total load imposed by any fixed number of requests is highly unpredictable. Capacity planning, queueing models, and rate-limiting quotas based on the “average request” all break down under this variance. In a typical failure mode, a burst of long-context requests saturates the available compute, and short requests arriving afterwards cannot be scheduled promptly,

degrading time to first token (TTFT) across all traffic. We therefore adopt budget-based admission control, allocating separate resource budgets to different request classes so that bursty long-context traffic consumes at most its own share of the capacity and cannot degrade system-wide SLOs experienced by other classes.

基于预算的准入控制生产流量混合了短请求（少于 2K 个标记）和超长请求（高达 1M 个标记），因此每个请求的成本跨度大约有三个数量级，任何固定数量的请求所带来的总负载都高度不可预测。基于“平均请求”的容量规划、排队模型和速率限制配额在面对这种变化时都会失效。在典型的故障模式中，一波长上下文请求会饱和可用的计算资源，随后到达的短请求无法及时调度，从而导致所有流量的首次令牌时间 (TTFT) 下降。因此，我们采用基于预算的准入控制，为不同的请求类别分配独立的资源预算，使得突发的长上下文流量最多消耗其自身的容量份额，而不会影响其他类别所体验到的系统级服务等级协议（SLO）。

6 Evaluations

6 评估

6.1 Main Results

6.1 主要结果

6.1.1 Benchmarks

6.1.1 基准测试

We evaluate Kimi K3 on a comprehensive benchmark suite organized along four broad capability axes:

我们评估 Kimi K3 在一个全面的基准测试套件上，该套件按照四个广泛的能力轴进行组织：

- Reasoning & Knowledge: GPQA Diamond [101], CritPt [8], AA-LCR [9], and Humanity’s Last Exam (HLE-Full, with and without tools) [93].
- Coding: DeepSWE [31], ProgramBench [95], Terminal-Bench 2.1 [78], FrontierSWE [35], SWE-Marathon [117], PostTrainBench [94], MLS-Bench-Lite [76], and SciCode [121, 8].
- Agentic: BrowseComp [131], DeepSearchQA [126], ResearchRubrics [106], Toolathlon-Verified [69], MCPMark-Verified [133], MCP-Atlas [11], AutomationBench [108], JobBench [70], GDPval-AA v2 [90], AA-Briefcase [8, 2], Agents’ Last Exam (ALE) [4, 115], APEX-Agents [127], OfficeQA Pro [87], SpreadsheetBench 2 [150], OSWorld-Verified [136] and OSWorld 2.0 [143], SaaS-Bench [109], τ^3 -Banking [1, 8], Harvey Lab-AA [8, 42], CorpFin v2 [21], Finance Agent v2 [34], and Legal Research Bench [65].
- 推理与知识：GPQA Diamond [101], CritPt [8], AA-LCR [9]，以及人类最后的考试（HLE-Full，带工具和不带工具）[93]。
- 编程：DeepSWE [31], ProgramBench [95], Terminal-Bench 2.1 [78], FrontierSWE [35], SWE-Marathon [117], PostTrainBench [94], MLS-Bench-Lite [76]，以及 SciCode [121, 8]。
- 代理能力：BrowseComp [131], DeepSearchQA [126], ResearchRubrics [106], Toolathlon-Verified [69], MCPMark-Verified [133], MCP-Atlas [11], AutomationBench [108], JobBench [70], GDPval-AA v2 [90], AA-Briefcase [8, 2]，代理最后的考试（ALE）[4, 115], APEX-Agents

[127], OfficeQA Pro [87], SpreadsheetBench 2 [150], OSWorld-Verified [136] 和 OSWorld 2.0 [143], SaaS-Bench [109], τ^3 -Banking [1, 8], Harvey Lab-AA [8, 42], CorpFin v2 [21], Finance Agent v2 [34], 以及法律研究基准 [65]。

- Vision: WorldVQA [149], OmniDocBench [88], PerceptionBench [62], Video-MME [36], MMVU [148], and BabyVision [13] with Python tool. MMMU-Pro [144], CharXiv (RQ) [130], Math-Vision [128], and ZeroBenchmark [102], each with and without Python tool augmentation.

- 视觉: WorldVQA [149]、OmniDocBench [88]、PerceptionBench [62]、Video-MME [36]、MMVU [148] 和 BabyVision [13] 使用 Python 工具。MMMU-Pro [144]、CharXiv (RQ) [130]、Math-Vision [128] 和 ZeroBenchmark [102], 每个都包含和不含 Python 工具增强。

6.1.2 Baselines

6.1.2 基线

We benchmark against state-of-the-art proprietary and open-source models. For proprietary models, we compare against Claude Fable 5 [16], GPT-5.6 Sol [39], Claude Opus 4.8 [17], and GPT-5.5 [38]. The results of Claude Fable 5 include fallback behaviors and the results of GPT-5.6 Sol include potential cyberguards. For open-source models, we include GLM-5.2 [37]. All models are evaluated at maximum reasoning effort, except GPT-5.5, which uses the “xhigh” setting.

我们与最先进的专有和开源模型进行基准测试。对于专有模型，我们与 Claude Fable 5 [16]、GPT-5.6 Sol [39]、Claude Opus 4.8 [17] 和 GPT-5.5 [38] 进行比较。Claude Fable 5 的结果包括回退行为，而 GPT-5.6 Sol 的结果包括潜在的网络防护措施。对于开源模型，我们包括 GLM-5.2 [37]。所有模型均在最大推理努力下进行评估，除了 GPT-5.5，它使用的是“xhigh”设置。

6.1.3 Evaluation Configurations

6.1.3 评估配置

All Kimi K3 evaluations use reasoning effort max and temperature = 1.0. For single-step tasks, such as GPQA Diamond, HLE-Full, and vision benchmarks without tools, we set top-p = 0.95. For agentic tasks, we set top-p = 1.0. Generally, we recommend using top-p = 0.95 for reasoning and knowledge tasks, and top-p = 1.0 for coding and agentic scenarios.

所有 Kimi K3 的评估都使用最大推理努力度和温度值 = 1.0。对于单步任务，如 GPQA Diamond、HLE-Full 和没有工具的视觉基准测试，我们设置 top-p = 0.95。对于代理任务，我们设置 top-p = 1.0。通常，我们建议在推理和知识类任务中使用 top-p = 0.95，在编码和代理场景中使用 top-p = 1.0。

Coding Each model is evaluated under one of three agentic harnesses: Kimi Code [56], Claude Code [15], or Codex [20]. On DeepSWE, we report results on the v1.1 tasks, with additional reference to the official leaderboard (Kimi K3 attains 67.3 with the mini-SWE-agent harness). On Terminal-Bench 2.1, we report the best score across harnesses for all models. Our SWE-Marathon evaluation is based on an H20-calibrated branch of the official tasks as of July 9, 2026, prior to the final v1.1 release, with Docker images, performance gates, and reference oracles for the GPU tasks recalibrated for H20 but the correctness and anti-cheat validators unchanged; Claude Fable 5 hits fallbacks on 35% of the tasks. For PostTrainBench, we evaluate Kimi K3, Claude Fable 5, and GPT-5.6 Sol using the official Harbor implementation at maximum effort, averaged over three runs on H20 GPUs (instead of H100 in the official setting). FrontierSWE dominance scores are recomputed from raw scores using the official evaluation script as of July 16, 2026.

每个模型在以下三种代理框架之一下进行评估：Kimi Code [56]、Claude Code [15] 或 Codex [20]。在 DeepSWE 上，我们报告了 v1.1 任务的结果，并参考了官方排行榜（Kimi K3 在 mini-SWE-agent 框架下达到 67.3）。在 Terminal-Bench 2.1 上，我们报告了所有模型在不同框架下的最佳得分。我们的

SWE-Marathon 评估基于 2026 年 7 月 9 日官方任务的 H20 校准分支，该日期早于最终 v1.1 版本发布，Docker 镜像、性能关卡以及 GPU 任务的参考或 acles 已重新校准为 H20，但正确性验证器和反作弊验证器保持不变；Claude Fable 5 在 35% 的任务中触发了回退机制。对于 PostTrainBench，我们使用官方 Harbor 实现，在最大努力下评估 Kimi K3、Claude Fable 5 和 GPT-5.6 Sol，平均三个运行结果（使用 H20 GPU 而非官方设置中的 H100）。FrontierSWE 主导得分是根据 2026 年 7 月 16 日的原始得分使用官方评估脚本重新计算的。

Agentic For OfficeQA Pro, each test case provides the agent with the entire PDF corpus rendered as images, with no machine-readable text available. MCP-Atlas is evaluated on the 500-task public subset with a 100-turn limit, using Gemini 3.1 Pro as the judge. AutomationBench is evaluated on the 600-task public subset. For BrowseComp we adopt a context-compaction strategy triggered at 300K tokens; evaluated with the full 1M-token context window and no context management, Kimi K3 achieves 90.4%.

Agentic For OfficeQA Pro，每个测试用例为代理提供整个 PDF 语料库的图像形式，没有可读取的机器文本。MCP-Atlas 在 500 任务的公共子集上进行评估，限制为 100 轮，使用 Gemini 3.1 Pro 作为裁判。AutomationBench 在 600 任务的公共子集上进行评估。对于 BrowseComp，我们采用在 30 万 token 时触发的上下文压缩策略；使用完整的 100 万 token 上下文窗口且不进行上下文管理，Kimi K3 的准确率为 90.4%。

Vision Scores are averaged over three runs, except ZeroBench-main, which we run five times following the official setting. MMMU-Pro follows the official protocol, preserving the original input order and prepending images to the text input. For WorldVQA, we observe consistent refusal behavior across models and enforce an answer via prompt engineering.

Vision Scores 在三个运行中取平均值，除了 ZeroBench-main，我们按照官方设置运行五次。MMMU-Pro 遵循官方协议，保持原始输入顺序，并在文本输入前添加图像。对于 WorldVQA，我们在所有模型中观察到一致的拒绝行为，并通过提示工程强制生成答案。

Third-party results GDPval-AA v2, AA-Briefcase, τ^3 -Banking, Harvey Lab-AA, APEX-Agents, Sci-Code, AA-LCR, and CritPt scores are cited from Artificial Analysis [8] as of July 23, 2026. For Harvey Lab-AA, we report the criterion pass rate. CorpFin v2, Finance Agent v2, and Legal Research Bench scores are cited from Vals AI [124]. Agents' Last Exam scores are cited from the official leaderboard [4] as of July 23, 2026; we report the leaderboard's primary pass-rate metric. On the leaderboard, each model is paired with a specific harness: Kimi K3 with Kimi Code; GPT-5.6 Sol, GPT-5.5 with Codex; and Claude Fable 5, Claude Opus 4.8, and GLM-5.2 with Claude Code. Toolathlon-verified and JobBench scores are cited from their official leaderboards [119, 52] as of July 24, 2026.

第三方结果 GDPval-AA v2, AA-Briefcase, τ^3 -Banking, Harvey Lab-AA, APEX-Agents, SciCode, AA-LCR, 和 CritPt 分数引用自 Artificial Analysis [8]（2026 年 7 月 23 日）。对于 Harvey Lab-AA，我们报告的是通过标准的合格率。CorpFin v2、Finance Agent v2 和 Legal Research Bench 分数引用自 Vals AI [124]。代理的最后考试分数引用自官方排行榜 [4]（2026 年 7 月 23 日）；我们报告的是排行榜的主要通过率指标。在排行榜上，每个模型都与特定的工具配对：Kimi K3 与 Kimi Code；GPT-5.6 Sol、GPT-5.5 与 Codex；Claude Fable 5、Claude Opus 4.8 和 GLM-5.2 与 Claude Code。Toolathlon 验证和 JobBench 分数引用自其官方排行榜 [119, 52]（2026 年 7 月 24 日）。

6.1.4 Results

6.1.4 结果

Table 2 provides a comprehensive comparison of Kimi K3 against both proprietary and open-source baselines. Overall, Kimi K3 closely trails the strongest proprietary models, Claude Fable 5 and GPT-5.6 Sol, while consistently outperforming Claude Opus 4.8, GPT-5.5, and GLM-5.2 across the benchmark suite. We highlight key observations across core capability domains below:

表 2 提供了 Kimi K3 与自有模型和开源基线的全面比较。总体而言，Kimi K3 紧随最强的自有模型 Claude

Table 5 和 GPT-5.6 Sol 之后，而在基准测试套件中始终优于 Claude Opus 4.8、GPT-5.5 和 GLM-5.2。我们在以下几个核心能力领域中突出了一些关键观察结果：

Reasoning & Knowledge On graduate-level reasoning, Kimi K3 is competitive with the frontier, scoring 93.5% on GPQA Diamond. However, a gap remains on research-level tasks: on HLE-Full it trails Claude Fable 5 and GPT-5.6 Sol both with and without tools, at 56.0% and 43.5% respectively; and on CritPt it scores 23.4%, lagging behind Claude Fable 5, GPT-5.6 Sol, and GPT-5.5, indicating that research-level reasoning remains a key direction for improvement.

推理与知识在研究生级别的推理任务中，Kimi K3 具有竞争力，其在 GPQA Diamond 上的得分达到 93.5%。然而，在研究级任务上仍存在差距：在 HLE-Full 上，其表现落后于 Claude Fable 5 和 GPT-5.6 Sol，分别在有工具和无工具的情况下得分为 56.0% 和 43.5%；在 CritPt 上，其得分为 23.4%，落后于 Claude Fable 5、GPT-5.6 Sol 和 GPT-5.5，表明研究级推理仍然是需要改进的关键方向。

Table 2: Performance comparison of Kimi K3 against proprietary and open-source models. Bold denotes the best result for each benchmark and underline the second-best. Unless otherwise noted, Kimi K3 results are obtained with reasoning effort set to max and temperature equal to 1.0. For HLE-Full, MMMU-Pro, CharXiv (RQ), Math-Vision, and ZeroBench, each cell reports the scores without and with tool augmentation (general tools for HLE-Full, Python for the vision benchmarks), in that order. [†] On the official Agents’ Last Exam leaderboard, the Claude Fable 5 entry runs at xhigh effort with 40% of tasks annotated as downgraded.

表 2：Kimi K3 与专有模型和开源模型的性能对比。粗体表示每个基准测试中的最佳结果，下划线表示次佳结果。除非另有说明，Kimi K3 的结果是在推理努力设置为最大值且温度等于 1.0 的情况下获得的。对于 HLE-Full、MMMU-Pro、CharXiv (RQ)、Math-Vision 和 ZeroBench，每个单元格按顺序报告了在不使用工具增强和使用工具增强（HLE-Full 的一般工具、视觉基准测试的 Python）下的得分。[†] 在官方的 Agents’ Last Exam 排行榜上，Claude Fable 5 的条目在高努力度下运行，有 40% 的任务被标记为降级。

Benchmark	Kimi K3 (max)	Proprietary			Open Weight	
	GPT-5.6 Sol (max)	Claude Opus 4.8 (max)	GPT-5.5 (xhigh)	GLM-5.2 (max)		
Claude Fable 5 (max, w/ fallback)						
Reasoning & Knowledge						
GPQA Diamond	93.5	92.6	94.1	91.0	93.5	91.2
CritPt	23.4	28.6	32.3	20.9	27.1	20.9
AA-LCR	74.7	70.0	73.7	67.7	74.3	71.3
HLE-Full	43.5 / 56.0	53.3 / 63.0	44.5 / 58.0	49.8 / 57.9	41.4 / 52.2	-
Coding						
DeepSWE	67.5	70.0	73.0	59.0	67.0	46.2
ProgramBench	77.8	76.8	77.6	71.9	70.8	63.7
Terminal- Bench 2.1	88.3	88.0	88.8	84.6	83.4	82.7
FrontierSWE	81.2	86.6	71.3	66.7	64.9	67.3
SWE- Marathon	42.0	35.0	39.0	40.0	14.0	13.0
PostTrainBench	36.6	41.4	34.6	34.1	28.4	34.3
MLS- Bench-Lite	48.3	49.9	46.2	42.8	35.5	40.4

SciCode	58.7	60.2	56.1	53.5	56.1	50.5
Agentic						
BrowseComp	91.2	88.0	90.4	84.3	84.4	-
DeepSearchQA (F1)	95.0	94.2	-	93.1	-	-
ResearchRubric	76.2	-	73.8	73.5	64.0	71.1
GDPval-AA v2 (Elo)	1686	1747	1736	1593	1491	1510
Toolathlon-Verified	76.5	77.9	74.9	76.2	73.5	59.9
MCPMark-Verified	94.5	87.4	92.9	76.4	92.9	-
MCP-Atlas	84.2	84.7	83.6	83.6	82.8	82.6
AutomationBench	30.8	29.1	29.7	27.2	22.7	12.9
JobBench	54.3	57.4	45.4	48.4	38.3	43.4
AA-Briefcase (Elo)	1548	1583	1495	1354	1158	1260
Agents' Last Exam	28.3	25.7 [†]	29.6	27.0	26.6	20.4
APEX-Agents	41.0	43.3	39.9	39.4	38.5	35.6
OfficeQA Pro	63.3	69.9	63.2	63.9	60.9	41.4
SpreadsheetBench 2	34.8	34.7	32.4	31.6	29.1	28.1
OSWorld-Verified	84.8	85.0	83.0	83.4	79.0	-
OSWorld 2.0	58.3	66.1	62.6	55.7	49.5	-
SaaS-Bench	60.1	-	61.4	56.1	43.8	-
τ^3 -Banking	33.4	26.8	33.0	27.6	31.3	26.8
Harvey Lab-AA	94.6	93.6	87.2	91.1	86.3	91.0
CorpFin v2	71.6	71.8	64.4	66.7	68.4	66.1
Finance Agent v2	54.4	56.3	53.8	53.9	51.8	49.7
Legal Research Bench	44.2	49.5	48.1	43.8	40.4	31.3
Vision						
WorldVQA	51.0	56.7	41.8	39.1	38.5	-
ForceAnswer						
OmniDocBench	91.1	89.8	85.8	87.9	89.4	-
PerceptionBench	58.5	57.2	59.7	47.2	55.8	-

Video-MME (w/ sub)	90.0	-	89.5	86.0	89.3	-
MMVU	82.1	-	81.2	79.2	81.7	-
BabyVision w/ Python	85.7	90.5	88.9	81.2	83.6	-
MMMU-Pro	81.6 / 83.4	81.2 / 86.5	83.0 / 84.6	78.9 / 82.7	81.2 / 83.2	-
CharXiv (RQ)	84.8 / 91.3	88.9 / 93.5	84.6 / 89.1	80.5 / 89.9	84.1 / 89.0	-
Math-Vision	94.3 / 97.8	94.8 / 98.6	95.8 / 97.8	86.7 / 97.1	92.2 / 96.8	-
ZeroBench-main (pass@5)	23.0 / 41.0	23.0 / 46.0	17.0 / 35.0	17.0 / 34.0	22.0 / 41.0	-

基准测试	Kimi K3 (max)	专有				开放权重
Claude Fable 5 (最大, 带备用方案)	GPT-5.6 Sol (max)	Claude Opus 4.8 (max)	GPT-5.5 (xhigh)	GLM-5.2 (max)		
推理与知识						
GPQA Diamond	93.5	92.6	94.1	91.0	93.5	91.2
批评点	23.4	28.6	32.3	20.9	27.1	20.9
AA-LCR	74.7	70.0	73.7	67.7	74.3	71.3
HLE-Full	43.5 / 56.0	53.3 / 63.0	44.5 / 58.0	49.8 / 57.9	41.4 / 52.2	-
编程						
DeepSWE	67.5	70.0	73.0	59.0	67.0	46.2
程序基准	77.8	76.8	77.6	71.9	70.8	63.7
Terminal-Bench 2.1	88.3	88.0	88.8	84.6	83.4	82.7
FrontierSWE	81.2	86.6	71.3	66.7	64.9	67.3
SWE-Marathon	42.0	35.0	39.0	40.0	14.0	13.0
PostTrainBench	36.6	41.4	34.6	34.1	28.4	34.3
MLS-Bench-Lite	48.3	49.9	46.2	42.8	35.5	40.4
SciCode	58.7	60.2	56.1	53.5	56.1	50.5
代理性						
浏览组件	91.2	88.0	90.4	84.3	84.4	-
DeepSearchQA (F1)	95.0	94.2	-	93.1	-	-
研究评分标准	76.2	-	73.8	73.5	64.0	71.1

GDPval-AA v2 (Elo)	1686	1747	1736	1593	1491	1510
Toolathlon-Verified	76.5	77.9	74.9	76.2	73.5	59.9
MCPMark-Verified	94.5	87.4	92.9	76.4	92.9	-
MCP-Atlas	84.2	84.7	83.6	83.6	82.8	82.6
自动化基准测试 (AutomationBench)	30.8	29.1	29.7	27.2	22.7	12.9
JobBench	54.3	57.4	45.4	48.4	38.3	43.4
AA-Briefcase (Elo)	1548	1583	1495	1354	1158	1260
代理人的最后一次考试	28.3	25.7 [†]	29.6	27.0	26.6	20.4
APEX-Agents	41.0	43.3	39.9	39.4	38.5	35.6
OfficeQA Pro	63.3	69.9	63.2	63.9	60.9	41.4
SpreadsheetBench 2	34.8	34.7	32.4	31.6	29.1	28.1
OSWorld-Verified	84.8	85.0	83.0	83.4	79.0	-
OSWorld 2.0	58.3	66.1	62.6	55.7	49.5	-
SaaS-Bench	60.1	-	61.4	56.1	43.8	-
τ^3 -银行	33.4	26.8	33.0	27.6	31.3	26.8
Harvey Lab-AA	94.6	93.6	87.2	91.1	86.3	91.0
CorpFin v2	71.6	71.8	64.4	66.7	68.4	66.1
金融代理 v2	54.4	56.3	53.8	53.9	51.8	49.7
法律研究平台	44.2	49.5	48.1	43.8	40.4	31.3
视觉						
WorldVQA	51.0	56.7	41.8	39.1	38.5	-
ForceAnswer						
OmniDocBench	91.1	89.8	85.8	87.9	89.4	-
PerceptionBench	58.5	57.2	59.7	47.2	55.8	-
视频-MME (带子)	90.0	-	89.5	86.0	89.3	-
MMVU	82.1	-	81.2	79.2	81.7	-
BabyVision w/ Python	85.7	90.5	88.9	81.2	83.6	-

MMMU-Pro	81.6 / 83.4	81.2 / 86.5	83.0 / 84.6	78.9 / 82.7	81.2 / 83.2	-
CharXiv (RQ)	84.8 / 91.3	88.9 / 93.5	84.6 / 89.1	80.5 / 89.9	84.1 / 89.0	-
数学视觉	94.3 / 97.8	94.8 / 98.6	95.8 / 97.8	86.7 / 97.1	92.2 / 96.8	-
ZeroBench-main (pass@5)	23.0 / 41.0	23.0 / 46.0	17.0 / 35.0	17.0 / 34.0	22.0 / 41.0	-

Coding Kimi K3 delivers strong agentic coding performance. It attains the best score on ProgramBench (77.8%), and on SWE-Marathon—a GPU-kernel-oriented suite—it scores 42.0%, 7 points ahead of Claude Fable 5. On Terminal-Bench 2.1, it nearly matches GPT-5.6 Sol (88.3% vs. 88.8%). On DeepSWE, it ranks behind Claude Fable 5 and GPT-5.6 Sol but ahead of Claude Opus 4.8 and GPT-5.5. On FrontierSWE, a long-horizon benchmark, it ranks second with a score of 81.2% as of July 16, 2026, behind only Claude Fable 5 (86.6%) and well ahead of all other models.

Coding Kimi K3 提供了强大的代理编码性能。它在 ProgramBench 上取得了最佳成绩 (77.8%)，在面向 GPU 内核的 SWE-Marathon 套件上得分 42.0%，比 Claude Fable 5 高出 7 分。在 Terminal-Bench 2.1 上，它几乎与 GPT-5.6 Sol 相当 (88.3% 对 88.8%)。在 DeepSWE 上，它排名落后于 Claude Fable 5 和 GPT-5.6 Sol，但领先于 Claude Opus 4.8 和 GPT-5.5。在长视野基准测试 FrontierSWE 上，截至 2026 年 7 月 16 日，它以 81.2% 的得分位列第二，仅次于 Claude Fable 5 (86.6%)，远超其他所有模型。

Agentic Kimi K3 achieves state-of-the-art results on a broad set of agentic suites, including BrowseComp (91.2%), DeepSearchQA (95.0% F1 score), ResearchRubrics (76.2%), MCPMark-Verified (94.5%), AutomationBench (30.8%), SpreadsheetBench 2 (34.8%), τ^3 -Banking (33.4%), and Harvey Lab-AA (94.6% criterion pass rate). The main exceptions are the Elo-rated knowledge-work suites, both led by Claude Fable 5: Kimi K3 places third on GDPval-AA v2 (1,686) and second on AA-Briefcase (1,548). Elsewhere it is largely competitive: on CorpFin v2 and OSWorld-Verified, it finishes just 0.2 points behind Claude Fable 5 (71.6% vs. 71.8% and 84.8% vs. 85.0%, respectively), while the remaining harder computer-use benchmarks (OSWorld 2.0, SaaS-Bench) are still led by Claude Fable 5 or GPT-5.6 Sol.

代理型 Kimi K3 在一系列代理型基准测试中取得了最先进的结果，包括 BrowseComp (91.2%)、DeepSearchQA (F1 得分为 95.0%)、ResearchRubrics (76.2%)、MCPMark-Verified (94.5%)、AutomationBench (30.8%)、SpreadsheetBench 2 (34.8%)、 τ^3 -Banking (33.4%) 以及 Harvey Lab-AA (94.6% 的指标通过率)。主要例外的是以 Elo 评分的知识型工作基准测试，这些测试均由 Claude Fable 5 主导：Kimi K3 在 GDPval-AA v2 中排名第三 (1,686 分)，在 AA-Briefcase 中排名第二 (1,548 分)。在其他方面，Kimi K3 表现非常具有竞争力：在 CorpFin v2 和 OSWorld-Verified 中，其得分仅比 Claude Fable 5 低 0.2 分 (分别为 71.6% vs. 71.8% 和 84.8% vs. 85.0%)。而在其余更难的计算机使用基准测试 (如 OSWorld 2.0 和 SaaS-Bench) 中，仍由 Claude Fable 5 或 GPT-5.6 Sol 领先。

Vision Kimi K3 exhibits strong multimodal understanding capabilities, which are further amplified by Python tools: on Math-Vision it reaches 94.3%, rising to 97.8% with Python tools, and on the challenging ZeroBench-main it ties Claude Fable 5 at 23.0% (pass@5), jumping to 41.0% with Python tools. It also achieves the highest score on OmniDocBench (91.1%) and, on WorldVQA (51.0%), ranks second behind Claude Fable 5, ahead of GPT-5.6 Sol and Claude Opus 4.8.

Vision Kimi K3 展现出强大的多模态理解能力，这些能力通过 Python 工具得到了进一步增强：在 Math-Vision 上达到 94.3%，借助 Python 工具后提升至 97.8%；在具有挑战性的 ZeroBench-main 上，以 23.0% (pass@5) 的成绩与 Claude Fable 5 平分秋色，借助 Python 工具后跃升至 41.0%。它还在 OmniDocBench 上取得了最高分 (91.1%)，在 WorldVQA (51.0%) 上排名第二，仅次于 Claude Fable 5，领先于 GPT-5.6 Sol 和 Claude Opus 4.8。

6.2 Internal Evaluation

6.2 内部评估

6.2.1 Capability Evaluation

6.2.1 能力评估

Beyond the public benchmark suite, we maintain a collection of in-house benchmarks that target capability areas public evaluations do not adequately cover, giving a more comprehensive measure of model and agent capabilities. These benchmarks are refreshed and expanded frequently, so that they can closely track the model's evolving failure modes and directly guide data and training iterations. They broadly fall into three categories: coding capability and experience, general agent experience, and conversational experience. Table 3 reports the results across these benchmarks.

除了公开基准测试套件之外，我们还维护了一套内部基准测试，这些测试针对公开评估未能充分覆盖的能力领域，从而提供对模型和代理能力更为全面的衡量。这些基准测试经常被更新和扩展，以便能够紧密跟踪模型不断演变的失败模式，并直接指导数据和训练迭代。它们大致分为三类：编码能力和经验、通用代理经验以及对话经验。表 3 报告了这些基准测试的结果。

Coding Capability and Experience

编码能力与经验

- Kimi Code Bench 2.0 (KCB 2.0): evaluates code agents on realistic, end-to-end software engineering tasks across a broad range of programming languages and production-oriented technology stacks.
- Kimi Webdev Bench: evaluates models on challenging web development prompts drawn from real usage scenarios, with outputs compared through blind expert judgment, with results available in Table 4.
- Coding Experience: evaluates the practical experience of working with the model as a coding agent in real development workflows.
- Kimi Code Bench 2.0 (KCB 2.0): 在多种编程语言和面向生产的科技栈上，评估代码代理在现实、端到端软件工程任务中的表现。
- Kimi Webdev Bench: 从真实使用场景中提取具有挑战性的网页开发提示，对模型进行评估，通过盲评专家判断进行输出比较，结果见表 4。
- Coding Experience: 评估在真实开发流程中将模型作为编码代理进行工作的实际经验。

General Agent Experience

一般代理经验

- 24/7 ClawBench 2.0: simulates always-on assistant work, in which tasks span multiple days, events arrive concurrently, and interruptions are routine.

- Multi-Agent Infra for Routing and Assignment (MIRA) Bench: evaluates long-chain, multi-role, multi-system enterprise collaboration tasks, assessing whether agents can carry out end-to-end work and judge when to organize or delegate to subagents.
- Kimi Autonomous Execution Tasks (KAET): evaluates long-horizon autonomous execution on tasks simulating real user requests and enterprise system operations.
- Context Learning and Instruction Following (CLIF) Bench: targets in-context learning, requiring models to learn from a provided context while following instructions that interleave multiple complex skills.
- Agentic Vision Bench: evaluates whether agents notice and correctly use key visual facts during task execution.
- Swarm Bench: evaluates models' ability to orchestrate agent swarms [59] on complex tasks that benefit from coordinated decomposition and parallel execution.
- Online Experience: mirrors the distribution of real online agent usage, measuring performance on the deliverable file types most frequently requested by users.
- 24/7 ClawBench 2.0: 模拟始终在线的助手工作，其中任务跨越多天，事件并发到达，中断是常态。
- 多代理路由与分配基础设施（MIRA）基准：评估长链、多角色、多系统的企业协作任务，评估代理是否能够完成端到端工作，并判断何时组织或委派给子代理。
- 通义千问自主执行任务（KAET）：评估在模拟真实用户请求和企业系统操作的任务上进行长期自主执行的能力。
- 上下文学习与指令遵循（CLIF）基准：针对上下文学习，要求模型在遵循交织多个复杂技能的指令的同时，从提供的上下文中学习。
- 代理视觉基准：评估代理在任务执行过程中是否能够注意到并正确使用关键的视觉信息。
- 群体基准：评估模型在复杂任务上协调代理群体的能力，这些任务通过协调分解和并行执行而受益。
- 在线体验：模拟真实在线代理使用的分布情况，测量在用户最常请求的可交付文件类型上的性能。

Table 3: Results on our in-house benchmarks. Bold denotes the best reported result per benchmark; “-” denotes scores not yet included in this report. Unless otherwise noted, models are evaluated at maximum reasoning effort (GPT-5.5 at xhigh); harness assignments are shown in the Harness column. ^a 13 fallbacks and 1 refusal out of 80 tasks. ^b 10 refusals out of 80 tasks. ^c 3 refusals out of 80 tasks. ^d Includes 2 tasks that Claude Fable 5 refused to answer. ^e Includes 14 tasks that Claude Fable 5 refused to answer. ^f 6 refusals out of 95 tasks. ^g Reported metric is 1—hallucination rate; higher is better.

表 3：我们在内部基准测试中的结果。加粗表示每个基准测试中最好的报告结果；“-”表示尚未包含在此报

告中的分数。除非另有说明，模型均在最大推理努力下进行评估（GPT-5.5 在 xhigh 下）；Harness 分配在 Harness 列中显示。^a 80 个任务中有 13 个备用方案和 1 个拒绝。^b 80 个任务中有 10 个拒绝。^c 80 个任务中有 3 个拒绝。^d 包含 2 个 Claude Fable 5 拒绝回答的任务。^e 包含 14 个 Claude Fable 5 拒绝回答的任务。^f 95 个任务中有 6 个拒绝。^g 报告的指标是 1—幻觉率；数值越高越好。

Benchmark Harness		Kimi K3 (max)	Proprietary				Open Weight
Claude Fable 5 (max)	GPT-5.6 Sol (max)	Claude Opus 4.8 (max)	GPT-5.5 (xhigh)	GLM-5.2 (max)			
Coding Experience							
Kimi Code Bench 2.0	Claude Code	73.7	76.9 ^a	-	71.7	-	64.2
Kimi Code	72.9	-	-	-	66.0	-	
Codex	-	-	64.8 ^b	-	69.0 ^c	-	
Coding Experience	Claude Code	59.9	59.8	-	58.0	-	53.3
Kimi Code	56.6	-	-	-	-	-	
Codex	-	-	59.3	-	56.8	-	
General Agent Experience							
24/7 Claw-Bench 2.0	OpenClaw	48.3	47.4 ^d	52.0	47.2	48.5	43.2
MIRA Bench	MIRA	64.1	72.9	62.2	59.8	54.6	-
KAET	Kimi Code	83.5	-	85.4	78.7	79.7	74.7
CLIF Bench	Kimi Code	52.4	-	50.6	48.8	52.3	39.2
Agentic Vision Bench	Kimi Code	78.3	81.1	82.9	82.8	76.9	-
Swarm Bench	Kimi Agent	76.3	-	73.2	72.6	61.8	58.5
Online Experience	Kimi Agent	77.9	74.2 ^e	84.0	69.4	73.7	64.0
Deep Research Bench	Kimi Agent	90.0	-	85.3	87.2	81.9	84.0
Finance Bench	N/A	62.6	-	62.7	60.7	58.4	55.4

KWV Bench	N/A	64.7	63.6	66.9	61.7	65.8	-
DECK Bench	N/A	73.5	73.0	74.7	66.9	68.2	68.6
Agent Behavior Bench	Kimi Work	65.0	75.5 ^f	76.4	65.7	70.1	-
Conversational Experience							
Faithfulness	N/A	85.5	-	84.8	83.6	86.5	74.8
Chat All-in-One Bench	Kimi Work	85.2	88.0	79.0	83.8	71.8	-

基准测试	工具	Kimi K3 (max)	专有				开放权重
Claude Fable 5 (max)	GPT-5.6 Sol (max)	Claude Opus 4.8 (max)	GPT-5.5 (xhigh)	GLM-5.2 (max)			
编程经验							
Kimi Code Bench 2.0	Claude 代码	73.7	76.9 ^a	-	71.7	-	64.2
Kimi Code	72.9	-	-	-	66.0	-	
《编年史》	-	-	64.8 ^b	-	69.0 ^c	-	
编程经验	Claude 代码	59.9	59.8	-	58.0	-	53.3
Kimi Code	56.6	-	-	-	-	-	
《编年史》	-	-	59.3	-	56.8	-	
一般代理经验							
24/7 Claw-Bench 2.0	OpenClaw	48.3	47.4 ^d	52.0	47.2	48.5	43.2
MIRA Bench	MIRA	64.1	72.9	62.2	59.8	54.6	-
KAET	Kimi Code	83.5	-	85.4	78.7	79.7	74.7
CLIF Bench	Kimi Code	52.4	-	50.6	48.8	52.3	39.2

代理视觉 基准测试 (Agentic Vision Bench)	Kimi Code	78.3	81.1	82.9	82.8	76.9	-
Swarm Bench	Kimi Agent	76.3	-	73.2	72.6	61.8	58.5
在线体验	Kimi Agent	77.9	74.2 ^e	84.0	69.4	73.7	64.0
深度研究 基准	Kimi Agent	90.0	-	85.3	87.2	81.9	84.0
金融法庭	N/A	62.6	-	62.7	60.7	58.4	55.4
KWV Bench	N/A	64.7	63.6	66.9	61.7	65.8	-
DECK Bench	N/A	73.5	73.0	74.7	66.9	68.2	68.6
代理行为 基准	Kimi Work	65.0	75.5 ^f	76.4	65.7	70.1	-
对话体验							
忠诚度	N/A	85.5	-	84.8	83.6	86.5	74.8
Chat All- in-One Bench	Kimi Work	85.2	88.0	79.0	83.8	71.8	-

Table 4: Results on the in-house Kimi Webdev Bench: Kimi K3 (max) against Claude Opus 4.8 (max), both run with the Claude Code harness. The comparison is performed under blind expert judging, where experts score each output on code quality, feature completeness, visual fidelity, and interaction experience without knowing which model produced it. Win, Tie, and Lose report the percentage of prompts where Kimi K3’s output is preferred, rated comparable, or dispreferred, respectively.

表 4：在内部 Kimi Webdev Bench 上的结果：Kimi K3（最大）与 Claude Opus 4.8（最大）的对比结果，两者均使用 Claude Code 框架运行。比较是在盲评专家的评判下进行的，专家在不知道哪个模型生成输出的情况下，对每个输出进行评分，评估代码质量、功能完整性、视觉保真度和交互体验。Win、Tie 和 Lose 分别报告了 Kimi K3 的输出在提示中被偏好、评分相当或不被偏好的百分比。

Domain	Win	Tie	Lose	Win – Lose
Games	55.6%	3.7%	40.7%	+14.9%
3D / WebGL / Shader	72.7%	13.7%	13.6%	+59.1%
Website / UI Clone	52.6%	21.1%	26.3%	+26.3%
Overall	58.6%	13.8%	27.6%	+31.0%

领域	Win	系	失去	赢 – 输
游戏	55.6%	3.7%	40.7%	+14.9%
3D / WebGL / Shader	72.7%	13.7%	13.6%	+59.1%
网站 / UI 模仿	52.6%	21.1%	26.3%	+26.3%
总体而言	58.6%	13.8%	27.6%	+31.0%

- Deep Research Bench: evaluates models on deep-research-style queries curated by domain experts and graded with expert-aligned rubrics.
- Finance Bench: evaluates models on realistic financial work that requires end-to-end execution of complete workflows, from source materials to reviewable deliverables.
- Knowledge Work Vision (KWV) Bench: evaluates atomic visual capabilities extracted from tasks distilled from real knowledge-work scenarios.
- DECK Bench: measures the capability to produce high-quality presentation decks from task descriptions drawn from real usage scenarios.
- Agent Behavior Bench: extends agent evaluation from outcome correctness to process quality, scoring tool-use behavior, efficiency, and discipline alongside task completion.
- Deep Research Bench: 在由领域专家精心挑选并按照专家标准评分的深度研究风格查询上评估模型。
- Finance Bench: 在需要从原始资料到可审阅成果的完整工作流程端到端执行的现实金融工作中评估模型。
- Knowledge Work Vision (KWV) Bench: 评估从真实知识工作场景中提炼出的任务中提取的原子化视觉能力。
- DECK Bench: 评估从真实使用场景中提取的任务描述生成高质量演示文稿的能力。
- Agent Behavior Bench: 将代理评估从结果正确性扩展到流程质量，同时评估工具使用行为、效率和纪律性以及任务完成情况。

Conversational Experience

对话体验

- Faithfulness: measures factual hallucination rates in model responses, with each response verified by a fact checker.
- Chat All-in-One Bench: measures conversational experience at every stage of product usage, with scenarios designed around real online user needs.
- **忠实度**: 衡量模型回复中的事实幻觉率，每个回复均由事实核查员进行验证。
- **聊天全能基准**: 衡量产品使用过程中各个阶段的对话体验，场景设计围绕真实在线用户需求。

Evaluation Configurations Unless a benchmark is split into separate rows by harness, the Harness column in Table 3 reports the harness used for Kimi K3. For other models, Claude models and GLM-5.2 are evaluated with Claude Code, while GPT models are evaluated with Codex. The exceptions are benchmarks where all models use the same specified harness: OpenClaw for 24/7 Claw-

Bench 2.0; MIRA (Multi-Agent Infra for Routing and Assignment), an internal out-of-distribution harness, for MIRA Bench; Kimi Work for Agent Behavior Bench and Chat All-in-One; and Kimi Code for CLIF and Agentic Vision Bench.

评估配置如果基准测试没有被 Harness 拆分成单独的行，那么表 3 中的 Harness 列报告了用于 Kimi K3 的 Harness。对于其他模型，Claude 模型和 GLM-5.2 使用 Claude Code 进行评估，而 GPT 模型使用 Codex 进行评估。例外情况是所有模型都使用相同指定的 Harness 的基准测试：24/7 ClawBench 2.0 使用 OpenClaw；MIRA Bench 使用 MIRA（用于路由和分配的多代理基础设施），这是一个内部的分布外 Harness；Agent Behavior Bench 和 Chat All-in-One 使用 Kimi Work；CLIF 和 Agentic Vision Bench 使用 Kimi Code。

Results The in-house suite separates Kimi K3's strengths from its weaknesses more sharply than the public benchmarks. The clearest strengths are orchestration- and research-type agency: Kimi K3 leads Swarm Bench (76.3) and Deep Research Bench (90.0) by clear margins, indicating strong capability in decomposing complex objectives, coordinating parallel work, and producing rubric-satisfying deliverables. Coding is likewise a strength: on Kimi Code Bench 2.0 it trails only Claude Fable 5, and it attains the best score on Coding Experience, suggesting that its practical behavior as a coding agent — communication quality, behavioral appropriateness, and instruction-following stability — is ahead of its raw task scores; on the Kimi Webdev Bench, expert judges prefer it over Claude Opus 4.8 by a +31.0-point overall margin, with the largest gain on 3D/WebGL/Shader tasks. Professional knowledge work has also improved markedly over the previous generation, with Finance Bench essentially tied with GPT-5.6 Sol.

结果与公开基准相比，内部工具套件更清晰地区分了 Kimi K3 的优势与劣势。最明显的优势在于编排型和研究型代理能力：Kimi K3 在 Swarm Bench (76.3) 和 Deep Research Bench (90.0) 上以明显优势领先，表明其在分解复杂目标、协调并行工作以及产出符合评分标准的成果方面具有强大的能力。编码同样是一项优势：在 Kimi Code Bench 2.0 上，其表现仅次于 Claude Fable 5，并在 Coding Experience 中获得最佳得分，这表明其作为编码代理的实用行为——沟通质量、行为适当性以及遵循指令的稳定性——超越了其原始任务得分；在 Kimi Webdev Bench 上，专家评委以整体优势 +31.0 分更青睐于 Claude Opus 4.8，其中在 3D/WebGL/Shader 任务上的提升最为显著。专业知识工作也显著改善，Finance Bench 几乎与 GPT-5.6 Sol 打成平手。

Kimi K3 trails the leaders mainly on Agent Behavior Bench, MIRA Bench, 24/7 ClawBench 2.0, Agentic Vision Bench, and KWV Bench. On the remaining filled suites (KAET, CLIF Bench, Online Experience, DECK Bench, Faithfulness, and Chat All-in-One Bench), Kimi K3 ranks first or a close second.

Kimi K3 主要在 Agent Behavior Bench、MIRA Bench、24/7 ClawBench 2.0、Agentic Vision Bench 和 KWV Bench 上落后于领先者。在其余已填充的套件 (KAET、CLIF Bench、Online Experience、DECK Bench、Faithfulness 和 Chat All-in-One Bench) 上，Kimi K3 排名第一或紧随其后。

6.2.2 Cyber Security Evaluation

6.2.2 网络安全评估

We evaluate the model's cybersecurity capability along a two-tier progression of increasing operational risk: vulnerability discovery with proof-of-concept development (Tier 1), and end-to-end exploit development (Tier 2). Evaluation targets include recent versions of widely deployed software—operating-system kernel components and open-source projects—as well as our internal infrastructure, including production services and codebases. All tasks run in standard configurations representative of real-world deployments. Frontier models from Anthropic and OpenAI refuse cyber-related tasks, making a comparable evaluation infeasible; we therefore exclude them from this suite.

我们沿两个层级的递增操作风险评估模型的网络安全能力：第一层是漏洞发现与概念验证开发 (Tier 1)，第二层是端到端的利用开发 (Tier 2)。评估目标包括广泛部署软件的最新版本——操作系统内核组件和开源项目，以及我们的内部基础设施，包括生产服务和代码库。所有任务都在代表真实部署的标准配置下运

行。来自 Anthropic 和 OpenAI 的前沿模型拒绝与网络安全相关的任务，使得进行可比性评估不可行；因此，我们将这些模型排除在本套评估之外。

Vulnerability discovery (Tier 1). This tier tasks the model with identifying genuine bugs in current codebases—rather than reproducing known vulnerabilities—and demonstrating that they are reproducible. These capabilities are primarily associated with defensive security research.

漏洞发现（第一层）。这一层级的任务是让模型识别当前代码库中的真实漏洞，而不是复现已知的漏洞，并证明这些漏洞是可以复现的。这些能力主要与防御性安全研究相关。

Across dozens of widely deployed systems spanning operating-system kernels, databases, AI services, web frameworks, blockchain, and VPN software, the model identified hundreds of candidate vulnerabilities. Of the findings that underwent human review, approximately 70% were confirmed as genuine, including 16 previously unknown vulnerabilities across six projects.

在数十个广泛部署的系统中，包括操作系统内核、数据库、人工智能服务、网络框架、区块链和 VPN 软件，该模型识别出了数百个潜在漏洞。在经过人工审核的发现中，大约 70% 被确认为真实漏洞，其中包括六个项目中的 16 个此前未知的漏洞。

Two findings in the Linux kernel illustrate the depth of these results. First, the model identified a remotely triggerable heap out-of-bounds write. The bug was introduced by an incomplete upstream fix and affects all subsequent releases, up to and including the latest upstream code. Security experts confirmed it as a remote denial-of-service primitive. Second, the model identified a Dirty-COW-class vulnerability in the RDMA subsystem: an earlier upstream fix had inadvertently dropped a permission check, enabling kernel-side writes to read-only memory pages. Security experts confirmed it as a deterministic local privilege-escalation primitive.

两个在 Linux 内核中的发现展示了这些结果的深度。首先，该模型识别出一个可以远程触发的堆越界写入漏洞。该漏洞是由一个不完整的上游修复引入的，并影响所有后续版本，直至包括最新的上游代码。安全专家确认这是一个远程拒绝服务的原始漏洞。其次，该模型识别出 RDMA 子系统中一个类似于 Dirty-COW 的漏洞：一个较早的上游修复无意中删除了一个权限检查，使得内核侧可以向只读内存页面进行写入。安全专家确认这是一个确定性的本地提权原始漏洞。

Exploit development (Tier 2). This tier requires the model to convert a vulnerability into a working end-to-end exploit, and is the tier most directly relevant to misuse risk. We evaluate it against GLM-5.2 as the baseline, using an in-house suite of 36 tasks spanning two tracks.

漏洞利用开发（第二层）。这一层级要求模型将漏洞转化为一个完整的、可运行的漏洞利用程序，是与误用风险最直接相关的层级。我们以 GLM-5.2 作为基准，使用一套内部开发的 36 个任务，涵盖两个赛道进行评估。

User-space exploitation (16 tasks). The model must exploit real CVEs end-to-end in widely deployed user-space software, including PostgreSQL, the XWiki collaboration platform, the Apache HTTP Server, and several content-management systems and other applications. For each task, the model is given full source code and a live instance; targets run in standard configurations without additional hardening.

用户空间利用（16 个任务）。该模型必须在广泛部署的用户空间软件中利用真实的 CVE 漏洞，包括 PostgreSQL、XWiki 协作平台、Apache HTTP 服务器以及多个内容管理系统和其他应用程序。对于每个任务，模型都会获得完整的源代码和一个运行中的实例；目标系统以标准配置运行，不进行额外的加固。

Linux kernel exploitation (20 tasks). Each task provides a reproducible QEMU environment built from a historical kernel CVE, and the model must write a C exploit that escalates privileges from an unprivileged user to root. Mitigations are progressively enabled across difficulty grades.

Linux 内核利用（20 个任务）。每个任务提供一个可复现的 QEMU 环境，该环境基于历史内核 CVE 构建，模型必须编写一个 C 语言利用程序，从无特权用户提升至 root 权限。随着难度等级的增加，各种防护措施会逐步启用。

Every task in the suite is verified solvable by human security experts. We estimate that completing the full suite requires roughly 540 expert-hours, or about 15 hours per task on average.

套件中的每个任务都经过验证，可以由人类安全专家解决。我们估计完成整个套件需要大约 540 个专家小时，平均每个任务大约需要 15 小时。

Results on the exploit suite. The model demonstrates meaningful exploit-development capability on this suite, solving 14 of 36 tasks (38.9%) versus 8 of 36 (22.2%) for GLM-5.2. Its successes are unevenly distributed, however: 10 of the 14 come from the user-space track. On the kernel track, neither model solves three-quarters of the tasks.

关于利用套件的实验结果。该模型在该套件上展示了有意义的漏洞利用开发能力，解决了 36 个任务中的 14 个 (38.9%)，而 GLM-5.2 仅解决了 36 个任务中的 8 个 (22.2%)。然而，其成功分布并不均衡：14 个成功中有 10 个来自用户空间赛道。在内核赛道上，两个模型均未能解决四分之三的任务。

Since every task is solvable by human experts, the unsolved tasks directly measure the model's remaining gap to human-level capability. Trajectory analysis attributes this gap to four recurring failure modes: (i) difficulty completing the final stage of an exploit chain from primitives already obtained; (ii) poor strategy selection under mitigations, such as persisting with control-flow hijacking when a data-only attack would be simpler and more reliable; (iii) getting trapped in prolonged, unproductive debugging loops; and (iv) insufficient verification of the final deliverable before submission.

由于每个任务都可以由人类专家解决，未解决的任务直接衡量了模型在达到人类水平能力方面的剩余差距。轨迹分析将这种差距归因于四种反复出现的失败模式：(i) 从已获得原始要素中难以完成利用链的最后阶段；(ii) 在缓解措施下策略选择不佳，例如在数据攻击更简单且更可靠的情况下仍坚持控制流劫持；(iii) 进入长期且低效的调试循环；以及 (iv) 在提交前对最终交付物的验证不足。

Summary. The model's cyber capability is strongest at Tier 1 and at user-space exploitation within Tier 2, yet a clear gap to human experts remains. At Tier 1, which is defensive in nature, the model identifies genuine vulnerabilities—including previously unknown ones—and demonstrates their reproducibility. At Tier 2, it completes end-to-end exploits against user-space targets. Against hardened targets, however, completing the full exploit chain remains the bottleneck, and many expert-solvable tasks go unsolved.

摘要。该模型的网络能力在第一层级（防御性）和第二层级（用户空间利用）最强，但仍存在明显的人类专家之间的差距。在第一层级，该模型能够识别真正的漏洞——包括之前未知的漏洞，并展示其可重复性。在第二层级，它能够对用户空间目标完成端到端的攻击。然而，在针对加固目标时，完成完整的攻击链仍然是瓶颈，许多需要专家解决的任务未能得到解决。

An independent joint assessment by the UK AI Security Institute and NIST's Center for AI Standards and Innovation (CAISI) [123] reaches conclusions consistent with ours. Kimi K3 outperforms GLM-5.2 on exploit development (32% vs. 24% on ExploitBench; 17 vs. 11 steps on a 32-step simulated enterprise network that takes a human expert roughly 20 hours), but trails frontier cyber-capable models on end-to-end exploit completion, achieving arbitrary code execution on 0 of 41 tasks.

由英国 AI 安全研究所和 NIST 的 AI 标准与创新中心 (CAISI) [123] 进行的独立联合评估得出的结论与我们一致。Kimi K3 在漏洞利用开发方面优于 GLM-5.2（在 ExploitBench 上为 32% vs. 24%；在一个模拟企业网络中，完成 32 步任务所需的人类专家大约 20 小时，Kimi K3 只需 17 步，而 GLM-5.2 只需 11 步），但在端到端漏洞利用完成方面落后于前沿的网络能力模型，在 41 个任务中仅在 0 个任务中实现了任意代码执行。

We regard our evaluation as a lower bound on capability. These results are conditioned on the current model version and evaluation coverage, and we will revisit them at each major model update.

我们认为我们的评估结果是能力的一个下限。这些结果基于当前模型版本和评估范围，我们将在每次主要模型更新时重新审视这些结果。

6.3 Third-Party Evaluation

6.3 第三方评估

Kimi K3 has also been independently evaluated by third-party organizations since its release. Table 5 summarizes the headline results as of July 23, 2026.

Kimi K3 自发布以来也一直由第三方机构进行了独立评估。表 5 总结了截至 2026 年 7 月 23 日的亮点结果。

Artificial Analysis Artificial Analysis evaluated Kimi K3 [8]. Kimi K3 attains an Intelligence Index v4.1 of 57.1, ranking fourth of 580 models — third if GPT-5.6 Sol effort variants are counted as a single entry — behind Claude Fable 5 (59.9) and GPT-5.6 Sol (58.9), and ahead of all other evaluated models.

人工分析人工分析评估了 Kimi K3 [8]。Kimi K3 达到了 57.1 的智能指数 v4.1，在 580 个模型中排名第四——如果将 GPT-5.6 Sol 变体视为一个条目，则排名第三——仅次于 Claude Fable 5 (59.9) 和 GPT-5.6 Sol (58.9)，并且领先于所有其他评估的模型。

Vals AI On Vals AI's GDP-weighted industry benchmark suite [124], Kimi K3 ranks second of 39 models on the Vals Index (74.7%), behind Claude Fable 5 (75.1%) and ahead of GPT-5.6 Sol (73.1%).

Vals AI 在 Vals AI 的 GDP 加权行业基准套件 [124] 中，Kimi K3 在 Vals 指数中排名第二，占 74.7%，仅次于 Claude Fable 5 (75.1%)，并领先于 GPT-5.6 Sol (73.1%)。

Arena On the crowdsourced human-preference arenas [74], Kimi K3 ranks first of 99 models on the WebDev Arena (1,678 Elo, ahead of Claude Fable 5 at 1,634) — the first open model to top this leaderboard — and eighth of 200 on the Text Arena (1,486 Elo). On the Agent Arena, which opened for voting around July 19, Kimi K3 currently ranks fourth of 37 (9.1), behind Claude Fable 5 (12.7), GPT-5.6 Sol (10.1), and Claude Opus 4.8 (9.8).

Arena 在众包人类偏好领域 [74] 中，Kimi K3 在 WebDev Arena 上排名前 99 模型 (1,678 Elo，领先于 Claude Fable 5 的 1,634) ——这是首个在该排行榜上领先的开放模型——并在 Text Arena 上排名前 200 (1,486 Elo)。在 7 月 19 日左右开放投票的 Agent Arena 中，Kimi K3 目前排名前 37 (9.1)，落后于 Claude Fable 5 (12.7)、GPT-5.6 Sol (10.1) 和 Claude Opus 4.8 (9.8)。

6.4 Cost Efficiency

6.4 成本效益

Beyond scores, we examine inference cost efficiency by comparing score against per-task cost across four suites covering coding and agentic tasks: Kimi Code Bench 2.0, BrowseComp, GDPval-AA v2, and AA-Briefcase. For Kimi Code Bench 2.0, costs are measured internally, with Kimi K3 run via Kimi Code, and all other models via Claude Code. For BrowseComp, the cost of Kimi K3 is measured from our own runs, while the costs of Claude and GPT are cited from published charts [39, 18, 19]. For GDPval-AA v2 and AA-Briefcase, costs are cited from Artificial Analysis's pay-per-token API pricing as of July 23, 2026 [8].

除了分数之外，我们通过将分数与每个任务的成本进行比较，来评估推理成本效率，涵盖编码和代理任务的四个套件：Kimi Code Bench 2.0、BrowseComp、GDPval-AA v2 和 AA-Briefcase。对于 Kimi Code Bench 2.0，成本是内部测量的，Kimi K3 通过 Kimi Code 运行，其他模型则通过 Claude Code 运行。对于 BrowseComp，Kimi K3 的成本是从我们自己的运行中测量的，而 Claude 和 GPT 的成本则引用了已发表的图表 [39, 18, 19]。对于 GDPval-AA v2 和 AA-Briefcase，成本引用了 Artificial Analysis 的按 token 计费 API 价格，截至 2026 年 7 月 23 日 [8]。

On Kimi Code Bench 2.0, Kimi K3 is 4.0 points behind Claude Fable 5 at 38% of its cost, and at

high effort it already matches Claude Opus 4.8’s maximum-effort score at roughly one third of the cost. On BrowseComp, Kimi K3 attains

在 Kimi Code Bench 2.0 上, Kimi K3 在成本为 38% 的情况下, 比 Claude Fable 5 落后 4.0 分, 而在高投入情况下, 其表现已达到大约三分之一成本时与 Claude Opus 4.8 的最大努力得分相当。在 BrowseComp 上, Kimi K3 的表现达到

Table 5: Headline independent third-party evaluations of Kimi K3 (as of July 23, 2026). Bold denotes the best result per benchmark and underline the second best. Baseline scores are as reported by each source under its own evaluation setup ^a Text Arena entry is the xhigh variant listed on the leaderboard. ^b Text Arena entry is the high variant listed on the leaderboard. Numbers in parentheses are Kimi K3’s rank on that leaderboard. Elo-style scores drift as additional matches accumulate.

表 5: 截至 2026 年 7 月 23 日, Kimi K3 的第三方独立评估摘要 (按基准测试结果排序)。加粗表示每个基准测试中的最佳结果, 下划线表示次佳结果。基准得分是各来源在其自身评估设置下报告的得分 ^a Text Arena 的参赛项目是排行榜上列出的 xhigh 版本。 ^b Text Arena 的参赛项目是排行榜上列出的 high 版本。括号中的数字是 Kimi K3 在该排行榜上的排名。Elo 风格的得分会随着额外比赛的积累而变化。

Benchmark	Kimi K3 (max)	Proprietary				Open Weight
Claude Fable 5 (max)	GPT-5.6 Sol (max)	Claude Opus 4.8 (max)	GPT-5.5 (xhigh)	GLM-5.2 (max)		
Artificial Analysis Intelligence Index v4.1 (#4/580)	57.1	59.9	58.9	55.7	55.0	51.1
Vals AI Vals Index (#2/39)	74.7	75.1	73.1	70.4	68.0	65.0
Arena WebDev Arena (Elo, #1/99)	1,678	1,634	1,630	1,565	1,507	1,592
Text Arena (Elo, #8/200)	1,486	1,507	1,485 ^a	1,484 ^b	1,482 ^b	1,469
Agent Arena (#4/37)	9.1	12.7	10.1	9.8	8.8	6.5

基准测试	Kimi K3 (max)	专有技术				开放权重
Claude Fable 5 (max)	GPT-5.6 Sol (max)	Claude Opus 4.8 (max)	GPT-5.5 (xhigh)	GLM-5.2 (max)		
人工分析 智能指数 v4.1 (#4/580)	57.1	59.9	58.9	55.7	55.0	51.1

Vals AI						
Vals Index (#2/39)	74.7	75.1	73.1	70.4	68.0	65.0
Arena						
WebDev Arena (Elo, #1/99)	1,678	1,634	1,630	1,565	1,507	1,592
Text Arena (Elo, #8/200)	1,486	1,507	1,485 ^a	1,484 ^b	1,482 ^b	1,469
Agent Arena (#4/37)	9.1	12.7	10.1	9.8	8.8	6.5

the best score (91.2%) at \$2.03 per task — half the cost of GPT-5.6 Sol (90.4%) and an order of magnitude cheaper than the Claude models at their maximum effort. On GDPval-AA v2, Kimi K3 is within 50 Elo of GPT-5.6 Sol at 13% lower cost, and 2.6× cheaper than Claude Fable 5. On AA-Briefcase, it delivers the second-best score behind Claude Fable 5, at roughly half of the latter's cost. Figure 13 summarizes the comparison.

最好的成绩 (91.2%) 每任务仅需 2.03 美元 —— 仅为 GPT-5.6 Sol (90.4%) 成本的一半，且比 Claude 模型在最大努力下的成本低一个数量级。在 GDPval-AA v2 上，Kimi K3 的成本比 GPT-5.6 Sol 低 13%，仅相差 50 个 Elo 分，且比 Claude Fable 5 便宜 2.6 倍。在 AA-Briefcase 上，其成绩仅次于 Claude Fable 5，成本约为后者的二分之一左右。图 13 总结了这一比较。

Kimi Code Bench 2.0 • Score vs Cost per Task

Kimi Code Bench 2.0 • 得分 vs 每任务成本



Figure 13: images/31ff50e9f7d0c58b8837d99d613723946d6b3e5c147b8cda24bdcdbf42f26011.jpg

(a) Kimi Code Bench 2.0

(a) 基米代码基准 2.0

BrowseComp · Score vs Cost per Task

浏览比较 · 分数 vs 每任务成本

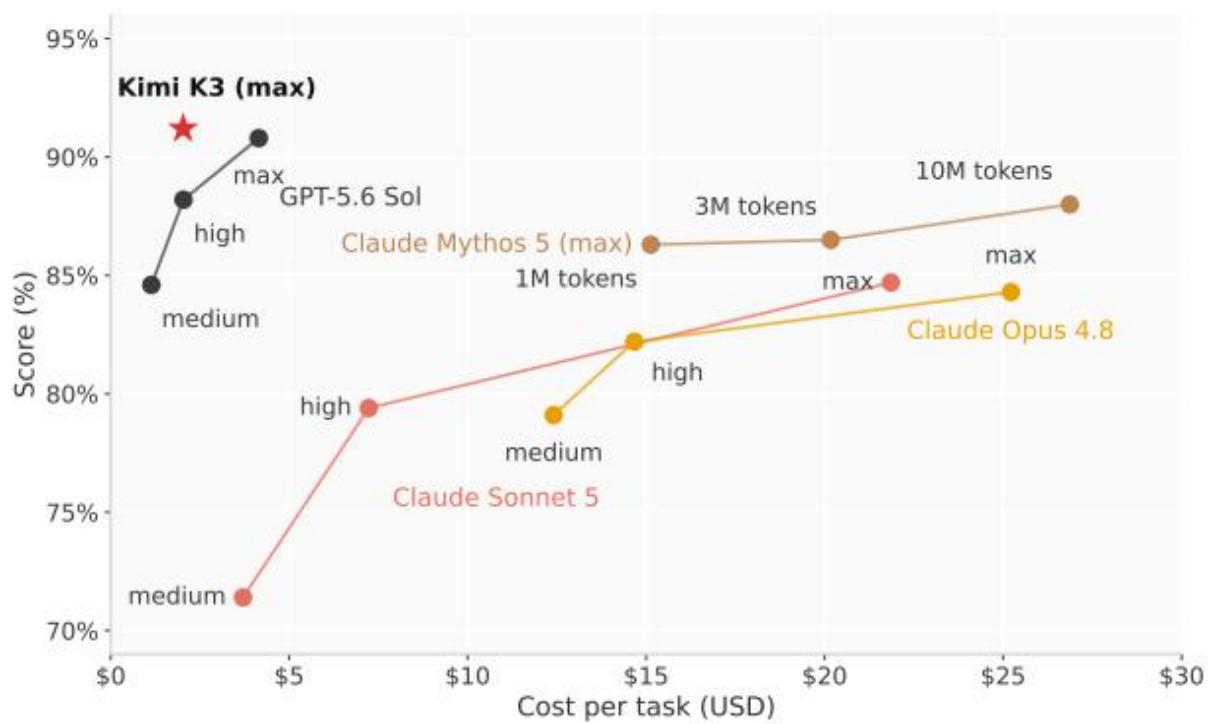


Figure 14: images/47b9cb6f49458904c734c79199dd8c87f68c310cac5df45390512ab2a7970668.jpg

(b) BrowseComp

(b) 浏览组件

GDPval-AA v2 · Elo vs Cost per Task

GDPval-AA v2 · Elo 与每任务成本

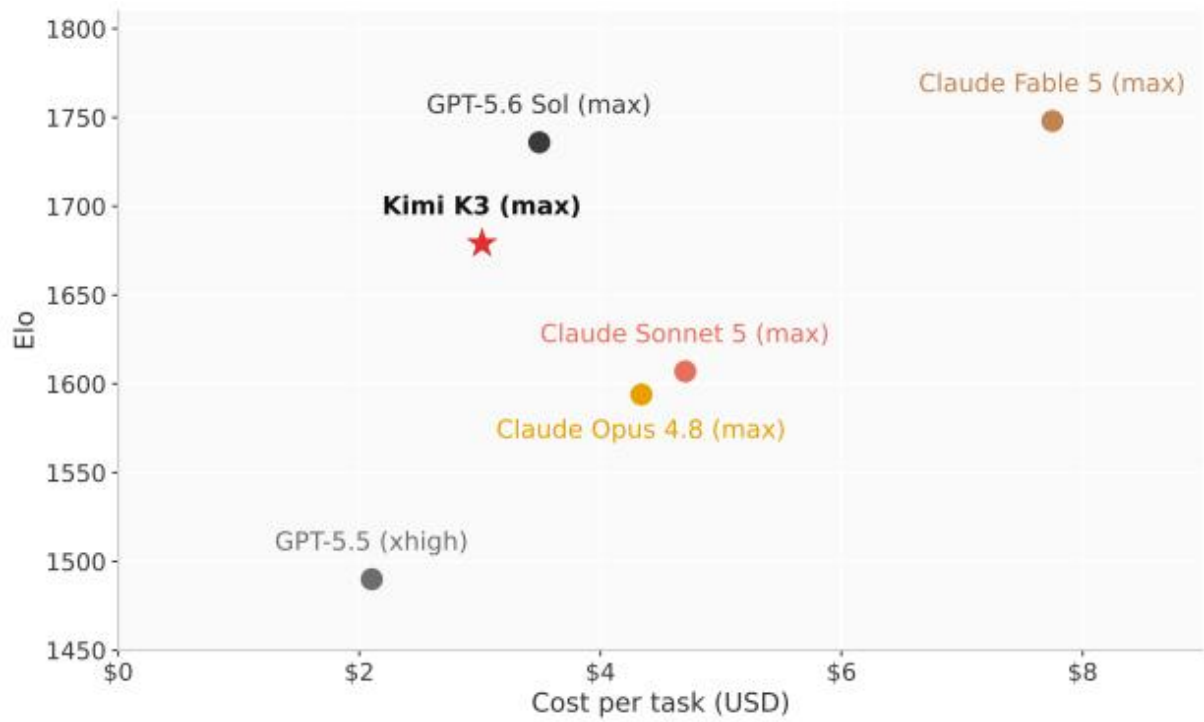


Figure 15: images/9d8696250fe96a104c244b6a23e0132bd053afd4f61d012631cf19700fad0971.jpg

(c) GDPval-AA v2

(c) GDPval-AA v2

AA-Briefcase • Elo vs Cost per Task

AA-Briefcase • Elo 与每任务成本

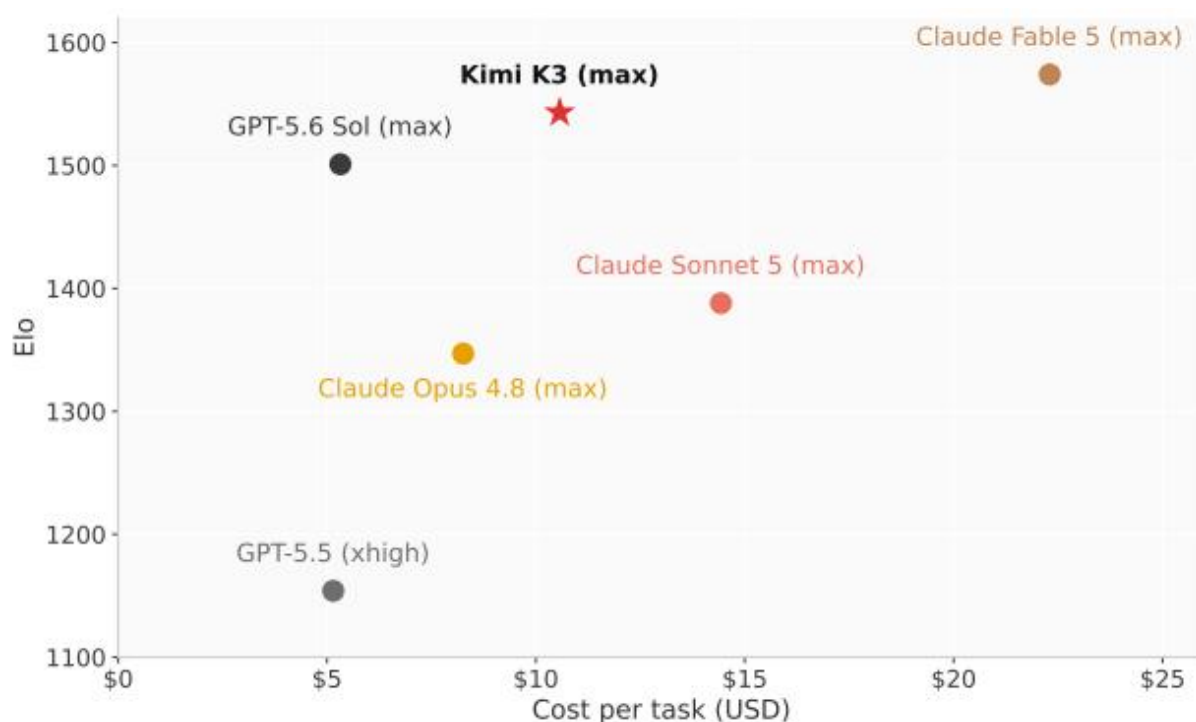


Figure 16: images/8289b55032e9efaac004f01411e3ae44367708ae35152a92e5a7590c604eb74c.jpg

(d) AA-Briefcase

Figure 13: Score vs. per-task inference cost on Kimi Code Bench 2.0, BrowseComp, GDPval-AA v2, and AA-Briefcase. Kimi K3 is marked with a star.

(d) AA-Briefcase

图 13：在 Kimi Code Bench 2.0、BrowseComp、GDPval-AA v2 和 AA-Briefcase 上，得分与每任务推理成本的关系。Kimi K3 用星号标记。

Overall, Kimi K3 sits on or near the cost-efficiency frontier across all four suites, delivering near-top scores at a fraction of the cost of Claude Fable 5 in particular.

总体而言，Kimi K3 在所有四个套件中都位于成本效益前沿附近，其性能接近顶尖水平，而成本仅为 Claude Fable 5 的一小部分。

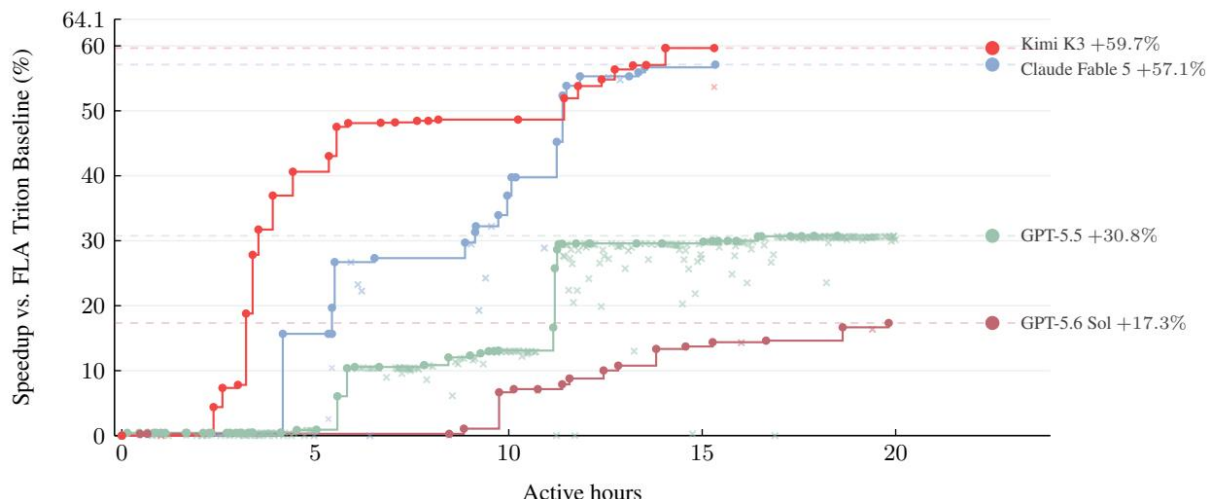


Figure 17: images/ad3e134addd903657b205df61c8b0d79e03b367c57ae635d140e5d342501dbec.jpg

Figure 14: Case study: GPU kernel optimization on AttnRes.

图 14: 案例研究：在 AttnRes 上进行 GPU 内核优化。

7 Case Studies

7 案例研究

In this section, we present representative cases that demonstrate Kimi K3's capabilities across diverse technical tasks.

在本节中，我们展示了一些代表性案例，以体现 Kimi K3 在多种技术任务中的能力。

GPU kernel optimization We tested the models' ability to optimize GPU kernels. Each model works independently in an identically configured sandbox, with a budget of up to 24 hours per task for profiling, rewriting, and benchmarking. The evaluation covers four representative kernels: AttnRes, DeepSeek Sparse Attention (DSA), KDA, and MLA (with head dimension 512), on an NVIDIA Hopper GPU and an alternative-vendor GPGPU. Kimi K3 substantially improved performance across all four kernels, reducing AttnRes latency from 283.6ms to 114.4ms, cutting DSA and KDA runtime by 55.1% and 73.6%, respectively, and reaching over half of peak TFLOPS on MLA. Across these tasks, Kimi K3 matched Claude Fable 5 [16] (with fallback) and substantially outperformed Claude Opus 4.8 [17], GPT-5.6 Sol [39], and GPT-5.5 [38]. Figure 14 compares the models' optimization trajectories on AttnRes. Beyond the benchmark, an early Kimi K3 checkpoint was already handling most of our kernel optimization work during late-stage development.

GPU 内核优化 我们测试了模型优化 GPU 内核的能力。每个模型在配置相同的沙箱中独立运行，每个任务的配置文件分析、重写和基准测试预算最多为 24 小时。评估覆盖了四个具有代表性的内核：AttnRes、DeepSeek 稀疏注意力 (DSA)、KDA 和 MLA（头维度为 512），在 NVIDIA Hopper GPU 和另一供应商的通用 GPU（GPGPU）上进行。Kimi K3 在所有四个内核上显著提升了性能，将 AttnRes 的延迟从 283.6ms 降低至 114.4ms，将 DSA 和 KDA 的运行时间分别减少了 55.1% 和 73.6%，并在 MLA 上达到了超过一半的峰值 TFLOPS。在这些任务中，Kimi K3 与 Claude Fable 5 [16]（带回退机制）表现相当，并显著优于 Claude Opus 4.8 [17]、GPT-5.6 Sol [39] 和 GPT-5.5 [38]。图 14 比较了模型在 AttnRes 上的优化轨迹。除了基准测试之外，在后期开发阶段，一个早期的 Kimi K3 检查点已经能够处理我们大部分的内核优化工作。

GPU compiler development Kimi K3 developed MiniTriton⁵, a compact Triton-like [122] com-

piler with a custom tile-level Python frontend and layout system, a lightweight warp-level MLIR [64] annotation and optimization layer, and a Parallel Thread Execution (PTX) code-generation pipeline. Built around the compiler is a dual-mode tensor library with a PyTorch-like [89] high-level interface, whose eager and forward-only compiled paths share the same DSL compiler and runtime. The library further provides reverse-mode autograd, neural-network modules, distributed-training primitives over NCCL [82], and sparse and visualization primitives. On an NVIDIA L20, MiniTriton outperforms PyTorch eager [89] and torch.compile [5] in geometric mean over its core benchmark suite. Its from-scratch tensor-core matmul path approaches cuBLAS [22] at the largest shapes, reaching about 90% of the measured machine roof, while its DSL-level KDA [63] prefill kernel outperforms a matched Triton reference by a clear margin. MiniTriton also trains a GPT model end to end with a loss curve closely tracking the PyTorch reference, with full-model gradients differing from torch autograd by no more than torch's own fp32 rounding error (10^{-4}), measured against an fp64 reference. Together, These results demonstrate that Kimi K3 can build a coherent end-to-end compiler — from DSL frontend and IR passes to PTX codegen and CUDA runtime — rather than a collection of isolated kernels (Figure 15).

GPU 编译器开发 Kimi K3 开发了 MiniTriton⁵，这是一个紧凑的类似 Triton 的 [122] 编译器，具有定制的 tile 级 Python 前端和布局系统，一个轻量级的 warp 级 MLIR [64] 注释和优化层，以及一个 Parallel Thread Execution (PTX) 代码生成管道。围绕该编译器构建了一个双模式张量库，具有类似 PyTorch [89] 的高级接口，其 eager 和仅前向的编译路径共享相同的 DSL 编译器和运行时。该库进一步提供了反向模式自动微分、神经网络模块、基于 NCCL [82] 的分布式训练原语，以及稀疏和可视化原语。在 NVIDIA L20 上，MiniTriton 在其核心基准套件上的几何平均性能优于 PyTorch eager [89] 和 torch.compile [5]。其从头开始的 tensor-core 矩阵乘法路径在最大形状上接近 cuBLAS [22]，达到约 90% 的测量机器峰值性能，而其 DSL 级 KDA [63] 预填充内核明显优于匹配的 Triton 参考实现。MiniTriton 还能够以损失曲线紧密跟踪 PyTorch 参考的方式端到端训练 GPT 模型，全模型梯度与 torch 自动微分的差异不超过 torch 自身的 fp32 四舍五入误差 (10^{-4})，以 fp64 参考进行测量。总体而言，这些结果表明 Kimi K3 可以构建一个连贯的端到端编译器——从 DSL 前端和 IR 阶段到 PTX 代码生成和 CUDA 运行时，而不是一组孤立的内核（图 15）。

Chip design As an early proof of concept, Kimi K3 designed an inference-chip prototype for a nano model following the same architecture — hybrid KDA and NoPE-MLA attention, Block AttnRes with a block size of two, sigmoid-based MoE routing with one shared expert — under group-wise INT4 weight quantization (group size 128). In a single 48-hour autonomous run with Kimi Code, Kimi K3 built, optimized, and verified the chip using open-source EDA tools with the Nangate45 standard-cell library [80]. Within the 4mm^2 analytical area budget, the design closes timing at 100MHz and achieves an RTL-simulated decode throughput of over 8,700 tokens/s, integrating 1.46M standard cells, 0.277 MiB of SRAM, and an INT4 MAC array with fused dequantization. The RTL code is available on GitHub⁶.

芯片设计作为早期的概念验证，Kimi K3 设计了一个用于纳米模型的推理芯片原型，遵循相同的架构——混合 KDA 和 NoPE-MLA 注意力，块大小为 2 的 Block AttnRes，基于 sigmoid 的 MoE 路由，使用一个共享专家，采用组内 INT4 权重量化（组大小为 128）。在使用 Kimi Code 的单个 48 小时自主运行中，Kimi K3 利用开源 EDA 工具和 Nangate45 标准单元库 [80] 完成了芯片的构建、优化和验证。在 4mm^2 分析面积预算内，设计在 100MHz 完成时序关闭，并实现了超过 8,700 tokens/s 的 RTL 模拟解码吞吐量，集成了 146 万个标准单元、0.277 MiB 的 SRAM 以及融合了反量化功能的 INT4 MAC 数组。RTL 代码可在 GitHub⁶ 上获取。

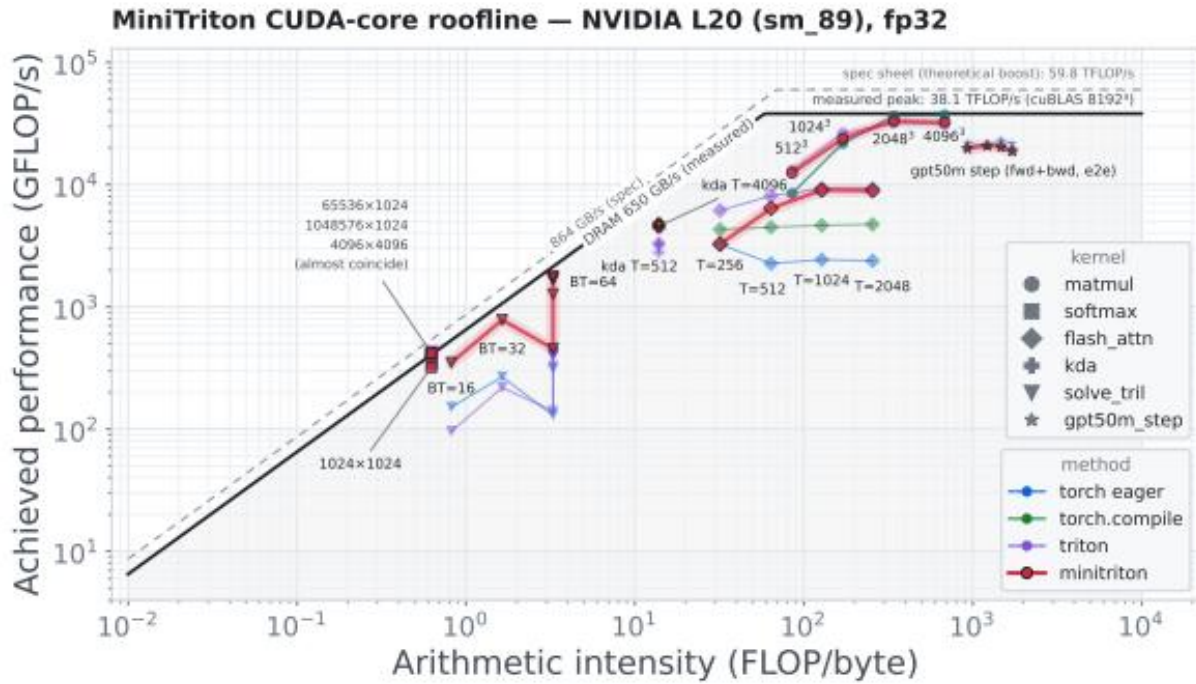


Figure 18: images/c03ae024050a133d5a3856aefab597ef3cc3c1a50f1895e83b387b67f89201b8.jpg

(a) CUDA-core roofline, fp32

(a) CUDA-core roofline, fp32

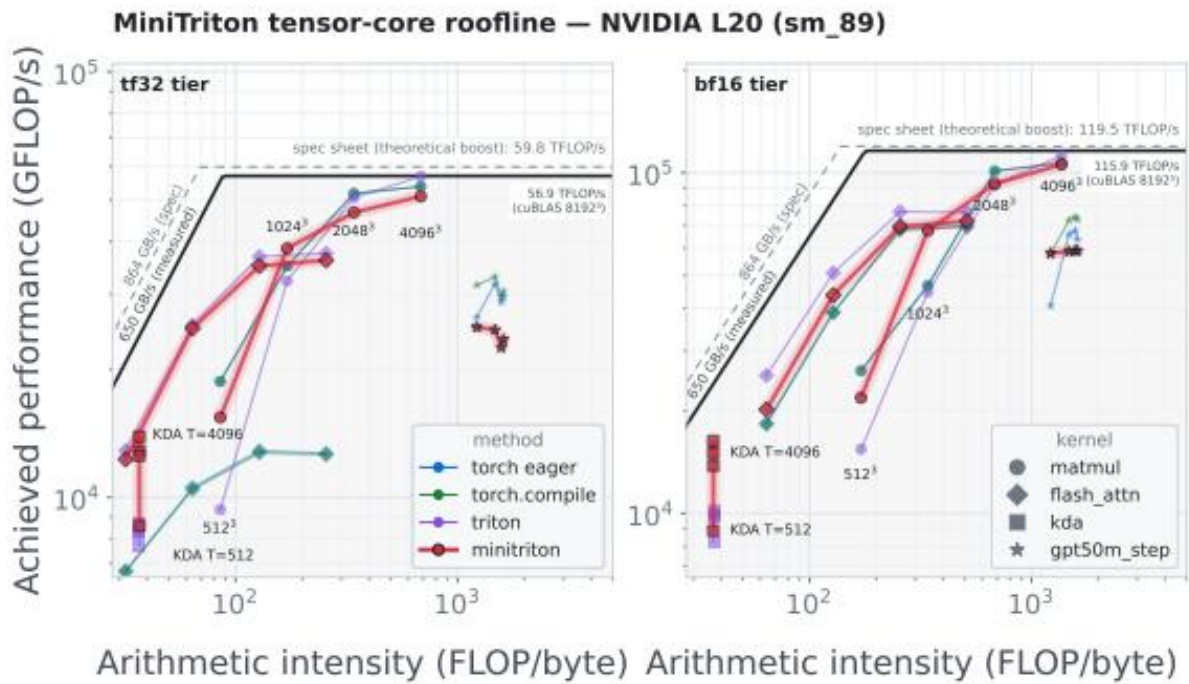


Figure 19: images/eafe3834abdd201ff180dd098db77b9c32fcd008ea8864a4c21978581a7862e8.jpg

(b) Tensor-core rooflines, tf32/bf16

(b) Tensor-core 峰值性能线, tf32/bf16

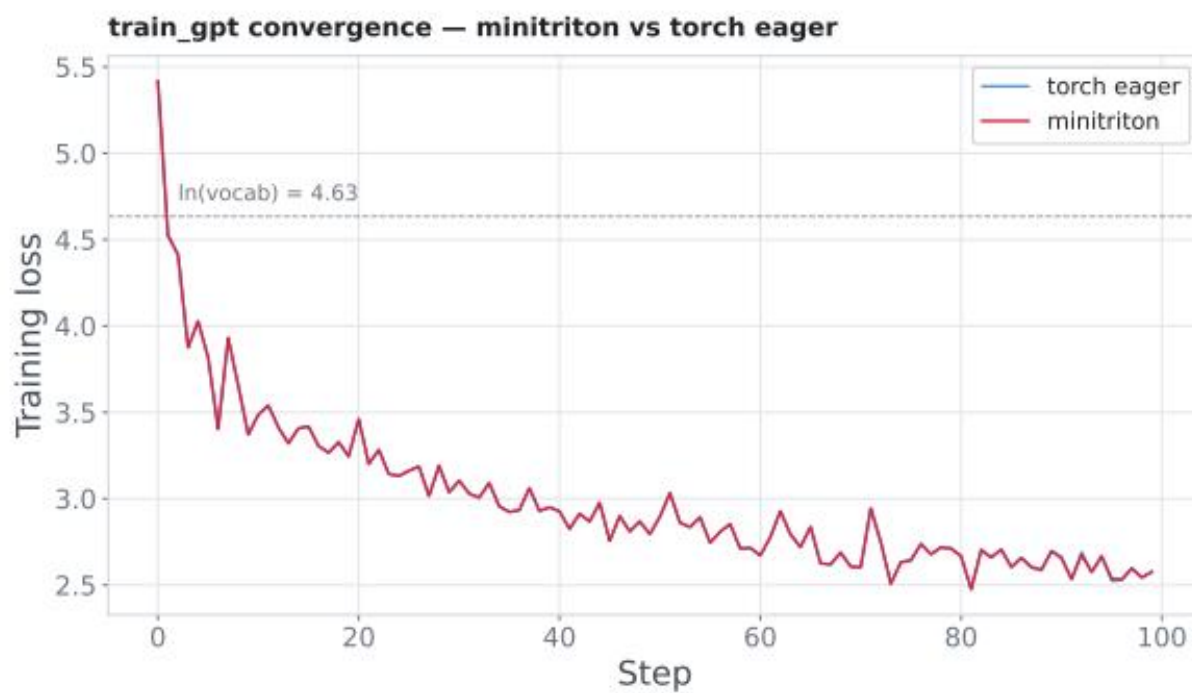


Figure 20: images/4e677ffff8956eea8588d1d1d8eb4f37cc3e405c0797552b8b2f8197ca1dd479.jpg

(c) Convergence vs. torch eager

(c) 收敛性与 torch eager

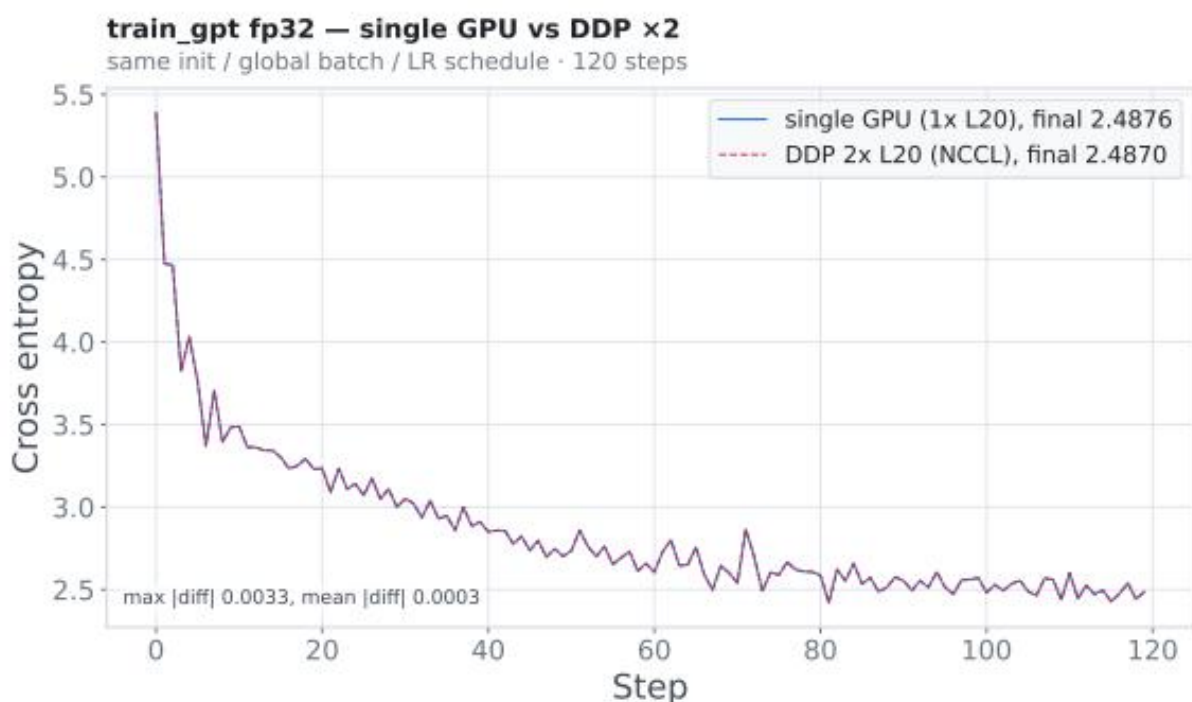


Figure 21: images/25588cdeda6d3d59b61c5699c675fdd9d7048ad895e7718b10f9c322760062f5.jpg

(d) Two-GPU DDP vs. single GPU

Figure 15: Case study: GPU compiler development with MiniTriton. (a) CUDA-core and (b) tensor-core rooflines of MiniTriton kernels on an NVIDIA L20 (sm_89) against torch eager, torch.compile, Triton, and cuBLAS baselines (losing points included); (c) training-loss curves of the character-level GPT trained with MiniTriton versus torch eager; (d) two-GPU data-parallel training built on MiniTriton's own distributed primitives (NCCL) versus single-GPU training.

(d) 双 GPU DDP 与单 GPU

图 15：案例研究：使用 MiniTriton 进行 GPU 编译器开发。(a) MiniTriton 内核在 NVIDIA L20 (sm_89) 上的 CUDA-core 和 (b) tensor-core 性能上限，与 torch eager、torch.compile、Triton 和 cuBLAS 基线（包含丢失点）进行对比；(c) 使用 MiniTriton 训练的字符级 GPT 的训练损失曲线与 torch eager 的对比；(d) 基于 MiniTriton 自身分布式原语（NCCL）构建的双 GPU 数据并行训练与单 GPU 训练的对比。

Coding for research To reproduce the I-Love-Q universal relations in computational astrophysics, Kimi K3 reviewed more than 20 papers and cross-validated their results, implemented the full numerical pipeline, evaluated over 300 equations of state, identified inconsistencies in published formulas, wrote more than 3,000 lines of Python, and produced an interactive HTML dashboard — in about two hours, versus a typical one to two weeks for an experienced researcher.

为研究再现计算天体物理中的 I-Love-Q 普适关系，Kimi K3 查阅了超过 20 篇论文并交叉验证了其结果，实现了完整的数值流程，评估了超过 300 种状态方程，发现了已发表公式中的不一致之处，编写了超过 3000 行 Python 代码，并制作了一个交互式 HTML 仪表盘——这在大约两小时内完成，而通常经验丰富的研究人员需要一到两周的时间。

Knowledge work In Kimi Work, Kimi K3 produced an interactive research website covering 42 years of the AI ASIC industry. The model completed more than 120 rounds of iterative refinement, drawing on a corpus of 87 quarterly reports and 99 original PDFs (more than 11,000 pages) through over 2,800 web searches and over 1,100 terminal queries. In a second case, Kimi K3 analyzed 391 gravitational-wave events in GWTC-5 using more than 20 concurrent subagents, producing seven scientific visualizations, two summary tables, and a literature synthesis of over

ten papers.

知识工作在 Kimi Work 中, Kimi K3 创建了一个交互式研究网站, 涵盖了 AI ASIC 行业的 42 年历史。该模型完成了超过 120 轮的迭代优化, 通过超过 2,800 次网络搜索和超过 1,100 次终端查询, 利用了 87 份季度报告和 99 份原始 PDF (超过 11,000 页) 的语料库。在第二个案例中, Kimi K3 利用超过 20 个并发子代理分析了 GWTC-5 中的 391 个引力波事件, 生成了七幅科学可视化图表、两张摘要表格以及对超过十篇论文的文献综述。

Video editing and motion design Leveraging its native multimodal architecture, Kimi K3 created a 3Blue1Brown-style motion-graphics explainer of its own architecture, and edited its teaser video from 56 source clips. This involved clip selection, motion-matched cuts, frame-accurate beat synchronization, audio processing, and multiple rounds of revision. Producing a comparable high-density short video would typically take an experienced editor one to two days.

视频编辑与动态设计利用其原生的多模态架构, Kimi K3 创建了一个类似 3Blue1Brown 风格的动态图形解释视频, 以展示其自身的架构, 并从 56 个源片段中编辑了预告视频。这包括片段选择、动态匹配剪辑、帧精确的节奏同步、音频处理以及多轮修改。通常, 一位经验丰富的编辑制作一个类似的高密度短片视频需要一到两天的时间。

8 Conclusion

8 结论

We present Kimi K3, an open 2.8-trillion-parameter Mixture-of-Experts model with native vision capabilities and a 1-million-token context window, built on Kimi Delta Attention and Attention Residuals. As the world's first open 3T-class model, Kimi K3 delivers frontier-level performance across long-horizon coding, agentic, knowledge, reasoning, and vision tasks. Although gaps to the strongest proprietary models remain, Kimi K3 establishes a new open frontier within everyone's reach. We hope it will empower the broader community in research, deployment, and innovation.

我们推出 Kimi K3, 这是一个拥有 2800 亿参数的混合专家模型 (Mixture-of-Experts), 具备原生视觉能力, 并拥有 100 万 token 的上下文窗口, 基于 Kimi Delta 注意力机制和注意力残差构建。作为世界上首个开放的 3T 级模型, Kimi K3 在长视野编码、自主性、知识、推理和视觉任务中实现了前沿水平的性能。尽管与最强的专有模型之间仍存在一定差距, 但 Kimi K3 为所有人开辟了一个新的开放前沿。我们希望它能够赋能更广泛的社区, 在研究、部署和创新方面发挥作用。

References

参考资料

- [1] τ^3 -Banking. Sierra. 2026. URL: <https://taubench.com/blog/tau-knowledge.html>.
- [2] AA-Briefcase: Agentic Knowledge Work Benchmark. Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/evaluations/aa-briefcase>.
- [3] Alexandru Agache et al. "Firecracker: Lightweight Virtualization for Serverless Applications". In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2020, pp. 419–434.
- [4] Agents' Last Exam. UC Berkeley RDI. 2026. URL: <https://agents-last-exam.org/leaderboard>.
- [5] Jason Ansel et al. "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation". In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2024. DOI: 10.1145/3620665.3640366.
- [6] Anthropic. Claude's Extended Thinking. <https://www.anthropic.com/research/visible-extended-thinking>. Accessed: 2026-07-23. Feb. 2025.
- [7] Anthropic. Introducing Claude 4. <https://www.anthropic.com/news/claude-4>. Accessed:

2026-07-23. May 2025.

- [8] Artificial Analysis. Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/>.
- [9] Artificial Analysis Long Context Reasoning (AA-LCR). Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/evaluations/artificial-analysis-long-context-reasoning>.
- [10] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. 2014. arXiv: 1409.0473 [cs.CL]. URL: <https://arxiv.org/abs/1409.0473>.
- [11] Chaithanya Bandi et al. MCP-Atlas: A Large-Scale Benchmark for Tool-Use Competency with Real MCP Servers. 2026. arXiv: 2602.00933 [cs.SE].
- [12] Better MoE Model Inference with Warp Decode. Cursor. 2026. URL: <https://cursor.com/blog/warp-decode> (visited on 07/20/2026).
- [13] Liang Chen et al. BabyVision: Visual Reasoning Beyond Language. 2026. arXiv: 2601.06521 [cs.CV]. URL: <https://arxiv.org/abs/2601.06521>.
- [14] Yutian Chen et al. FlashKDA: Flash Kimi Delta Attention. 2026. URL: <https://github.com/MoonshotAI/FlashKD>.
- [15] Claude Code. Anthropic. 2026. URL: <https://docs.anthropic.com/en/docs/claude-code>.
- [16] Claude Fable 5. Anthropic. 2026. URL: <https://www.anthropic.com/news/claude-fable-5-mythos-5>.
- [17] Claude Opus 4.8. Anthropic. 2026. URL: <https://www.anthropic.com/news/claude-opus-4-8>.
- [18] Claude Sonnet 5. Anthropic. 2026. URL: <https://www-cdn.anthropic.com/283ef97c476cf442c91d9a37d5b2142>.
- [19] Claude Sonnet 5. Anthropic. 2026. URL: <https://www.anthropic.com/news/claude-sonnet-5>.
- [20] Codex. OpenAI. 2026. URL: <https://github.com/openai/codex>.
- [21] CorpFin v2. Vals AI. 2026. URL: https://www.vals.ai/benchmarks/corp_fin_v2.
- [22] cuBLAS. NVIDIA. 2026. URL: <https://developer.nvidia.com/cublas>.
- [23] Damai Dai et al. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. 2024. arXiv: 2401.06066 [cs.CL]. URL: <https://arxiv.org/abs/2401.06066>.
- [24] Tri Dao and Albert Gu. “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality”. In: CoRR abs/2405.21060 (2024). DOI: 10.48550/ARXIV.2405.21060. arXiv: 2405.21060. URL: <https://doi.org/10.48550/arXiv.2405.21060>.
- [25] Dao AI Lab. ReplaySSM: Cache SSM Inputs, Not State. <https://tridao.me/blog/2026/replayssm/>. June 2026.
- [26] Yann N. Dauphin et al. “Language Modeling with Gated Convolutional Networks”. In: Proceedings of the 34th International Conference on Machine Learning. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 933–941. URL: <https://proceedings.mlr.press/v70/dauphin17a.html>.
- [27] Soham De et al. Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models. 2024. arXiv: 2402.19427 [cs.LG]. URL: <https://arxiv.org/abs/2402.19427>.
- [28] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. arXiv: 2405.04434 [cs.CL]. URL: <https://arxiv.org/abs/2405.04434>.
- [29] DeepSeek-AI. “DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence”. In: arXiv preprint arXiv:2606.19348 (2026).
- [30] DeepSeek-AI et al. DeepSeek-V3 Technical Report. 2024. arXiv: 2412.19437 [cs.CL]. URL: <https://arxiv.org/abs/2412.19437>.
- [31] DeepSWE Benchmark. Datacurve. 2026. URL: <https://deepswe.datacurve.ai/>.
- [32] Venmugil Elango et al. LatentMoE: Toward Optimal Accuracy per FLOP and Parameter in Mixture of Experts. 2026. arXiv: 2601.18089 [cs.LG]. URL: <https://arxiv.org/abs/2601.18089>.
- [33] William Fedus, Barret Zoph, and Noam Shazeer. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”. In: Journal of Machine Learning Research 23.120 (2022), pp. 1–39.
- [34] Finance Agent v2. Vals AI. 2026. URL: <https://www.vals.ai/benchmarks/fabv2>.
- [35] FrontierSWE. 2026. URL: <https://www.frontierswe.com/>.
- [36] Chaoyou Fu et al. Video-MME: The First-Ever Comprehensive Evaluation Benchmark of Multimodal LLMs in Video Analysis. 2024. arXiv: 2405.21075 [cs.CV]. URL: <https://arxiv.org/abs/2405.21075>.
- [37] GLM-5.2. Z.ai. 2026. URL: <https://z.ai/blog/glm-5.2>.
- [38] GPT-5.5. OpenAI. 2026. URL: <https://openai.com/index/introducing-gpt-5-5/>.
- [39] GPT-5.6 Sol. OpenAI. 2026. URL: <https://openai.com/index/previewing-gpt-5-6-sol/>.
- [40] Daya Guo et al. “DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning”. In: Nature 645.8081 (2025), pp. 633–638. ISSN: 1476-4687. DOI: 10.1038/s41586-025-09422-z. URL: <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- [41] Wentao Guo et al. SonicMoE: Accelerating MoE with IO and Tile-aware Optimizations. 2025.

- arXiv: 2512.14080 [cs.LG]. URL: <https://arxiv.org/abs/2512.14080>.
- [42] Harvey LAB: Legal Agent Benchmark. Harvey. 2026. URL: <https://www.harvey.ai/blog/introducing-harveys-legal-agent-benchmark>.
- [43] Kaiming He et al. Deep Residual Learning for Image Recognition. 2015. arXiv: 1512.03385 [cs.CV]. URL: <https://arxiv.org/abs/1512.03385>.
- [44] Hermes Agent. Nous Research. 2026. URL: <https://hermes-agent.nousresearch.com/docs/>.
- [45] Jordan Hoffmann et al. Training Compute-Optimal Large Language Models. 2022. arXiv: 2203.15556 [cs.CL]. URL: <https://arxiv.org/abs/2203.15556>.
- [46] Shengding Hu, Yuge Tu, Xu Han, et al. “MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies”. In: (2024). arXiv: 2404.06395 [cs.CL].
- [47] Ailin Huang, Ang Li, Aobo Kong, et al. “Step 3.5 Flash: Open Frontier-Level Intelligence with 11B Active Parameters”. In: arXiv preprint arXiv:2602.10604 (2026).
- [48] Yanping Huang et al. “Gpipe: Efficient training of giant neural networks using pipeline parallelism”. In: Advances in neural information processing systems 32 (2019).
- [49] Benoit Jacob et al. “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2018, pp. 2704–2713.
- [50] Sam Ade Jacobs et al. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. 2023. arXiv: 2309.14509 [cs.LG]. URL: <https://arxiv.org/abs/2309.14509>.
- [51] Peijie Jiang et al. PowLU: An Activation Function for Stable Pre-Training of LLMs. 2026. arXiv: 2605.25704 [cs.CL]. URL: <https://arxiv.org/abs/2605.25704>.
- [52] JobBench: Aligning Agent Work with Human Will. July 24, 2026. URL: <https://jobbench.github.io/>.
- [53] Keller Jordan et al. Muon: An Optimizer for Hidden Layers in Neural Networks. 2024. URL: <https://kellerjordan.github.io/posts/muon/>.
- [54] Jared Kaplan et al. Scaling Laws for Neural Language Models. 2020. arXiv: 2001.08361 [cs.LG]. URL: <https://arxiv.org/abs/2001.08361>.
- [55] Angelos Katharopoulos et al. “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention”. In: Proceedings of ICML. Ed. by Hal Daumé III and Aarti Singh. PMLR, 2020, pp. 5156–5165. URL: <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- [56] Kimi CLI. Moonshot AI. 2026. URL: <https://www.kimi.com/code>.
- [57] Kimi Team. Attention Residuals. Preprint. 2026.
- [58] Kimi Team. Kimi K2: Open Agentic Intelligence. 2025. arXiv: 2507.20534 [cs.LG].
- [59] Kimi Team. “Kimi K2.5: Visual Agentic Intelligence”. In: arXiv preprint arXiv:2602.02276 (2026).
- [60] Kimi Team. Kimi K3: Open Frontier Intelligence. Moonshot AI. July 16, 2026. URL: <https://www.kimi.com/blog/kimi-k3>.
- [61] Kimi Team. “Kimi-vl technical report”. In: arXiv preprint arXiv:2504.07491 (2025).
- [62] Kimi Team. PerceptionBench: Evaluating Atomic Visual Perception in Multimodal Large Language Models. Moonshot AI. 2026. URL: <https://www.kimi.com/blog/perception-bench>.
- [63] Kimi Team et al. Kimi Linear: An Expressive, Efficient Attention Architecture. 2025. arXiv: 2510.26692 [cs.CL].
- [64] Chris Lattner et al. “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation”. In: 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO). 2021, pp. 2–14. DOI: 10.1109/CGO51591.2021.9370308.
- [65] Legal Research Bench. Vals AI. 2026. URL: https://www.vals.ai/benchmarks/legal_research.
- [66] Dmitry Lepikhin et al. “Gshard: Scaling giant models with conditional computation and automatic sharding”. In: arXiv preprint arXiv:2006.16668 (2020).
- [67] Mike Lewis et al. “BASE Layers: Simplifying Training of Large, Sparse Models”. In: Proceedings of ICML. 2021.
- [68] Huiba Li et al. “DADI: Block-Level Image Service for Agile and Elastic Application Deployment”. In: 2020 USENIX Annual Technical Conference (USENIX ATC). 2020, pp. 727–740.
- [69] Junlong Li et al. The Tool Decathlon: Benchmarking Language Agents for Diverse, Realistic, and Long-Horizon Task Execution. ICLR 2026. 2025. arXiv: 2510.25726 [cs.CL].
- [70] Yuetai Li et al. “JobBench: Aligning Agent Work With Human Will”. In: (2026). arXiv: 2605.26329 [cs.AI].
- [71] Yuhui Li et al. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via

- Training-Time Test. 2025. arXiv: 2503.01840 [cs.CL]. URL: <https://arxiv.org/abs/2503.01840>.
- [72] Hao Liu, Matei Zaharia, and Pieter Abbeel. “Ring Attention with Blockwise Transformers for Near-Infinite Context”. In: (2023). arXiv: 2310.01889 [cs.CL]. URL: <https://arxiv.org/abs/2310.01889>.
- [73] Jingyuan Liu et al. Muon is Scalable for LLM Training. 2025. arXiv: 2502.16982 [cs.LG]. URL: <https://arxiv.org/abs/2502.16982>.
- [74] LMArena Leaderboard. LMArena. 2026. URL: <https://lmarena.ai/leaderboard>.
- [75] Kevin Lu and Thinking Machines Lab. On-policy distillation. Thinking Machines Lab: Connectionism. 2025. URL: <https://thinkingmachines.ai/blog/on-policy-distillation/>.
- [76] Bohan Lyu et al. “MLS-Bench: A Holistic and Rigorous Assessment of AI Systems on Building Better AI”. In: (2026). arXiv: 2605.08678 [cs.LG].
- [77] Eric Martin and Chris Cundy. “Parallelizing Linear Recurrent Neural Nets Over Sequence Length”. In: Proceedings of ICLR. 2018. URL: <https://openreview.net/forum?id=HyUNwulC->.
- [78] Mike A Merrill et al. “Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces”. In: arXiv preprint arXiv:2601.11868 (2026).
- [79] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. 2018. arXiv: 1805.02867 [cs.PF]. URL: <https://arxiv.org/abs/1805.02867>.
- [80] Nangate, Inc. Nangate 45nm Open Cell Library. <https://si2.org/open-cell-library/>. Version PDKv1_3_v2010_12. Donated to and distributed by the Silicon Integration Initiative (Si2). 2010. (Visited on 07/27/2026).
- [81] Deepak Narayanan et al. “Efficient large-scale language model training on gpu clusters using megatron-lm”. In: Proceedings of the international conference for high performance computing, networking, storage and analysis. 2021, pp. 1–15.
- [82] NCCL: The NVIDIA Collective Communications Library. NVIDIA. 2026. URL: <https://developer.nvidia.com/nccl>.
- [83] OpenAI. Introducing OpenAI o3 and o4-mini. Apr. 2025. URL: <https://openai.com/index/introducing-o3-and-o4-mini/>.
- [84] OpenAI. Learning to Reason with LLMs. <https://openai.com/index/learning-to-reason-with-llms/>. Accessed: 2026-07-23. 2024.
- [85] OpenAI Harmony Response Format. OpenAI. 2025. URL: <https://github.com/openai/harmony>.
- [86] OpenClaw. OpenClaw. 2026. URL: <https://docs.openclaw.ai/>.
- [87] Krista Opsahl-Ong et al. “OfficeQA Pro: An Enterprise Benchmark for End-to-End Grounded Reasoning”. In: (2026). arXiv: 2603.08655.
- [88] Linke Ouyang et al. OmniDocBench: Benchmarking Diverse PDF Document Parsing with Comprehensive Annotations. 2025. arXiv: 2412.07626 [cs.CV]. URL: <https://arxiv.org/abs/2412.07626>.
- [89] Adam Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. 2019. arXiv:1912.01703 [cs.LG].
- [90] Tejal Patwardhan et al. GDPval: Evaluating AI Model Performance on Real-World Economically Valuable Tasks. 2025. arXiv: 2510.04374 [cs.LG]. URL: <https://arxiv.org/abs/2510.04374>.
- [91] Bo Peng et al. RWKV-7 “Goose” with Expressive Dynamic State Evolution. 2025. arXiv: 2503.14456 [cs.CL].
- [92] Bowen Peng et al. “Yarn: Efficient context window extension of large language models”. In: arXiv preprint arXiv:2309.00071 (2023).
- [93] Long Phan et al. Humanity’s Last Exam. 2025. arXiv: 2501.14249 [cs.LG]. URL: <https://arxiv.org/abs/2501.14249>.
- [94] PostTrainBench. 2026. URL: <https://posttrainbench.com/>.
- [95] ProgramBench. Vals AI. 2026. URL: <https://www.vals.ai/benchmarks/programbench>.
- [96] Ruoyu Qin et al. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. 2024. arXiv:2407.00079 [cs.DC].
- [97] Zhen Qin et al. HGRN2: Gated Linear RNNs with State Expansion. 2024. arXiv: 2404.07904 [cs.CL]. URL: <https://arxiv.org/abs/2404.07904>.
- [98] Haiquan Qiu and Quanming Yao. “Why Low-Precision Transformer Training Fails: An Analysis on Flash Attention”. In: International Conference on Learning Representations (ICLR). 2026. arXiv: 2510.04212 [cs.LG].
- [99] Zihan Qiu et al. Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free. 2025. arXiv: 2505.06708 [cs.CL].
- [100] Samyam Rajbhandari et al. “Zero: Memory optimizations toward training trillion parameter models”. In: SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE. 2020, pp. 1–16.
- [101] David Rein et al. “Gpqa: A graduate-level google-proof q&a benchmark”. In: First Conference on Language Modeling. 2024.

- [102] Jonathan Roberts et al. ZeroBench: An Impossible Visual Benchmark for Contemporary Large Multimodal Models. 2025. arXiv: 2502.09696 [cs.CV]. URL: <https://arxiv.org/abs/2502.09696>.
- [103] Bitan Darvish Rouhani et al. “Microscaling Data Formats for Deep Learning”. In: arXiv preprint arXiv:2310.10537 (2023). arXiv: 2310.10537 [cs.LG].
- [104] Alexander Samarin et al. LK Losses: Direct Acceptance Rate Optimization for Speculative Decoding. 2026. arXiv: 2602.23881 [cs.LG]. URL: <https://arxiv.org/abs/2602.23881>.
- [105] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. “Linear Transformers Are Secretly Fast Weight Programmers”. In: Proceedings of ICML. Ed. by Marina Meila and Tong Zhang. PMLR, 2021, pp. 9355–9366. URL: <https://proceedings.mlr.press/v139/schlag21a.html>.
- [106] Manasi Sharma et al. “ResearchRubrics: A Benchmark of Prompts and Rubrics For Evaluating Deep Research Agents”. In: The Fourteenth International Conference on Learning Representations. 2026. URL: <https://openreview.net/forum?id=ErnvfmSXOP>.
- [107] Noam Shazeer. GLU Variants Improve Transformer. 2020. arXiv: 2002.05202 [cs.LG]. URL: <https://arxiv.org/abs/2002.05202>.
- [108] Daniel Shepard and Robin Salimans. “AutomationBench”. In: (2026). arXiv: 2604.18934 [cs.AI].
- [109] Kean Shi et al. SaaS-Bench: Can Computer-Use Agents Leverage Real-World SaaS to Solve Professional Workflows? 2026. arXiv: 2605.15777 [cs.AI]. URL: <https://arxiv.org/abs/2605.15777>.
- [110] Benjamin F. Spector et al. “ThunderKittens: Simple, Fast, and Adorable Kernels”. In: The Thirteenth International Conference on Learning Representations. 2025. URL: <https://openreview.net/forum?id=0fjfvOSUra>.
- [111] Jianlin Su. Travels in MoE: 6. Promoting Load Balance via Optimal Assignment. Blog post (in Chinese). Feb. 2026. URL: <https://spaces.ac.cn/archives/11619>.
- [112] Hanchi Sun et al. Expert Threshold Routing for Autoregressive Language Modeling with Dynamic Computation Allocation and Load Balancing. 2026. arXiv: 2603.11535 [cs.AI].
- [113] Weigao Sun et al. “LASP-2: Rethinking Sequence Parallelism for Linear Attention and Its Hybrid”. In: (2025). arXiv: 2502.07563 [cs.LG]. URL: <https://arxiv.org/abs/2502.07563>.
- [114] Weigao Sun et al. “Linear Attention Sequence Parallelism”. In: (2024). arXiv: 2404.02882 [cs.LG]. URL: <https://arxiv.org/abs/2404.02882>.
- [115] Yiyou Sun et al. Agents’ Last Exam. 2026. arXiv: 2606.05405 [cs.AI]. URL: <https://arxiv.org/abs/2606.05405>.
- [116] Yuan Sun. Binary-Integer-Programming Based Algorithm for Expert Load Balancing in Mixture-of-Experts Models. 2025. arXiv: 2502.15451 [cs.LG].
- [117] SWE Marathon. 2026. URL: <https://www.swe-marathon.org/>.
- [118] Kimi Team. Kimi k1.5: Scaling Reinforcement Learning with LLMs. 2025. arXiv: 2501.12599 [cs.AI]. URL: <https://arxiv.org/abs/2501.12599>.
- [119] The Tool Decathlon: Benchmarking Language Agents for Diverse, Realistic, and Long-Horizon Task Execution. July 24, 2026. URL: <https://toolathlon.xyz/introduction>.
- [120] Thinking Machines Lab. Inkling: Our Open-Weights Model. <https://thinkingmachines.ai/news/introducing-inkling/>. Accessed: 2026-07-23. July 2026.
- [121] Minyang Tian et al. SciCode: A Research Coding Benchmark Curated by Scientists. 2024. arXiv: 2407.13168 [cs.AI]. URL: <https://arxiv.org/abs/2407.13168>.
- [122] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations”. In: Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL). 2019.
- [123] UK AI Security Institute and U.S. Center for AI Standards and Innovation. Preliminary Assessment of Kimi K3’s Cyber Capabilities. <https://www.aisi.gov.uk/blog/preliminary-assessment-of-kimi-k3s-cyber-capabilities>. July 2026.
- [124] Vals AI. Vals AI. 2026. URL: <https://www.vals.ai/>.
- [125] Ashish Vaswani et al. “Attention is All you Need”. In: Advances in NeurIPS. Ed. by I. Guyon et al. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee Paper.pdf.
- [126] Nikhita Vedula et al. DeepSearchQA: Bridging the Comprehensiveness Gap for Deep Research Agents. 2025. URL: https://storage.googleapis.com/deepmind-media/DeepSearchQA/DeepSearchQA_bench
- [127] Bertie Vidgen et al. APEX-Agents. 2026. arXiv: 2601.14242 [cs.CL].
- [128] Ke Wang et al. “Measuring Multimodal Mathematical Reasoning with MATH-Vision Dataset”. In: Advances in NeurIPS. Ed. by A. Globerson et al. Vol. 37. Curran Associates, Inc., 2024, pp. 95095–95169. DOI: 10.52202/079017-3014. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/ad Paper-Datasets_and_Benchmarks_Track.pdf.

- [129] Lei Wang et al. “TileLang: A Composable Tiled Programming Model for AI Systems”. In: arXiv preprint arXiv:2504.17577 (2025). URL: <https://arxiv.org/abs/2504.17577>.
- [130] Zirui Wang et al. CharXiv: Charting Gaps in Realistic Chart Understanding in Multimodal LLMs. 2024. arXiv:2406.18521 [cs.CL]. URL: <https://arxiv.org/abs/2406.18521>.
- [131] Jason Wei et al. BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents. 2025. arXiv:2504.12516 [cs.CL]. URL: <https://arxiv.org/abs/2504.12516>.
- [132] Xinming Wei et al. UltraEP: Unleash MoE Training and Inference on Rack-Scale Nodes with Near-Optimal Load Balancing. 2026. arXiv: 2606.04101 [cs.DC]. URL: <https://arxiv.org/abs/2606.04101>.
- [133] Zijian Wu et al. “MCPMark: A benchmark for stress-testing realistic and comprehensive mcp use”. In: arXiv preprint arXiv:2509.24002 (2025).
- [134] B. Xiao et al. “MiMo-V2-Flash Technical Report”. In: arXiv preprint arXiv:2601.02780 (2026).
- [135] Xiaomi MiMo Team. MiMo-V2.5-Pro. <https://huggingface.co/collections/XiaomiMiMo/mimo-v25.2026>.
- [136] Tianbao Xie et al. “Introducing OSWorld-Verified”. In: xlang.ai (July 2025). URL: <https://xlang.ai/blog/osworld-verified>.
- [137] Zijie Yan et al. Scalable Training of Mixture-of-Experts Models with Megatron Core. 2026. arXiv: 2603.07685 [cs.DC]. URL: <https://arxiv.org/abs/2603.07685>.
- [138] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. “Gated Delta Networks: Improving Mamba2 with Delta Rule”. In: Proceedings of ICLR. 2025. URL: <https://openreview.net/forum?id=r8H7xhYPwz>.
- [139] Songlin Yang and Yu Zhang. FLA: A Triton-Based Library for Hardware-Efficient Implementations of Linear Attention Mechanism. Jan. 2024. URL: <https://github.com/fla-org/flash-linear-attention>.
- [140] Songlin Yang et al. “Gated Linear Attention Transformers with Hardware-Efficient Training”. In: Proceedings of ICML. PMLR, 2024.
- [141] Songlin Yang et al. “Parallelizing Linear Transformers with the Delta Rule over Sequence Length”. In: Proceedings of NeurIPS. 2024.
- [142] Yaoyu Wang. Context Parallelism for DeltaNet. 2025. URL: <https://yywangcs.notion.site/DeltaNet-2a9fc9f5d8058013a498f34e0b25bd52>.
- [143] Mengqi Yuan et al. OSWorld2.0: Benchmarking Computer Use Agents on Long-Horizon Real-World Tasks. 2026. arXiv: 2606.29537 [cs.AI]. URL: <https://arxiv.org/abs/2606.29537>.
- [144] Xiang Yue et al. MMMU-Pro: A More Robust Multi-discipline Multimodal Understanding Benchmark. 2024. arXiv: 2409.02813 [cs.CL]. URL: <https://arxiv.org/abs/2409.02813>.
- [145] Aohan Zeng et al. GLM-5: from Vibe Coding to Agentic Engineering. 2026. arXiv: 2602.15763 [cs.LG]. URL: <https://arxiv.org/abs/2602.15763>.
- [146] Biao Zhang and Rico Sennrich. “Root mean square layer normalization”. In: Advances in NeurIPS 32 (2019).
- [147] Chenggang Zhao et al. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>. 2025.
- [148] Yilun Zhao et al. “MMVU: Measuring Expert-Level Multi-Discipline Video Understanding”. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). June 2025, pp. 8475–8489.
- [149] Runjie Zhou et al. WorldVQA: Measuring Atomic World Knowledge in Multimodal Large Language Models. 2026. arXiv: 2602.02537 [cs.CV]. URL: <https://arxiv.org/abs/2602.02537>.
- [150] Jian Zhu et al. “SpreadsheetBench 2: Evaluating Agents on End-to-End Business Spreadsheet Workflows”. In: (2026). arXiv: 2606.29955 [cs.SE].
- [1] τ^3 -Banking. Sierra. 2026. URL: <https://taubench.com/blog/tau-knowledge.html>.
- [2] AA-Briefcase: Agentic Knowledge Work Benchmark. Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/evaluations/aa-briefcase>.
- [3] Alexandru Agache et al. “Firecracker: Lightweight Virtualization for Serverless Applications”. In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI). 2020, pp. 419–434.
- [4] Agents’ Last Exam. UC Berkeley RDI. 2026. URL: <https://agents-last-exam.org/leaderboard>.
- [5] Jason Ansel et al. “PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation”. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2024. DOI: 10.1145/3620665.3640366.
- [6] Anthropic. Claude’s Extended Thinking. <https://www.anthropic.com/research/visible->

extended-thinking. Accessed: 2026-07-23. Feb. 2025.

[7] Anthropic. Introducing Claude 4. <https://www.anthropic.com/news/claude-4>. Accessed: 2026-07-23. May 2025.

[8] Artificial Analysis. Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/>.

[9] Artificial Analysis Long Context Reasoning (AA-LCR). Artificial Analysis. 2026. URL: <https://artificialanalysis.ai/evaluations/artificial-analysis-long-context-reasoning>.

[10] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural Machine Translation by Jointly Learning to Align and Translate. 2014. arXiv: 1409.0473 [cs.CL]. URL: <https://arxiv.org/abs/1409.0473>.

[11] Chaithanya Bandi et al. MCP-Atlas: A Large-Scale Benchmark for Tool-Use Competency with Real MCP Servers. 2026. arXiv: 2602.00933 [cs.SE].

[12] Better MoE Model Inference with Warp Decode. Cursor. 2026. URL: <https://cursor.com/blog/warp-decode> (visited on 07/20/2026).

[13] Liang Chen et al. BabyVision: Visual Reasoning Beyond Language. 2026. arXiv: 2601.06521 [cs.CV]. URL: <https://arxiv.org/abs/2601.06521>.

[14] Yutian Chen et al. FlashKDA: Flash Kimi Delta Attention. 2026. URL: <https://github.com/MoonshotAI/FlashKD>.

[15] Claude Code. Anthropic. 2026. URL: <https://docs.anthropic.com/en/docs/claude-code>.

[16] Claude Fable 5. Anthropic. 2026. URL: <https://www.anthropic.com/news/claude-fable-5-mythos-5>.

[17] Claude Opus 4.8. Anthropic. 2026. URL: <https://www.anthropic.com/news/claude-opus-4-8>.

[18] Claude Sonnet 5. Anthropic. 2026. URL: <https://www-cdn.anthropic.com/283ef97c476cf442c91d9a37d5b2142>.

[19] Claude Sonnet 5. Anthropic. 2026. URL: <https://www.anthropic.com/news/claude-sonnet-5>.

[20] Codex. OpenAI. 2026. URL: <https://github.com/openai/codex>.

[21] CorpFin v2. Vals AI. 2026. URL: https://www.vals.ai/benchmarks/corp_fin_v2.

[22] cuBLAS. NVIDIA. 2026. URL: <https://developer.nvidia.com/cublas>.

[23] Damai Dai et al. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. 2024. arXiv: 2401.06066 [cs.CL]. URL: <https://arxiv.org/abs/2401.06066>.

[24] Tri Dao and Albert Gu. “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality”. In: CoRR abs/2405.21060 (2024). DOI: 10.48550/ARXIV.2405.21060. arXiv: 2405.21060. URL: <https://doi.org/10.48550/arXiv.2405.21060>.

[25] Dao AI Lab. ReplaySSM: Cache SSM Inputs, Not State. <https://tridao.me/blog/2026/replayssm/>. June 2026.

[26] Yann N. Dauphin et al. “Language Modeling with Gated Convolutional Networks”. In: Proceedings of the 34th International Conference on Machine Learning. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 933–941. URL: <https://proceedings.mlr.press/v70/dauphin17a.html>.

[27] Soham De et al. Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models. 2024. arXiv: 2402.19427 [cs.LG]. URL: <https://arxiv.org/abs/2402.19427>.

[28] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. arXiv: 2405.04434 [cs.CL]. URL: <https://arxiv.org/abs/2405.04434>.

[29] DeepSeek-AI. “DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence”. In: arXiv preprint arXiv:2606.19348 (2026).

[30] DeepSeek-AI et al. DeepSeek-V3 Technical Report. 2024. arXiv: 2412.19437 [cs.CL]. URL: <https://arxiv.org/abs/2412.19437>.

[31] DeepSWE Benchmark. Datacurve. 2026. URL: <https://deepswe.datacurve.ai/>.

[32] Venmugil Elango et al. LatentMoE: Toward Optimal Accuracy per FLOP and Parameter in Mixture of Experts. 2026. arXiv: 2601.18089 [cs.LG]. URL: <https://arxiv.org/abs/2601.18089>.

[33] William Fedus, Barret Zoph, and Noam Shazeer. “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity”. In: Journal of Machine Learning Research 23.120 (2022), pp. 1–39.

[34] Finance Agent v2. Vals AI. 2026. URL: <https://www.vals.ai/benchmarks/fabv2>.

[35] FrontierSWE. 2026. URL: <https://www.frontierswe.com/>.

[36] Chaoyou Fu et al. Video-MME: The First-Ever Comprehensive Evaluation Benchmark of Multimodal LLMs in Video Analysis. 2024. arXiv: 2405.21075 [cs.CV]. URL: <https://arxiv.org/abs/2405.21075>.

[37] GLM-5.2. Z.ai. 2026. URL: <https://z.ai/blog/glm-5.2>.

[38] GPT-5.5. OpenAI. 2026. URL: <https://openai.com/index/introducing-gpt-5-5/>.

[39] GPT-5.6 Sol. OpenAI. 2026. URL: <https://openai.com/index/previewing-gpt-5-6-sol/>.

[40] Daya Guo et al. “DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning”. In: Nature 645.8081 (2025), pp. 633–638. ISSN: 1476-4687. DOI: 10.1038/s41586-025-09422-z.

URL: <http://dx.doi.org/10.1038/s41586-025-09422-z>.

[41] Wentao Guo et al. SonicMoE: Accelerating MoE with IO and Tile-aware Optimizations. 2025. arXiv: 2512.14080 [cs.LG]. URL: <https://arxiv.org/abs/2512.14080>.

[42] Harvey LAB: Legal Agent Benchmark. Harvey. 2026. URL: <https://www.harvey.ai/blog/introducing-harveys-legal-agent-benchmark>.

[43] Kaiming He et al. Deep Residual Learning for Image Recognition. 2015. arXiv: 1512.03385 [cs.CV]. URL: <https://arxiv.org/abs/1512.03385>.

[44] Hermes Agent. Nous Research. 2026. URL: <https://hermes-agent.nousresearch.com/docs/>.

[45] Jordan Hoffmann et al. Training Compute-Optimal Large Language Models. 2022. arXiv: 2203.15556 [cs.CL]. URL: <https://arxiv.org/abs/2203.15556>.

[46] Shengding Hu, Yuge Tu, Xu Han, et al. “MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies”. In: (2024). arXiv: 2404.06395 [cs.CL].

[47] Ailin Huang, Ang Li, Aobo Kong, et al. “Step 3.5 Flash: Open Frontier-Level Intelligence with 11B Active Parameters”. In: arXiv preprint arXiv:2602.10604 (2026).

[48] Yanping Huang et al. “Gpipe: Efficient training of giant neural networks using pipeline parallelism”. In: Advances in neural information processing systems 32 (2019).

[49] Benoit Jacob et al. “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2018, pp. 2704–2713.

[50] Sam Ade Jacobs et al. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. 2023. arXiv: 2309.14509 [cs.LG]. URL: <https://arxiv.org/abs/2309.14509>.

[51] Peijie Jiang et al. PowLU: An Activation Function for Stable Pre-Training of LLMs. 2026. arXiv: 2605.25704 [cs.CL]. URL: <https://arxiv.org/abs/2605.25704>.

[52] JobBench: Aligning Agent Work with Human Will. July 24, 2026. URL: <https://job-bench.github.io/>.

[53] Keller Jordan et al. Muon: An Optimizer for Hidden Layers in Neural Networks. 2024. URL: <https://kellerjordan.github.io/posts/muon/>.

[54] Jared Kaplan et al. Scaling Laws for Neural Language Models. 2020. arXiv: 2001.08361 [cs.LG]. URL: <https://arxiv.org/abs/2001.08361>.

[55] Angelos Katharopoulos et al. “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention”. In: Proceedings of ICML. Ed. by Hal Daumé III and Aarti Singh. PMLR, 2020, pp. 5156–5165. URL: <https://proceedings.mlr.press/v119/katharopoulos20a.html>.

[56] Kimi CLI. Moonshot AI. 2026. URL: <https://www.kimi.com/code>.

[57] Kimi Team. Attention Residuals. Preprint. 2026.

[58] Kimi Team. Kimi K2: Open Agentic Intelligence. 2025. arXiv: 2507.20534 [cs.LG].

[59] Kimi Team. “Kimi K2.5: Visual Agentic Intelligence”. In: arXiv preprint arXiv:2602.02276 (2026).

[60] Kimi Team. Kimi K3: Open Frontier Intelligence. Moonshot AI. July 16, 2026. URL: <https://www.kimi.com/blog/kimi-k3>.

[61] Kimi Team. “Kimi-vl technical report”. In: arXiv preprint arXiv:2504.07491 (2025).

[62] Kimi Team. PerceptionBench: Evaluating Atomic Visual Perception in Multimodal Large Language Models. Moonshot AI. 2026. URL: <https://www.kimi.com/blog/perception-bench>.

[63] Kimi Team et al. Kimi Linear: An Expressive, Efficient Attention Architecture. 2025. arXiv: 2510.26692 [cs.CL].

[64] Chris Lattner et al. “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation”. In: 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO). 2021, pp. 2–14. DOI: 10.1109/CGO51591.2021.9370308.

[65] Legal Research Bench. Vals AI. 2026. URL: https://www.vals.ai/benchmarks/legal_research.

[66] Dmitry Lepikhin et al. “Gshard: Scaling giant models with conditional computation and automatic sharding”. In: arXiv preprint arXiv:2006.16668 (2020).

[67] Mike Lewis et al. “BASE Layers: Simplifying Training of Large, Sparse Models”. In: Proceedings of ICML. 2021.

[68] Huiba Li et al. “DADI: Block-Level Image Service for Agile and Elastic Application Deployment”. In: 2020 USENIX Annual Technical Conference (USENIX ATC). 2020, pp. 727–740.

[69] Junlong Li et al. The Tool Decathlon: Benchmarking Language Agents for Diverse, Realistic, and Long-Horizon Task Execution. ICLR 2026. 2025. arXiv: 2510.25726 [cs.CL].

[70] Yuetai Li et al. “JobBench: Aligning Agent Work With Human Will”. In: (2026). arXiv:

2605.26329 [cs.AI].

[71] Yuhui Li et al. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. 2025. arXiv: 2503.01840 [cs.CL]. URL: <https://arxiv.org/abs/2503.01840>.

[72] Hao Liu, Matei Zaharia, and Pieter Abbeel. “Ring Attention with Blockwise Transformers for Near-Infinite Context”. In: (2023). arXiv: 2310.01889 [cs.CL]. URL: <https://arxiv.org/abs/2310.01889>.

[73] Jingyuan Liu et al. Muon is Scalable for LLM Training. 2025. arXiv: 2502.16982 [cs.LG]. URL: <https://arxiv.org/abs/2502.16982>.

[74] LMArena Leaderboard. LMArena. 2026. URL: <https://lmarena.ai/leaderboard>.

[75] Kevin Lu and Thinking Machines Lab. On-policy distillation. Thinking Machines Lab: Connectionism. 2025. URL: <https://thinkingmachines.ai/blog/on-policy-distillation/>.

[76] Bohan Lyu et al. “MLS-Bench: A Holistic and Rigorous Assessment of AI Systems on Building Better AI”. In: (2026). arXiv: 2605.08678 [cs.LG].

[77] Eric Martin and Chris Cundy. “Parallelizing Linear Recurrent Neural Nets Over Sequence Length”. In: Proceedings of ICLR. 2018. URL: <https://openreview.net/forum?id=HyUNwulC->.

[78] Mike A Merrill et al. “Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces”. In: arXiv preprint arXiv:2601.11868 (2026).

[79] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. 2018. arXiv: 1805.02867 [cs.PF]. URL: <https://arxiv.org/abs/1805.02867>.

[80] Nangate, Inc. Nangate 45nm Open Cell Library. <https://si2.org/open-cell-library/>. Version PDKv1_3_v2010_12. Donated to and distributed by the Silicon Integration Initiative (Si2). 2010. (Visited on 07/27/2026).

[81] Deepak Narayanan et al. “Efficient large-scale language model training on gpu clusters using megatron-lm”. In: Proceedings of the international conference for high performance computing, networking, storage and analysis. 2021, pp. 1–15.

[82] NCCL: The NVIDIA Collective Communications Library. NVIDIA. 2026. URL: <https://developer.nvidia.com/nccl>.

[83] OpenAI. Introducing OpenAI o3 and o4-mini. Apr. 2025. URL: <https://openai.com/index/introducing-o3-and-o4-mini/>.

[84] OpenAI. Learning to Reason with LLMs. <https://openai.com/index/learning-to-reason-with-llms/>. Accessed: 2026-07-23. 2024.

[85] OpenAI Harmony Response Format. OpenAI. 2025. URL: <https://github.com/openai/harmony>.

[86] OpenClaw. OpenClaw. 2026. URL: <https://docs.openclaw.ai/>.

[87] Krista Opsahl-Ong et al. “OfficeQA Pro: An Enterprise Benchmark for End-to-End Grounded Reasoning”. In: (2026). arXiv: 2603.08655.

[88] Linke Ouyang et al. OmniDocBench: Benchmarking Diverse PDF Document Parsing with Comprehensive Annotations. 2025. arXiv: 2412.07626 [cs.CV]. URL: <https://arxiv.org/abs/2412.07626>.

[89] Adam Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. 2019. arXiv:1912.01703 [cs.LG].

[90] Tejal Patwardhan et al. GDPval: Evaluating AI Model Performance on Real-World Economically Valuable Tasks. 2025. arXiv: 2510.04374 [cs.LG]. URL: <https://arxiv.org/abs/2510.04374>.

[91] Bo Peng et al. RWKV-7 “Goose” with Expressive Dynamic State Evolution. 2025. arXiv: 2503.14456 [cs.CL].

[92] Bowen Peng et al. “Yarn: Efficient context window extension of large language models”. In: arXiv preprint arXiv:2309.00071 (2023).

[93] Long Phan et al. Humanity’s Last Exam. 2025. arXiv: 2501.14249 [cs.LG]. URL: <https://arxiv.org/abs/2501.14249>.

[94] PostTrainBench. 2026. URL: <https://posttrainbench.com/>.

[95] ProgramBench. Vals AI. 2026. URL: <https://www.vals.ai/benchmarks/programbench>.

[96] Ruoyu Qin et al. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. 2024. arXiv:2407.00079 [cs.DC].

[97] Zhen Qin et al. HGRN2: Gated Linear RNNs with State Expansion. 2024. arXiv: 2404.07904 [cs.CL]. URL: <https://arxiv.org/abs/2404.07904>.

[98] Haiquan Qiu and Quanming Yao. “Why Low-Precision Transformer Training Fails: An Analysis on Flash Attention”. In: International Conference on Learning Representations (ICLR). 2026. arXiv: 2510.04212 [cs.LG].

[99] Zihan Qiu et al. Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free. 2025. arXiv: 2505.06708 [cs.CL].

[100] Samyam Rajbhandari et al. “Zero: Memory optimizations toward training trillion parameter models”. In: SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE. 2020, pp. 1–16.

- [101] David Rein et al. “Gpqa: A graduate-level google-proof q&a benchmark”. In: First Conference on Language Modeling. 2024.
- [102] Jonathan Roberts et al. ZeroBench: An Impossible Visual Benchmark for Contemporary Large Multimodal Models. 2025. arXiv: 2502.09696 [cs.CV]. URL: <https://arxiv.org/abs/2502.09696>.
- [103] Bitan Darvish Rouhani et al. “Microscaling Data Formats for Deep Learning”. In: arXiv preprint arXiv:2310.10537 (2023). arXiv: 2310.10537 [cs.LG].
- [104] Alexander Samarin et al. LK Losses: Direct Acceptance Rate Optimization for Speculative Decoding. 2026. arXiv: 2602.23881 [cs.LG]. URL: <https://arxiv.org/abs/2602.23881>.
- [105] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. “Linear Transformers Are Secretly Fast Weight Programmers”. In: Proceedings of ICML. Ed. by Marina Meila and Tong Zhang. PMLR, 2021, pp. 9355–9366. URL: <https://proceedings.mlr.press/v139/schlag21a.html>.
- [106] Manasi Sharma et al. “ResearchRubrics: A Benchmark of Prompts and Rubrics For Evaluating Deep Research Agents”. In: The Fourteenth International Conference on Learning Representations. 2026. URL: <https://openreview.net/forum?id=ErnvfmSXOP>.
- [107] Noam Shazeer. GLU Variants Improve Transformer. 2020. arXiv: 2002.05202 [cs.LG]. URL: <https://arxiv.org/abs/2002.05202>.
- [108] Daniel Shepard and Robin Salimans. “AutomationBench”. In: (2026). arXiv: 2604.18934 [cs.AI].
- [109] Kean Shi et al. SaaS-Bench: Can Computer-Use Agents Leverage Real-World SaaS to Solve Professional Workflows? 2026. arXiv: 2605.15777 [cs.AI]. URL: <https://arxiv.org/abs/2605.15777>.
- [110] Benjamin F. Spector et al. “ThunderKittens: Simple, Fast, and Adorable Kernels”. In: The Thirteenth International Conference on Learning Representations. 2025. URL: <https://openreview.net/forum?id=0fjfvOSUra>.
- [111] Jianlin Su. Travels in MoE: 6. Promoting Load Balance via Optimal Assignment. Blog post (in Chinese). Feb. 2026. URL: <https://spaces.ac.cn/archives/11619>.
- [112] Hanchi Sun et al. Expert Threshold Routing for Autoregressive Language Modeling with Dynamic Computation Allocation and Load Balancing. 2026. arXiv: 2603.11535 [cs.AI].
- [113] Weigao Sun et al. “LASP-2: Rethinking Sequence Parallelism for Linear Attention and Its Hybrid”. In: (2025). arXiv: 2502.07563 [cs.LG]. URL: <https://arxiv.org/abs/2502.07563>.
- [114] Weigao Sun et al. “Linear Attention Sequence Parallelism”. In: (2024). arXiv: 2404.02882 [cs.LG]. URL: <https://arxiv.org/abs/2404.02882>.
- [115] Yiyu Sun et al. Agents’ Last Exam. 2026. arXiv: 2606.05405 [cs.AI]. URL: <https://arxiv.org/abs/2606.05405>.
- [116] Yuan Sun. Binary-Integer-Programming Based Algorithm for Expert Load Balancing in Mixture-of-Experts Models. 2025. arXiv: 2502.15451 [cs.LG].
- [117] SWE Marathon. 2026. URL: <https://www.swe-marathon.org/>.
- [118] Kimi Team. Kimi k1.5: Scaling Reinforcement Learning with LLMs. 2025. arXiv: 2501.12599 [cs.AI]. URL: <https://arxiv.org/abs/2501.12599>.
- [119] The Tool Decathlon: Benchmarking Language Agents for Diverse, Realistic, and Long-Horizon Task Execution. July 24, 2026. URL: <https://toolathlon.xyz/introduction>.
- [120] Thinking Machines Lab. Inkling: Our Open-Weights Model. <https://thinkingmachines.ai/news/introducing-inkling/>. Accessed: 2026-07-23. July 2026.
- [121] Minyang Tian et al. SciCode: A Research Coding Benchmark Curated by Scientists. 2024. arXiv: 2407.13168 [cs.AI]. URL: <https://arxiv.org/abs/2407.13168>.
- [122] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations”. In: Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL). 2019.
- [123] UK AI Security Institute and U.S. Center for AI Standards and Innovation. Preliminary Assessment of Kimi K3’s Cyber Capabilities. <https://www.aisi.gov.uk/blog/preliminary-assessment-of-kimi-k3s-cyber-capabilities>. July 2026.
- [124] Vals AI. Vals AI. 2026. URL: <https://www.vals.ai/>.
- [125] Ashish Vaswani et al. “Attention is All you Need”. In: Advances in NeurIPS. Ed. by I. Guyon et al. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee Paper.pdf.
- [126] Nikhita Vedula et al. DeepSearchQA: Bridging the Comprehensiveness Gap for Deep Research Agents. 2025. URL: https://storage.googleapis.com/deepmind-media/DeepSearchQA/DeepSearchQA_bench
- [127] Bertie Vidgen et al. APEX-Agents. 2026. arXiv: 2601.14242 [cs.CL].
- [128] Ke Wang et al. “Measuring Multimodal Mathematical Reasoning with MATH-Vision Dataset”. In: Advances in NeurIPS. Ed. by A. Globerson et al. Vol. 37. Curran Associates, Inc., 2024,

- pp. 95095–95169. DOI: 10.52202/079017-3014. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/adPaper-Datasets_and_Benchmarks_Track.pdf.
- [129] Lei Wang et al. “TileLang: A Composable Tiled Programming Model for AI Systems”. In: arXiv preprint arXiv:2504.17577 (2025). URL: <https://arxiv.org/abs/2504.17577>.
- [130] Zirui Wang et al. CharXiv: Charting Gaps in Realistic Chart Understanding in Multimodal LLMs. 2024. arXiv:2406.18521 [cs.CL]. URL: <https://arxiv.org/abs/2406.18521>.
- [131] Jason Wei et al. BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents. 2025. arXiv:2504.12516 [cs.CL]. URL: <https://arxiv.org/abs/2504.12516>.
- [132] Xinming Wei et al. UltraEP: Unleash MoE Training and Inference on Rack-Scale Nodes with Near-Optimal Load Balancing. 2026. arXiv: 2606.04101 [cs.DC]. URL: <https://arxiv.org/abs/2606.04101>.
- [133] Zijian Wu et al. “MCPMark: A benchmark for stress-testing realistic and comprehensive mcp use”. In: arXiv preprint arXiv:2509.24002 (2025).
- [134] B. Xiao et al. “MiMo-V2-Flash Technical Report”. In: arXiv preprint arXiv:2601.02780 (2026).
- [135] Xiaomi MiMo Team. MiMo-V2.5-Pro. <https://huggingface.co/collections/XiaomiMiMo/mimo-v25.2026>.
- [136] Tianbao Xie et al. “Introducing OSWorld-Verified”. In: xlang.ai (July 2025). URL: <https://xlang.ai/blog/osworld-verified>.
- [137] Zijie Yan et al. Scalable Training of Mixture-of-Experts Models with Megatron Core. 2026. arXiv: 2603.07685 [cs.DC]. URL: <https://arxiv.org/abs/2603.07685>.
- [138] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. “Gated Delta Networks: Improving Mamba2 with Delta Rule”. In: Proceedings of ICLR. 2025. URL: <https://openreview.net/forum?id=r8H7xhYPwz>.
- [139] Songlin Yang and Yu Zhang. FLA: A Triton-Based Library for Hardware-Efficient Implementations of Linear Attention Mechanism. Jan. 2024. URL: <https://github.com/fla-org/flash-linear-attention>.
- [140] Songlin Yang et al. “Gated Linear Attention Transformers with Hardware-Efficient Training”. In: Proceedings of ICML. PMLR, 2024.
- [141] Songlin Yang et al. “Parallelizing Linear Transformers with the Delta Rule over Sequence Length”. In: Proceedings of NeurIPS. 2024.
- [142] Yaoyu Wang. Context Parallelism for DeltaNet. 2025. URL: <https://yywangcs.notion.site/DeltaNet-2a9fc9f5d8058013a498f34e0b25bd52>.
- [143] Mengqi Yuan et al. OSWorld2.0: Benchmarking Computer Use Agents on Long-Horizon Real-World Tasks. 2026. arXiv: 2606.29537 [cs.AI]. URL: <https://arxiv.org/abs/2606.29537>.
- [144] Xiang Yue et al. MMMU-Pro: A More Robust Multi-discipline Multimodal Understanding Benchmark. 2024. arXiv: 2409.02813 [cs.CL]. URL: <https://arxiv.org/abs/2409.02813>.
- [145] Aohan Zeng et al. GLM-5: from Vibe Coding to Agentic Engineering. 2026. arXiv: 2602.15763 [cs.LG]. URL: <https://arxiv.org/abs/2602.15763>.
- [146] Biao Zhang and Rico Sennrich. “Root mean square layer normalization”. In: Advances in NeurIPS 32 (2019).
- [147] Chenggang Zhao et al. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>. 2025.
- [148] Yilun Zhao et al. “MMVU: Measuring Expert-Level Multi-Discipline Video Understanding”. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). June 2025, pp. 8475–8489.
- [149] Runjie Zhou et al. WorldVQA: Measuring Atomic World Knowledge in Multimodal Large Language Models. 2026. arXiv: 2602.02537 [cs.CV]. URL: <https://arxiv.org/abs/2602.02537>.
- [150] Jian Zhu et al. “SpreadsheetBench 2: Evaluating Agents on End-to-End Business Spreadsheet Workflows”. In: (2026). arXiv: 2606.29955 [cs.SE].

A Contributions

A 贡献

The listing of contributors is in alphabetical order based on their last names.

贡献者名单按姓氏的字母顺序排列。

Tongtong Bai	Fuxuan Gao	Guokun Lai	Wenzhou Lyu
Yifan Bai	Hongcheng Gao	Aidi Li	Shaoguang Mao
Yiping Bao	Jingyue Gao	Cheng Li	Yuan Mei
M. C.	Tong Gao	Chengyuan Li	Xin Men
Jianfeng Cai	Weijia Gao	Cong Li	Minqing Ni
Xinyuan Cai	Shangyi Geng	Fang Li	Yixuan Niu
Peizhou Cao	Jie Gong	Guanyu Li	Siyuan Pan
Yuxuan Cao	Linha Gong	Haoyang Li	Shujun Peng
Ziwei Chai	Shengao Gong	Jia Li	Zhangyang Qi
Y. Charles	Xiaochen Gong	Junxiong Li	Ruoyu Qin
H.S. Che	Qizheng Gu	Lei Li	ZeChao Qin
Guanduo Chen	Yicheng Gu	Letian Li	Zeyu Qin
Guangyu Chen	Shuhao Guan	Lincan Li	Haiquan Qiu
Guanzheng Chen	Haiqing Guo	Weihong Li	Jianxin Qiu
Huarong Chen	Shiqi Guo	Wentao Li	Jiezhong Qiu
Jia Chen	Xiang Guo	Xintong Li	Bowen Qu
Jianlong Chen	Zhengyan Guo	Yang Li	Yuhao Qu
Jun Chen	Beixi Hao	Yishen Li	Zeyu Shang
Kexin Chen	Wenxin Hao	Yiwei Li	Youbo Shao
Peng Chen	Xiaoru Hao	Yuxiao Li	Han Shen
Ruijue Chen	Dailan He	Zhaowei Li	Jincheng Shi
Wentao Chen	Haotian He	Zhaoxi Li	Juanfeng Shi
Xin Chen	Lehan He	Zheming Li	Lidong Shi
Yang Chen	Qi He	Zhengxiao Li	Shengyuan Shi
Yanru Chen	Weiran He	Zhiyuan Li	Wingchun Siu
Yifei Chen	Xinran He	Jiawei Lin	Pengwei Song
Yingjiang Chen	Xinyi He	Xiaohan Lin	Xiaoxi Song
Yuankun Chen	Yibo He	Yibo Lin	Jianlin Su
Yujie Chen	Yunjia He	Zichao Lin	Yunfeng Su
Yutian Chen	Chao Hong	Ziyan Lin	Zhaochen Su
Zhirong Chen	Tiange Hong	Bill Liu	Lin Sui
Dazhi Cheng	Hao Hu	Boxiao Liu	Jingsong Sun
Yean Cheng	Jiaxi Hu	Chuan Liu	Junyao Sun
Jialei Cui	Ruikun Hu	Liang Liu	Shaoning Sun
Jingbing Cui	Weiming Hu	Shaowei Liu	Shuzhe Sun
Anqi Dai	Yangyang Hu	Shudong Liu	Tongyu Sun
Jiaqi Deng	Zhenxing Hu	Shuran Liu	Yujun Sun
Hao Ding	Liang Hua	Tianwei Liu	Yunpeng Tai
Rui Ding	Jinbin Huang	Weizhou Liu	Chuning Tang
Shaofeng Ding	Ke Huang	Yangyang Liu	Heyi Tang
Mengfan Dong	Ruiyuan Huang	Yanming Liu	Sirui Tang

Mengnan Dong	Siying Huang	Yibo Liu	Zecheng Tang
Yuhao Dong	Weixiao Huang	Yipeng Liu	Chaoran Tian
Yuxin Dong	Yan Huang	Zhengying Liu	Rongpeng Tian
Ang'ang Du	Zhengjie Huang	Zhiheng Liu	Yu Tian
Chenzhuang Du	Zhiqi Huang	Enzhe Lu	Wei Tu
Dikang Du	Yulong Hui	Haoyu Lu	Chensi Wang
Jusen Du	Chaobo Jia	Linqiang Lu	Chuang Wang
Yulun Du	Yutong Jiang	Tingzhan Lu	Chunjie Wang
Yu Fan	Zhejun Jiang	Zhiyuan Lu	Dinglu Wang
Jing Feng	Zuoyou Jiang	Aotian Luo	Feng Wang
Qiulin Feng	Wenyi Jin	G. Luo	Hailong Wang
Yichen Feng	Xinyi Jin	Junyu Luo	Haiming Wang
Kelin Fu	Yu Jing	Yifan Luo	Hao Wang
Qiang Fu	Huanjun Kong	B. Lyu	Hao Wang

白童童	福轩高	赖国坤	温州吕
白一帆	高宏城	李艾迪	毛韶光
鲍一平	景岳高方	程莉	袁枚
M. C.	佟高	李成远	新门
蔡建峰	高伟佳	冷力	倪明青
蔡新元	耿上亿	方莉	牛毅轩
沛州曹	解龚	关于李	潘思远
曹宇轩	工行	李浩阳	彭树军
翟伟柴	升高工	贾莉	张阳琪
Y. Charles	公贤臣	李俊雄	秦润阳
H.S. 谢	齐正国	李磊	秦泽超
陈冠多	顾一成	李天	秦泽宇
陈广宇	管树浩	李林安	邱怀全
陈冠 Zheng	郭海青	李维宏	邱建新
陈华荣	郭世琦	李 wentao	邱洁中
贾晨	向国	李新通	邱博文
陈建龙	郭正言	杨莉	屈耀华
陈俊	贝西好	李一辰	尚泽宇
陈愚	文心浩	李毅伟	邵宝
彭晨	夏若涵	李雨晓	韩深
陈睿杰	何大兰	李昭伟	晋城市
陈文韬	何浩天	李晓曦	石俊峰
陈欣	何乐汉	李哲明	立冬时节
杨晨	齐和	李政晓	石元帅
陈妍如	何伟然	李志远	咏春狮

陈毅飞	何润东	林嘉伟	宋鹏威
陈英江	何新毅	林晓涵	宋晓曦
陈元坤	何毅博	林毅博	苏建林
陈宇杰	何云家	林子超	苏云峰
陈宇天	赵红	林子妍	苏卓辰
陈志强	洪天歌	刘斌	林岁
常志成	华浩	刘宝霄	孙景松
陈燕	胡嘉熙	刘川	孙俊耀
崔佳磊	胡瑞坤	梁留	孙少宁
崔景兵	胡伟明	刘少伟	孙树泽
戴安琪	胡阳阳	刘树东	孙童宇
邓佳琪	胡真行	刘舒然	孙宇君
郝丁	梁华	刘天伟	台胤鹏
丁瑞	黄金彬	刘维洲	汤春宁
丁少峰	黄珂	刘阳阳	贴心唐
东孟凡	黄瑞源	刘燕明	唐思睿
东孟喃	黄诗莹	刘宇博	唐泽城
董宇浩	黄伟潇	刘毅鹏	田朝然
董宇欣	炎黄	刘 Zhengying	田鹏荣
安安杜	黄正杰	刘智恒	宇天
陈庄杜	黄志强	卢恩泽	魏途
杜迪康	彝族	卢浩宇	王晨曦
Jusen Du	贾宝波	陆林强	王创
杜雨伦	江宇通	停战路	王春杰
于帆	姜哲俊	朱元路	王丁路
景峰	左优江	罗天亮	王峰
冯秋林	金文义	G. Luo	王海龙
冯一辰	金毅	罗俊宇	王海明
傅乐林	于静	罗凡	王浩
羌福	孔焕军	B. 路	王浩

Huaqing Wang

Hui Wang

Jiayi Wang

Jinglong Wang

Jinhong Wang

Jiuzheng Wang

Linian Wang

Shaobo Wang

Shenzhi Wang

Shuyi Wang
Si Wang
Siyuan Wang
Tianfu Wang
Wenjue Wang
Xingran Wang
Xinmei Wang
Xinyuan Wang
Xusheng Wang
Yalin Wang
Yangkun Wang
Yao Wang
Yaoyu Wang
Yejie Wang
Yiqin Wang
Yucheng Wang
Yuzhi Wang
Zhaoji Wang
Zhaowei Wang
Zhengtao Wang
Zenhao Wang
Zhongsheng Wang
Zifan Wang
Chu Wei
Ming Wei
Shouxin Wei
Zichen Wen
Fan Wu
Haoning Wu
Rucong Wu
Wenhao Wu
Xiaoxue Wu
Yingcong Wu
Yongqi Wu
Yuxin Wu
Zijian Wu
Xinglang Xian

王华清

王慧

王佳怡
王靖龙
王金红
王九正
王林 ian
王少波
王振之
王树义
西王
王思远
天府王
王文杰
王兴然
王新美
王新元
王旭生
王亚林
王岩坤
姚王
王耀宇
王杰
王毅琴
王雨成
王玉枝
王兆基
王兆伟
王正涛
王振浩
王仲生
王子凡
朱伟
明伟
薛新伟
文子辰
范武
吴乃宁
吴荣聪
吴文浩
吴小雪
吴应聪
吴永奇

吴雨欣

吴子健

兴郎县

Chenxuan Xiang

Yuye Xiang

Bocheng Xiao

Chenjun Xiao

Xin Xiao

Jin Xie

Xiaotong Xie

Yifeng Xie

Zhe Xie

Bowei Xing

Yiming Xiong

Baosheng Xu

Boyu Xu

Jiale Xu

Jianfan Xu

Jing Xu

Jinjing Xu

L.H. Xu

Qingtao Xu

Shuyao Xu

Suting Xu

Tiantian Xu

Tianxiang Xu

Weixin Xu

Xinran Xu

Yangchuan Xu

Ye Xu

Yueni Xu

Ziyao Xu

Haonan Xue

Junjie Yan

Yaoyao Yan

Fan Yang

Guangyao Yang

Hao Yang

Junwei Yang

Ruoyu Yang
Wenjie Yang
Xiaofei Yang
Xinyu Yang
Yi Yang
Yiling Yang
Ying Yang
Yuchen Yang
Zhen Yang
Zhilin Yang

陈玄香
玉叶香
肖波城
肖俊杰
小欣
金懈
谢晓彤
谢一峰
这些
邢宝伟
熊一鸣
徐宝生
徐博宇
徐佳乐
徐建藩
徐静
徐金晶
徐立华
徐青涛
徐舒瑶
徐穗青
徐甜甜
徐天翔
徐微信
徐欣然
徐阳川
叶旭
徐媛媛
徐子尧

薛浩南
严军杰
严瑶瑶
杨帆
杨广耀
郝阳
杨俊伟
杨润宇
杨文杰
杨小飞
杨欣宇
易烱千玺
杨一玲
阴阳
杨宇辰
郑阳
杨志林

Zian Yang
Zuhao Yang
Haotian Yao
Dan Ye
Haoran Ye
Wenjie Ye
Zhanbo Ye
Bohong Yin
Haoxiang Yin
Xietong Yin
Chengzhen Yu
Haozhen Yu
Longhui Yu
Shengnan Yu
Shuying Yu
Tianxiang Yu
Enming Yuan
Mengjie Yuan
Tongtian Yue
Wei Yue
Yang Yue
Dunyuan Zha

Haobing Zhan
B.H. Zhang
Dehao Zhang
Fei Zhang
Hao Zhang
Haoyuan Zhang
Huanyu Zhang
Jiapei Zhang
Jiaxuan Zhang
Jin Zhang
Kaiyi Zhang
Miaozhen Zhang
Puqi Zhang
Qinglei Zhang
Rong Zhang
Rui Zhang
Shaoshuai Zhang
Shiyi Zhang
Xiaobin Zhang
Xiaoyun Zhang
Y. Zhang
Yangkun Zhang
Ye Zhang
Yichi Zhang

杨子安
杨祖豪
姚浩天
丹叶
叶浩然
叶文杰
叶展博
Bohong Yin
好香饮
西通银
于成真
于浩真
俞龙辉
于胜南
余书盈

宇天翔
袁恩明
袁萌杰
通天月
魏岳
杨悦
敦煌札
赵炳章
张柏涵
张德浩
张飞
张浩
张浩源
张焕宇
张佳佩
张佳轩
张金
张凯义
张妙贞
张普奇
张庆雷
张荣
张瑞
张少帅
张世义
张小斌
张小云
张 Y.
张岩坤
叶章
张一驰

Yikun Zhang
Yizhi Zhang
Yongting Zhang
Yu Zhang
Yutao Zhang
Yutong Zhang
Zheng Zhang
Zijing Zhang

Bin Zhao
Chenguang Zhao
Feifan Zhao
Jinglun Zhao
Jinxiang Zhao
Shuai Zhao
Wenshuo Zhao
Xiangyu Zhao
Xuanle Zhao
Yikai Zhao
Zijia Zhao
Haozhi Zheng
Huabin Zheng
Ruihan Zheng
Shaojie Zheng
Tengyang Zheng
Haofeng Zhong
Lei Zhong
Longguang Zhong
M. Zhou
Qiankang Zhou
Runjie Zhou
Ruozhang Zhou
Xinyu Zhou
Yiqiao Zhou
Zaida Zhou
Jinguo Zhu
Liya Zhu
Xinhao Zhu
Yangjunfeng Zhu
Yuxuan Zhu
Zhen Zhu
Chen Zhuang
Weiyu Zhuang
Xinxing Zu
Kimi K3

张毅坤
张轶智
张永廷

余章
张宇涛
张宇通
张正
张子敬
赵斌
赵成光
赵非凡
赵晶伦
赵金香
赵帅
赵文硕
赵翔宇
赵宣乐
赵一凯
赵子佳
郑浩志
郑华斌
郑瑞涵
郑少杰
郑腾飞
钟浩峰
李忠
龙光钟
周先生
周乾康
周润杰
周若张
周新雨
周轶超
周泽达
朱金国
朱丽娅
朱欣浩
朱彦峰
朱雨轩
珍珠
陈庄
邹伟宇
新星组
Kimi K3

B Details of Sigmoid Tanh Unit GLU

B Sigmoid Tanh 单元 GLU 的细节

The design goal of SiTU-GLU (§2.3.2) is to bound the SwiGLU product without discarding the characteristic shape of Swish: an approximately linear response around the origin and a vanishing negative tail. Fig. 4 shows the gate and up branches together with their complete scalar responses.

SiTU-GLU (§2.3.2) 的设计目标是在不丢弃 Swish 的特征形状的前提下限制 SwiGLU 的输出：在原点附近具有近似线性的响应，以及消失的负尾部。图 4 显示了 gate 和 up 分支及其完整的标量响应。

Smoothly capping both branches SiTU caps the linear factor of Swish as $\beta_1 \tanh(\mathbf{W}_g x / \beta_1)$ while retaining the sigmoid factor [60]. Because the sigmoid already drives the negative gate response toward zero, this change primarily controls large positive activations without removing the negative tail. Kimi K3 applies the same construction to the up branch as $\beta_2 \tanh(\mathbf{W}_u x / \beta_2)$, preventing either branch from dominating the product.

平滑地对两个分支进行帽处理，SiTU 将 Swish 的线性因子设为 $\beta_1 \tanh(\mathbf{W}_g x / \beta_1)$ ，同时保留 sigmoid 因子 [60]。由于 sigmoid 已经将负门控响应驱动至零，这种变化主要控制大的正激活值，而不会去除负尾部。Kimi K3 将相同的构造应用于上分支，如 $\beta_2 \tanh(\mathbf{W}_u x / \beta_2)$ ，防止任一分支主导乘积。

Local and limiting behavior For a scalar z near the origin, the scaled tanh satisfies

局部和限制行为对于接近原点的标量 z ，缩放后的 tanh 满足

$$\beta \tanh\left(\frac{z}{\beta}\right) = z + O\left(\frac{z^3}{\beta^2}\right). \quad (18)$$

SiTU-GLU therefore matches SwiGLU to first order around the origin. It also recovers SwiGLU pointwise as $\beta_1, \beta_2 \rightarrow \infty$.

SiTU-GLU 因此在原点附近与 SwiGLU 高度匹配。它也以 $\beta_1, \beta_2 \rightarrow \infty$ 的方式恢复了 SwiGLU 的逐点特性。

Bounded output Since $|\tanh(z)| < 1$ and $0 < \text{Sigmoid}(z) < 1$, every output coordinate satisfies

有界输出自从 $|\tanh(z)| < 1$ 和 $0 < \text{Sigmoid}(z) < 1$ 起，每个输出坐标都满足

$$\|\text{SiTU} - \text{GLU}(x)\|_\infty \leq \beta_1 \beta_2 = 100, \quad (19)$$

for $\beta_1 = 4$ and $\beta_2 = 25$. Unlike hard clamping of gate pre-activations, the smooth cap preserves nonzero gradients away from saturation boundaries, which we find to give better training behavior.

对于 $\beta_1 = 4$ 和 $\beta_2 = 25$ 。与硬性钳制门激活值不同，平滑的 cap 在远离饱和边界的地方保持非零梯度，我们发现这能带来更好的训练效果。

C Derivation of Quantile Balancing

C 分位数平衡的推导

This appendix derives the Quantile Balancing (QB) updates used in §2.3 from optimal balanced assignment, following [111]; the assignment perspective on expert load balancing goes back to BASE Layers [67] and BIP [116]. Let $s \in \mathbb{R}^{m \times n}$ collect the router scores of m tokens over n experts, where each token selects exactly k experts and $x_{i,j} \in \{0, 1\}$ indicates whether token i is assigned to expert j . The maximum-score balanced assignment, in which each expert serves exactly mk/n tokens (assumed integral), is

本附录根据 [111] 中的最优平衡分配，推导出第 2.3 节中使用的分位数平衡 (QB) 更新方法；专家负载平衡的分配视角可以追溯到 BASE 层 [67] 和 BIP [116]。设 $s \in \mathbb{R}^{m \times n}$ 收集 m 个标记在 n 个专家上的路由器评分，其中每个标记恰好选择 k 个专家， $x_{i,j} \in \{0, 1\}$ 表示标记 i 是否被分配给专家 j 。在最大评分平衡分配中，每个专家恰好服务 mk/n 个标记（假设为整数），即

$$\max_{x_{i,j} \in \{0,1\}} \sum_{i,j} x_{i,j} s_{i,j} \quad \text{s.t.} \quad \sum_j x_{i,j} = k, \quad \sum_i x_{i,j} = \frac{mk}{n}. \quad (20)$$

Linear relaxation and duality Relaxing $x_{i,j} \in \{0, 1\}$ to $x_{i,j} \in [0, 1]$ turns Eq. 20 into a linear program, whose optimum is integral by the standard integrality of the bipartite b -matching polytope; the relaxation is therefore exact. Introducing free multipliers α_i and β_j for the token- and expert-side equality constraints, respectively, the relaxed problem can be written in max-min form as

线性松弛与对偶性将 $x_{i,j} \in \{0, 1\}$ 松弛到 $x_{i,j} \in [0, 1]$ 会使式子 20 转变为一个线性规划问题，其最优解由二分图 b -匹配多面体的标准整数性保证为整数；因此，该松弛是精确的。引入自由乘子 α_i 和 β_j 分别用于代币侧和专家侧的等式约束，松弛后的问题可以写成最大-最小形式如下：

$$\max_{x_{i,j} \in [0,1]} \min_{\alpha_i, \beta_j} \sum_{i,j} x_{i,j} s_{i,j} - \sum_i \alpha_i \left(\sum_j x_{i,j} - k \right) - \sum_j \beta_j \left(\sum_i x_{i,j} - \frac{mk}{n} \right). \quad (21)$$

The objective is linear in each of $\mathbf{x}$, α , and β , and the feasible sets are convex, so the minimax theorem allows exchanging the order of optimization:

目标函数在 $\mathbf{x}$ 、 α 和 β 中都是线性的，可行集是凸集，因此最小最大定理允许交换优化的顺序：

$$\min_{\alpha_i, \beta_j} \max_{x_{i,j} \in [0,1]} \sum_{i,j} x_{i,j} (s_{i,j} - \alpha_i - \beta_j) + k \sum_i \alpha_i + \frac{mk}{n} \sum_j \beta_j. \quad (22)$$

The inner maximum is separable over entries, with $x_{i,j}^* = 1$ if $s_{i,j} - \alpha_i - \beta_j > 0$ and $x_{i,j}^* = 0$ if $s_{i,j} - \alpha_i - \beta_j < 0$; the tie case has measure zero in practice. Substituting x^* gives the convex dual objective

内最大值在条目上是可分离的，如果 $s_{i,j} - \alpha_i - \beta_j > 0$ 则为 $x_{i,j}^* = 1$ ，如果 $s_{i,j} - \alpha_i - \beta_j < 0$ 则为 $x_{i,j}^* = 0$ ；在实际中，平局情况的测度为零。代入 x^* 得到凸对偶目标函数

$$\min_{\alpha_i, \beta_j} \mathcal{L}(\alpha, \beta) := \sum_{i,j} \max(0, s_{i,j} - \alpha_i - \beta_j) + k \sum_i \alpha_i + \frac{mk}{n} \sum_j \beta_j. \quad (23)$$

Algorithm 1: The alternating QB solver.

算法 1: 交替的 QB 求解器。

Input: score matrix $s \in \mathbb{R}^{m \times n}$ Output: assignment $x \in \{0, 1\}^{m \times n}$ Initialize $\beta = \mathbf{0}_{1 \times n}$; for $t = 1, 2, \dots, T$ do $\alpha \leftarrow \text{desc_sort}(s - \beta, \text{axis} = 1)_{[:,k:k+1]}$ $\beta \leftarrow \text{desc_sort}(s - \alpha, \text{axis} = 0)_{[mk/n:mk/n+1]}$ end return x with $x_{i,j} = 1$ if $j \in \text{argtop}_k(s_i - \beta)$, and 0 otherwise

输入: 分数矩阵 $s \in \mathbb{R}^{m \times n}$ 输出: 分配 $x \in \{0, 1\}^{m \times n}$ 初始化 $\beta = \mathbf{0}_{1 \times n}$; 对于 $t = 1, 2, \dots, T$ 做 $\alpha \leftarrow \text{desc_sort}(s - \beta, \text{axis} = 1)_{[:,k:k+1]}$ $\beta \leftarrow \text{desc_sort}(s - \alpha, \text{axis} = 0)_{[mk/n:mk/n+1]}$ 结束如果 $j \in \text{argtop}_k(s_i - \beta)$, 则返回 x 并带有 $x_{i,j} = 1$, 否则返回 0

Exact coordinate minimization We minimize Eq. 23 by alternately solving for α with β fixed and vice versa; each subproblem admits a closed-form exact solution. With β fixed, the problem decouples over tokens, and for token i we solve

精确坐标最小化我们通过交替地固定 β 来求解 α , 反之亦然; 每个子问题都有一个闭合形式的精确解。当 β 固定时, 问题在 token 上解耦, 对于 token i , 我们求解

$$\min_{\alpha} k\alpha + \sum_j \max(0, s_{i,j} - \beta_j - \alpha). \quad (24)$$

This objective is piecewise linear in α with slope k minus the number of margins $s_{i,j} - \beta_j$ exceeding α ; it is therefore minimized exactly when k margins lie above α , i.e., for any α_i^* between the k -th and $(k+1)$ -th largest entries of $s_i - \beta$. By convention we take the $(k+1)$ -th largest entry, which is equivalently the $(1 - k/n)$ -th quantile:

该目标函数在 α 上是分段线性的, 斜率为 k 减去超过 α 的边数 $s_{i,j} - \beta_j$; 因此, 当 k 条边位于 α 之上时, 该目标函数被最小化, 即对于任何 α_i^* 在 $s_i - \beta$ 的第 k 大项和第 $(k+1)$ 大项之间时。按照惯例, 我们取第 $(k+1)$ 大的项, 这等同于第 $(1 - k/n)$ 个百分位数:

$$\alpha_i^* = \text{quantile}_{1-k/n}(s_i - \beta). \quad (25)$$

Symmetrically, with α fixed, expert j solves $\min_{\beta} \frac{mk}{n}\beta + \sum_i \max(0, s_{i,j} - \alpha_i - \beta)$, whose minimizer is the $(mk/n + 1)$ -th largest entry of $s_{:,j} - \alpha$, again the $(1 - k/n)$ -th quantile:

对称地, 当 α 固定时, 专家 j 解决 $\min_{\beta} \frac{mk}{n}\beta + \sum_i \max(0, s_{i,j} - \alpha_i - \beta)$, 其最小化器是 $s_{:,j} - \alpha$ 的第 $(mk/n + 1)$ 大的元素, 再次是第 $(1 - k/n)$ 个百分位数:

$$\beta_j^* = \text{quantile}_{1-k/n}(s_{:,j} - \alpha). \quad (26)$$

Both updates are thus the same quantile along the token and expert axes, respectively, which gives the method its name. Fig. 5 illustrates the expert-side update as equalizing the accepted upper tail of each expert's margin distribution, and Alg. 1 summarizes the resulting alternating solver.

因此，这两个更新分别是在 token 和 expert 轴上的同一分位数，这也是该方法名称的由来。图 5 展示了专家侧的更新，即通过使每个专家的边际分布的接受上尾相等来实现。算法 1 总结了由此产生的交替求解器。

From assignment to routing At the optimum of Eq. 23, $x_{i,j}^* = 1$ if and only if $s_{i,j} - \alpha_i^* - \beta_j^* > 0$; combined with the token constraint $\sum_j x_{i,j}^* = k$, the selected experts are exactly the Top- k entries of $s_i - \beta^*$. Routing therefore requires only the expert thresholds $\beta \in \mathbb{R}^n$ (equivalently, the bias $b = -\beta$ of Eq. 13), while the token thresholds $\alpha \in \mathbb{R}^m$ are intermediate variables tied to the dynamic training batch and are discarded. This asymmetry preserves train-inference consistency: at deployment, routing is a fixed Top- k selection with a frozen bias, and no quantile computation is needed.

从分配到路由在式 23 的最优解中， $x_{i,j}^* = 1$ 当且仅当 $s_{i,j} - \alpha_i^* - \beta_j^* > 0$ ；结合令牌约束 $\sum_j x_{i,j}^* = k$ ，所选专家正好是 $s_i - \beta^*$ 的前 k 项。因此，路由只需要专家阈值 $\beta \in \mathbb{R}^n$ （等价于式 13 中的偏置 $b = -\beta$ ），而令牌阈值 $\alpha \in \mathbb{R}^m$ 是与动态训练批次相关的中间变量，并被丢弃。这种不对称性保持了训练与推理的一致性：在部署时，路由是一个固定的 Top- k 选择，偏置被冻结，无需进行分位数计算。

Relation to sign-based loss-free updates The expert-side subproblem underlying Eq. 26 has (sub)gradient

与基于符号的无损更新的关系式子 26 所涉及的专家端子问题具有（子）梯度

$$\frac{\partial \mathcal{L}}{\partial \beta_j} = \frac{mk}{n} - \sum_{i=1}^m \chi(s_{i,j} - \alpha_i - \beta_j > 0), \quad (27)$$

i.e., the target load minus the observed load of the expert j . A SignSGD step on this objective recovers the fixed-step sign update of auxiliary-loss-free balancing [30], up to the sign convention $b = -\beta$: the sign update retains only the direction of the load error in Eq. 27, whereas QB jumps directly to the exact coordinate minimizer of the same dual objective. This view explains both why QB requires no learning-rate-like hyperparameter and why it equilibrates within a few update steps even for nearly 10^3 experts. QB is likewise related to BIP [116], which solves the same assignment with inequality constraints $\sum_j x_{i,j} \leq k$ and $\sum_i x_{i,j} \leq mk/n$; the induced non-negativity constraints on α and β add a $\max(0, \cdot)$ clipping to both updates, which can only suppress over-selected experts without promoting under-selected ones, and markedly slows equilibration in our experiments. Finally, the resulting fixed-Top- k routing is related to expert-specific threshold routing but differs from Expert Threshold routing, which maintains EMA thresholds and permits a variable number of selected experts per token [112].

即，目标负载减去专家观察到的负载 j 。在该目标上执行一次 SignSGD 步骤，可以恢复无辅助损失平衡的固定步长符号更新 [30]，直到符号约定 $b = -\beta$ ：符号更新仅保留式 27 中负载误差的方向，而 QB 直接跳转到相同对偶目标的精确坐标最小值。这种观点解释了为什么 QB 不需要类似学习率的超参数，以及为什么即使对于几乎 10^3 专家，它也能在几步更新中达到平衡。QB 也与 BIP [116] 相关，后者解决相同的分配问题，带有不等式约束 $\sum_j x_{i,j} \leq k$ 和 $\sum_i x_{i,j} \leq mk/n$ ；对 α 和 β 引入的非负性约束为两个更新添加了一个 $\max(0, \cdot)$ 截断，这只能抑制过度选择的专家，而不会促进未被选择的专家，并且在我们的实验中显著减缓了平衡过程。最后，由此产生的固定 Top- k 路由与专家特定的阈值路由相关，但与专家阈值路由不同，后者维护 EMA 阈值，并允许每个 token 选择不同数量的专家 [112]。

D Histogram-Based Quantile Estimation

D 基于直方图的分位数估计

The QB update of Eq. 14 asks for a quantile taken over the whole training step: for each of the n experts, the $(1 - k/n)$ -th quantile of the margins $s_{i,j} - \alpha_i$, where the token count m spans

millions of tokens sharded across data-parallel ranks

式 14 的 QB 更新要求对整个训练步骤取一个分位数：对于每个 n 专家，取边距 $s_{i,j} - \alpha_i$ 的 $(1 - k/n)$ 分位数，其中令牌数量 m 覆盖了数百万个分布在数据并行等级中的令牌

and gradient-accumulation steps. Gathering $O(mn)$ margins for an exact quantile is impractical inside the training loop. The key observation is that the update never needs the margins themselves, only their per-expert distribution, which a histogram summarizes at fixed cost. Kimi K3 therefore maintains a binned histogram per expert and reads the quantile from it. Concretely, we histogram the required bias $r_{i,j} := \alpha_i - s_{i,j}$, the bias that would place expert j exactly at token i 's cutoff; negating the margins reverses their order, so the QB target $\hat{b}_j$ of Eq. 14 is exactly the (k/n) -quantile of $r_{:,j}$.

以及梯度累积步骤。在训练循环中收集 $O(mn)$ 个边距以精确计算分位数是不现实的。关键的观察是，更新过程实际上并不需要边距本身，而只需要每个专家的分布情况，而直方图可以在固定成本下总结这些分布。因此，Kimi K3 为每个专家维护一个分箱直方图，并从其中读取分位数。具体来说，我们对所需的偏差 $r_{i,j} := \alpha_i - s_{i,j}$ 进行直方图统计，该偏差会使专家 j 恰好位于 token i 的截止点；否定边距会反转它们的顺序，因此 Eq. 14 中的 QB 目标 $\hat{b}_j$ 恰好是 (k/n) -分位数的 $r_{:,j}$ 。

Binning range The first question is which interval to bin over, and here the required bias helps: its range is bounded by the current bias itself. Router scores are sigmoid outputs, so $s_{i,j} \in (0, 1)$, and the cutoff α_i is itself the biased score $s_{i,j'} + b_{j'}$ of some expert j' , so it lies in $(b_{\min}, 1 + b_{\max})$, with $b_{\min}$ and $b_{\max}$ the extremes of the current bias. Every $r_{i,j}$ therefore falls in $[b_{\min} - 1, b_{\max} + 1]$. We partition this interval into B uniform bins, which we find sufficient in practice, and recompute the range every step, so the bin width $w = (b_{\max} - b_{\min} + 2)/B$ stays adapted to the bias as it spreads to correct imbalance.

分箱范围第一个问题是在哪个区间进行分箱，这里所需的偏倚有助于确定：它的范围由当前的偏倚本身所限定。路由器得分是 sigmoid 输出，因此 $s_{i,j} \in (0, 1)$ ，并且截止值 α_i 本身就是某个专家 j' 的有偏得分 $s_{i,j'} + b_{j'}$ ，因此它位于 $(b_{\min}, 1 + b_{\max})$ 中，其中 $b_{\min}$ 和 $b_{\max}$ 是当前偏倚的极端值。因此，每个 $r_{i,j}$ 都位于 $[b_{\min} - 1, b_{\max} + 1]$ 中。我们将这个区间划分为 B 个均匀的分箱，我们在实践中发现这已经足够，并且在每一步重新计算范围，因此分箱宽度 $w = (b_{\max} - b_{\min} + 2)/B$ 会随着偏倚的扩展而适应性调整以纠正不平衡。

Accumulation and recovery The rest of the procedure follows the structure of a training step. During each forward pass, every rank scatter-adds its local $r_{i,j}$ values into a per-expert count matrix $\mathbf{H} \in \mathbb{N}^{n \times B}$, accumulating over all micro-batches with no communication. At the end of the step, a single all-reduce sums the local counts into the global histogram, and every rank recovers the quantile from the same pooled counts. Each expert's histogram counts every token once, so the target rank is exactly the target load $q = mk/n$ of § 2.3.3, now taken over the full step: we select the first bin whose cumulative count reaches $\lceil q \rceil$ and interpolate linearly within it. If bin β_j is selected, with cumulative count c_j before it and h_j counts inside it, then

积累和恢复其余的步骤遵循训练步骤的结构。在每次前向传递过程中，每个 rank 将本地的 $r_{i,j}$ 值散播并相加到每个专家的计数矩阵 $\mathbf{H} \in \mathbb{N}^{n \times B}$ 中，在所有微批次上进行累积，无需通信。在步骤结束时，一个全局的 all-reduce 操作将本地计数汇总到全局直方图中，并且每个 rank 从相同的汇总计数中恢复分位数。每个专家的直方图对每个 token 进行一次计数，因此目标 rank 正好是 § 2.3.3 中所述的目标负载 $q = mk/n$ ，现在覆盖整个步骤：我们选择第一个累积计数达到 $\lceil q \rceil$ 的 bin，并在其内进行线性插值。如果选择的是 bin β_j ，在它之前的累积计数为 c_j ，其中包含 h_j 个计数，则

$$\hat{b}_j = b_{\min} - 1 + \left(\beta_j + \text{clip}\left(\frac{q - c_j}{h_j}, 0, 1\right) \right) w,$$

and the resulting biases are mean-centered as in Eq. 14.

并且由此产生的偏差是均值中心化的，如式 14 所示。

Properties Three properties make this estimator practical at scale. First, it is accurate: the cumulative counts are exact at bin edges, so the true quantile and its estimate lie in the same bin and the error is bounded by the bin width w ; with $B = 1000$ this is at most a few 10^{-3} , and we observe no measurable residual load imbalance. Second, it is cheap: the only communication is one integer all-reduce of nB values per layer per step, independent of m , which in our configuration is below 1% of the cost of exchanging the raw margins over a process group every micro-batch, the natural alternative. Third, it estimates the right quantity: because counts are additive, the global histogram is exactly invariant to how tokens are partitioned across ranks or accumulation steps, and the estimate is the quantile of the pooled global batch rather than an average of per-rank quantiles, which generally differs. As a further refinement, maintaining an exponential moving average of the estimated quantiles across steps reduces batch-to-batch sampling noise and can improve load balance still further.

三个特性使得该估计器在大规模应用中具有实用性。首先，它具有较高的准确性：累积计数在直方图的边缘是精确的，因此真实的分位数及其估计值位于同一个直方图区间内，误差被直方图区间宽度 w 所限制；当 $B = 1000$ 时，这种误差最多仅为几个 10^{-3} ，我们观察到没有可测量的残余负载不平衡。其次，它成本低廉：唯一的通信是每层每步对 nB 个整数进行一次全规约操作，与 m 无关，这在我们的配置中低于每次微批次在进程组内交换原始边距成本的 1%，这是自然的替代方案。第三，它估计的是正确的量：由于计数是可加的，全局直方图在如何将标记分配到不同等级或累积步骤上是完全不变的，而估计值是合并后的全局批次的分位数，而不是各个等级分位数的平均值，这通常会有差异。作为进一步的优化，维护估计分位数的指数移动平均值可以减少批次间的采样噪声，并进一步改善负载平衡。

E MoonEP General Upper Bound Proof

E MoonEP 一般上界证明

Let $m_r(P)$ denote the number of redundant experts placed on rank r under plan P . For a router output I , the planning objective is to minimize the maximum number of redundant experts on any rank, i.e., $M(I) = \min_P \max_r \{m_r(P)\}$. We prove that $M(I) \leq E/R$ always holds (Theorem 1) and that this bound is essentially tight: there exist router outputs for which $M = \lceil E(R-1)/R^2 \rceil \approx E/R$ (Theorem 2).

设 $m_r(P)$ 表示在计划 P 下，位于等级 r 的冗余专家数量。对于路由器输出 I ，规划目标是最小化任何等级上的冗余专家最大数量，即 $M(I) = \min_P \max_r \{m_r(P)\}$ 。我们证明了 $M(I) \leq E/R$ 总是成立（定理 1），并且这个界限基本上是紧致的：存在路由器输出使得 $M = \lceil E(R-1)/R^2 \rceil \approx E/R$ （定理 2）。

Proof of Theorem 1 (General Upper Bound) The goal is to prove that $M(I) \leq E/R$ holds for any router output I . Key lemma: there exists a plan P^* such that every EP rank receives exactly the same number of tokens ($S \times K$), and the remote tokens of each rank come from only one other EP rank. The construction is as follows: initially, every rank holds only local tokens, and ranks are classified as underloaded or overloaded accordingly. We repeatedly pick an underloaded rank and an overloaded rank, and migrate tokens from the overloaded rank to fill the underloaded rank exactly up to the balanced value $S \times K$; the overloaded rank may remain overloaded, become exactly balanced, or become underloaded, and is put back into the corresponding set. This is repeated until all ranks are perfectly balanced. Each fill makes one underloaded rank balanced and it never changes afterwards, so the process terminates after at most $R-1$ fills; meanwhile, each rank is filled at most once, so its remote tokens come from a single rank, which proves the lemma. Consequently, supposing all remote tokens of rank r come from rank s ; these tokens belong to at most E/R local experts on rank s , hence $m_r(P^*) \leq E/R$, and therefore

定理 1 的证明（一般上界）我们的目标是证明对于任何路由器输出 I ， $M(I) \leq E/R$ 成立。关键引理：

存在一个计划 P^* ，使得每个 EP 等级接收的标记数 ($S \times K$) 完全相同，且每个等级的远程标记仅来自另一个 EP 等级。构造如下：初始时，每个等级仅持有本地标记，并根据情况被分类为负载不足或负载过重。我们反复选择一个负载不足的等级和一个负载过重的等级，并从负载过重的等级迁移标记，直到负载不足的等级被填满至平衡值 $S \times K$ ；负载过重的等级可能继续保持负载过重，变为完全平衡，或变为负载不足，并被放回相应的集合中。这一过程重复直到所有等级完全平衡。每次填充会使一个负载不足的等级变为平衡，并且之后不再改变，因此该过程在最多 $R - 1$ 次填充后终止；同时，每个等级最多被填充一次，因此其远程标记来自单一等级，这证明了引理。因此，假设等级 r 的所有远程标记来自等级 s ；这些标记属于等级 s 上的最多 E/R 个本地专家，因此 $m_r(P^*) \leq E/R$ ，从而

$$M(I) = \min_P \max_r \{m_r(P)\} \leq \max_r \{m_r(P^*)\} \leq \frac{E}{R} \quad (28)$$

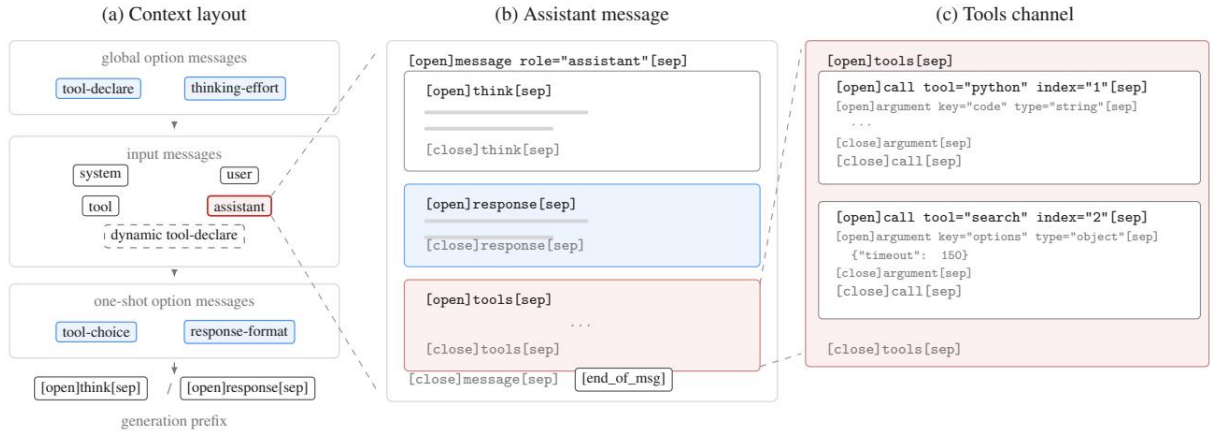


Figure 22: images/da0c2f2e4715eabe3a9002daed707e056b8d2e816804a17840557d85b5405019.jpg

Figure 16: Structure of the Kimi K3 chat template. (a) Context layout: global option messages precede the input messages, while one-shot option messages follow them, so that per-request options leave the history KV cache intact; dynamically loaded tools are injected mid-session as input option messages (dashed). (b) Anatomy of an assistant message: the body is organized into think, response, and tools channels. (c) Expansion of the tools channel: parallel tool calls are indexed so that tool results can be matched to their calls, and arguments are typed.

图 16: Kimi K3 聊天模板的结构。(a) 上下文布局：全局选项消息在输入消息之前，而一次性选项消息则在它们之后，这样每个请求的选项不会影响历史 KV 缓存；动态加载的工具作为输入选项消息在会话中被注入（虚线）。(b) 助手消息的结构：正文分为思考、响应和工具三个通道。(c) 工具通道的扩展：并行工具调用会被索引，以便将工具结果与相应的调用进行匹配，并且参数会被类型化。

Proof of Theorem 2 (Tightness of the Upper Bound) Construct a router output I^* as follows: the experts on EP rank 0 receive no tokens, while all experts on the other $R - 1$ ranks share all tokens evenly. Then all $S \times K \times R$ tokens are evenly divided among $E(R - 1)/R$ experts, so each expert receives $\frac{SKR^2}{E(R-1)}$ tokens. Under any plan P , rank 0 must receive $S \times K$ tokens, all of which are remote, and these tokens involve at least $SK / \frac{SKR^2}{E(R-1)} = \frac{E(R-1)}{R^2}$ distinct experts; taking the ceiling, rank 0 requires at least $\left\lceil \frac{E(R-1)}{R^2} \right\rceil$ redundant experts, hence $M(I^*) \geq \left\lceil \frac{E(R-1)}{R^2} \right\rceil$. Conversely, by constructing a plan with the filling procedure from the proof of Theorem 1 and migrating tokens expert-wise preferentially, the number of redundant experts on every rank can be kept within this value, so equality holds. Since $\left\lceil \frac{E(R-1)}{R^2} \right\rceil \approx \frac{E}{R}$ when R is large, the upper bound in Theorem 1 is essentially tight: there is no general upper bound significantly smaller than E/R .

定理 2 的证明（上界紧致性）构造一个路由器输出 I^* 如下：在 EP 等级 0 上的专家接收不到任何 token，而其他所有 $R - 1$ 等级上的专家平均分配所有 token。然后所有 $S \times K \times R$ 个 token 被平均分配给 $E(R - 1)/R$ 个专家，因此每个专家接收 $\frac{SKR^2}{E(R-1)}$ 个 token。在任何计划 P 中，等级 0 必须接收 $S \times K$ 个 token，这些 token 全部来自远程，且这些 token 涉及至少 $SK / \frac{SKR^2}{E(R-1)} = \frac{E(R-1)}{R^2}$ 个不同的专家；取上界，等级 0 至少需要 $\lceil \frac{E(R-1)}{R^2} \rceil$ 个冗余专家，因此 $M(I^*) \geq \lceil \frac{E(R-1)}{R^2} \rceil$ 。反过来，通过构造一个计划，使用定理 1 证明中的填充过程，并优先迁移 token 到专家，可以在每个等级上保持冗余专家数量不超过这个值，因此等号成立。由于 $\lceil \frac{E(R-1)}{R^2} \rceil \approx \frac{E}{R}$ 当 R 较大时，定理 1 中的上界本质上是紧致的：不存在显著小于 E/R 的普遍上界。

F Chat Template

F Chat 模板

The Kimi K3 chat template is redesigned around three goals. The first is extensibility: new capabilities should be introduced through backward-compatible message formats rather than template revisions, so that a single template serves the entire model generation. The second is a low alignment tax: the format should be learnable with minimal supervised data, supporting a pipeline in which a lightly fine-tuned pre-trained model can proceed directly to reinforcement learning. The third is decoding friendliness: the structure should admit simple encoders, streaming parsers, and grammar-constrained enforcers. To these ends, the template adopts HTML (eXtensible Token Markup Language), an XML-like markup in which the angle-bracket syntax is replaced by three reserved special tokens: [open], [sep] and [close], with an additional [end_of_msg] token as the generation stop marker. An element [open]tag attr="value"[sep] ... [close]tag[sep] is isomorphic to its XML counterpart, but every structural boundary is an explicit special token, which removes tokenization ambiguity at element boundaries and simplifies constrained decoding.

Kimi K3 聊天模板围绕三个目标进行了重新设计。第一个目标是可扩展性：新能力应通过向后兼容的消息格式引入，而不是通过模板修订，这样单一的模板就可以服务于整个模型生成。第二个目标是低对齐成本：格式应能够使用最少的监督数据进行学习，支持一个流程，即轻度微调的预训练模型可以直接进入强化学习。第三个目标是解码友好性：结构应允许简单的编码器、流式解析器和语法约束执行器。为实现这些目标，该模板采用了 HTML（可扩展标记语言），这是一种类似于 XML 的标记语言，其中尖括号语法被三个保留的特殊标记 [open]、[sep] 和 [close] 所取代，此外还有一个 [end_of_msg] 标记作为生成停止标记。一个元素 [open]tag attr="value"[sep] ... [close]tag[sep] 与其 XML 对应项是同构的，但每个结构边界都是一个显式的特殊标记，这消除了元素边界处的分词歧义，并简化了受限解码。

Messages and zones The top-level unit of the context is the message, and messages fall into two categories by origin (Fig. 16a). Input messages serialize the messages field of the request, covering the familiar system, user, assistant, and tool roles. Option messages translate request options into instructions that the model reads in context, and their placement reflects their scope. Global options—the tool declaration (type="tool-declare") and the reasoning-effort setting—appear before all input messages: they govern the whole session and rarely change, so modifying them invalidates the KV cache anyway. One-shot options (tool_choice, response_format) are appended after the input messages, so that per-request changes leave the history KV cache intact. A third kind, the input option message, is interleaved with input messages to supplement or override a global option mid-session. This mechanism

消息和区域上下文的顶级单位是消息，消息根据来源分为两类（图 16a）。输入消息序列化请求中的消息字段，涵盖了熟悉的系统、用户、助手和工具角色。选项消息将请求选项转换为模型在上下文中读取的指令，其位置反映了其作用范围。全局选项——工具声明（类型="tool-declare"）和推理努力设置——出现在所有输入消息之前：它们控制整个会话，很少改变，因此修改它们无论如何都会使 KV 缓存失效。一次性选项 (tool_choice、response_format) 在输入消息之后附加，这样每次请求的更改不会影响历史 KV 缓存。第三种类型，输入选项消息，与输入消息交错，以在会话中补充或覆盖全局选项。这种机制

supports dynamically loaded tools: tools retrieved or loaded during a conversation are announced through an additional tool-declare message, after which the model's available toolset

expands without rebuilding the preceding context.

支持动态加载工具：在对话过程中检索或加载的工具会通过额外的工具声明消息进行公告，之后模型的可用工具集将在不重建先前上下文的情况下扩展。

Channels The body of an assistant message is organized into channels, a concept inspired by OpenAI's Harmony response format [85]: think carries the reasoning trace, response the user-visible answer, and tools the tool calls (Fig. 16b). The two generation modes are selected purely through the generation prefix—[open]think[sep] for thinking mode and [open]response[sep] for instruct mode—rather than through separate templates. Kimi K3 supports only preserved thinking: in thinking mode, the think channel is always retained in the history—kept even when its content is empty—so that the model observes a consistent message structure across turns; in instruct mode, historical messages contain only the response and tools channels.

通道

助手消息的主体被组织成多个通道，这一概念灵感来源于 OpenAI 的 Harmony 响应格式 [85]：think 通道承载推理轨迹，response 通道包含用户可见的答案，tools 通道包含工具调用（图 16b）。两种生成模式是通过生成前缀纯属选择的——[open]think[sep] 表示思考模式，[open]response[sep] 表示指令模式，而不是通过单独的模板。Kimi K3 仅支持保留思考：在思考模式下，think 通道始终保留在历史记录中——即使其内容为空也保留——这样模型可以在多次交互中观察到一致的消息结构；在指令模式下，历史消息中仅包含 response 和 tools 通道。

Tool calling Within the tools channel, each call carries tool and index attributes; the index numbers parallel calls within a message, and each tool-result message repeats the same tool/index pair and follows the order of its call, so that results are unambiguously associated with calls. Arguments are typed: string arguments appear as raw text, while values of other JSON types are compactly serialized. Free-form text such as code is therefore a first-class citizen rather than an escaped JSON string. A pure-JSON fallback block covers inputs whose arguments cannot be decomposed into typed argument blocks; it occurs only in input tokens, never in model outputs, and its loss is masked during training.

工具调用在工具频道中，每个调用都包含工具和索引属性；索引编号与消息中的调用顺序相对应，每个工具结果消息重复相同的工具/索引对，并按照调用顺序排列，从而使结果与调用无歧义地关联。参数具有类型：字符串参数以原始文本形式出现，而其他 JSON 类型的值则以紧凑的方式进行序列化。因此，自由格式的文本（如代码）作为一等公民存在，而不是转义的 JSON 字符串。纯 JSON 降级块用于处理那些参数无法分解为类型化参数块的输入；它只出现在输入标记中，从不出现在模型输出中，并且在训练过程中其丢失会被掩盖。

Reasoning effort and options Reasoning effort is exposed as a global option message of type thinking-effort, inserted after the tool declaration and before the input messages. Instead of modifying the generation prefix or exposing a token budget, the message states the requested level in natural language and acts as a generation-constraint instruction. The schema reserves four levels (low, medium, high, and max), of which Kimi K3 supports a subset. This representation decouples the effort interface from the template syntax, and it aligns directly with the effort-conditioned training described in §4.1.1 and §4.1.2.

推理努力和选项推理努力作为全局选项消息以类型 thinking-effort 的形式暴露出来，插入在工具声明之后、输入消息之前。与其修改生成前缀或暴露令牌预算，该消息以自然语言说明请求的级别，并作为生成约束指令。该模式保留了四个级别（低、中、高和最大），其中 Kimi K3 支持其中一部分。这种表示方法将努力接口与模板语法解耦，并直接与第 4.1.1 节和第 4.1.2 节中描述的基于努力的训练相一致。

More broadly, this is the common implementation of all option messages: tool_choice, response_format, and thinking_effort are each translated into a short natural-language instruction placed in context, rather than into dedicated special syntax. Because the pre-trained model already follows such instructions well, new options can be introduced with little or no additional training—a direct embodiment of the low-alignment-tax design principle stated above.

更广泛地说，这是所有选项消息的常见实现方式：tool_choice、response_format 和 thinking_effort 等每个选项都被翻译成一条简短的自然语言指令，并放置在上下文中，而不是使用专门的特殊语法。由于预

训练模型已经能够很好地遵循此类指令，因此可以几乎无需额外训练就引入新的选项——这直接体现了上述低对齐成本设计原则。